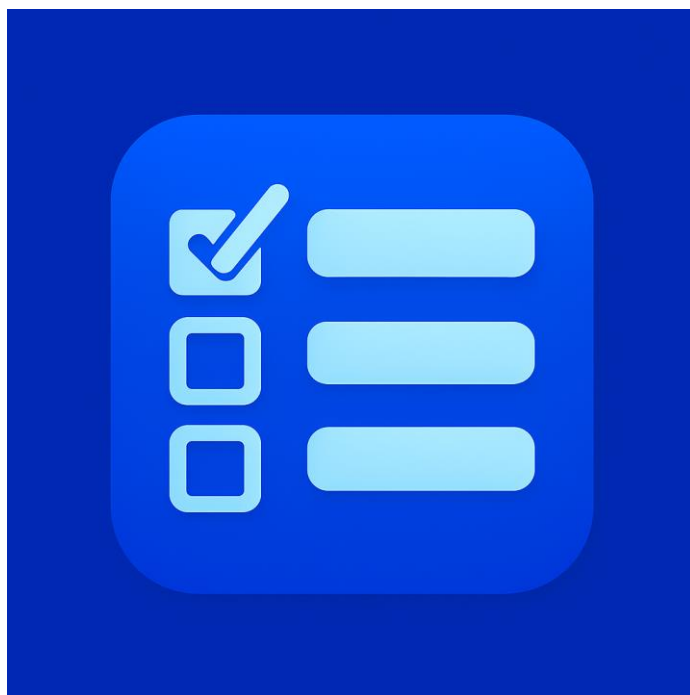


Gestor de Tareas

Documentación técnica del proyecto

Proyecto final de Desarrollo de Aplicaciones
Multiplataforma



Miguel Guerrero Murillo

<https://gestortareas-v2.vercel.app/app>

<https://github.com/TBBismuth/gestor-tareas.git>

Índice

Índice-----	2
1. Introducción-----	3
1.1. Contexto y problema que resuelve la app-----	4
1.2. Objetivos del proyecto -----	4
1.3. Alcance de la versión actual -----	5
2. Marco teórico / Estado del arte-----	7
2.1. Gestión de tareas y productividad (visión general)-----	8
2.2. Arquitectura web, API REST y JWT (conceptos básicos) -----	8
2.3. Tecnologías usadas y motivos de elección -----	9
3. Especificación de requisitos -----	11
3.1. Requisitos funcionales-----	11
3.1.1. Requisitos funcionales implementados en la versión actual-----	11
3.1.2. Requisitos funcionales previstos para versiones futuras-----	17
3.2. Requisitos no funcionales-----	19
3.2.1. Requisitos no funcionales implementados -----	19
3.2.2. Requisitos no funcionales previstos para versiones futuras-----	21
3.3. Casos de uso / Historias de usuario-----	22
3.3.1. Casos de uso implementados -----	23
3.3.2. Casos de uso previstos para versiones futuras-----	36
4. Diseño -----	38
4.1. Arquitectura general del sistema -----	38
4.1.1. Componentes principales -----	38
4.1.2. Flujo de comunicación-----	39
4.1.3. Arquitectura lógica del backend -----	41
4.1.4. Entornos de ejecución -----	42
4.2. Modelo de datos-----	43
4.2.1. Diagrama entidad-relación simplificado -----	45
4.2.2. Descripción de entidades -----	46
4.3. Diseño de la API -----	53

4.3.1. Recursos principales y organización de rutas -----	54
4.3.2. Autenticación, autorización y seguridad -----	56
4.3.3. Formato de datos y manejo de errores -----	57
4.3.4. Documentación y pruebas de la API -----	59
4.4. Diseño de la interfaz de usuario-----	60
5. Implementación -----	61
5.1. Backend (Spring Boot)-----	62
5.1.1. Estructura de paquetes-----	62
5.1.2. Servicios principales -----	65
5.1.3. Gestión de errores y validaciones -----	69
5.1.4. Manejo de categorías protegidas y otras lógicas especiales -----	72
5.2. Frontend web (React / PWA) -----	75
5.3. Experiencia móvil y PWA responsive -----	78
5.4. Seguridad -----	79
5.4.1. Arquitectura de seguridad-----	79
5.4.2. Autenticación con JWT-----	80
5.4.3. Autorización y restricciones de acceso -----	81
5.4.4. Filtro JWT y manejo de errores de seguridad -----	83
5.5. Otros aspectos relevantes -----	84
5.5.1. Lógica de ordenación de tareas-----	84
5.5.2. Lógica de filtrado de tareas -----	98
5.5.3. Gestión de las tareas del día-----	99
5.5.4. Cálculo del estado de una tarea y coherencia de fechas-----	100
6. Pruebas-----	101
6.1. Pruebas del backend -----	102
6.2. Pruebas de la API-----	104
6.3. Pruebas del frontend (web y móvil)-----	105
6.4. Resultados de las pruebas -----	107
7. Despliegue-----	109
7.1. Entorno de desarrollo local -----	109
7.2. Despliegue en la nube-----	110
7.3. Variables de entorno y configuración -----	112

8. Manual de usuario-----	113
8.1. Primeros pasos -----	114
8.2. Gestión de tareas -----	114
8.3. Categorías y filtros -----	115
8.4. Uso en móvil / PWA -----	116
9. Conclusiones y trabajo futuro -----	116
9.1. Funcionalidades futuras -----	118
10. Anexos -----	122
10.1. Documentación interactiva de la API -----	122
10.2. Listado completo de endpoints -----	123
10.3. Inventario de errores y respuestas DTO -----	125
10.4. Esquema físico de la base de datos -----	126

1. Introducción

1.1. Contexto y problema que resuelve la app

La gestión de tareas personales, académicas y profesionales se ha convertido en una actividad constante en el día a día. Estudiantes y trabajadores deben recordar plazos, trabajos pendientes, citas, tareas domésticas y otro tipo de compromisos que, con frecuencia, acaban repartidos entre diferentes soportes: agendas en papel, notas en el móvil, recordatorios aislados o aplicaciones genéricas que no se adaptan del todo a sus necesidades.

Esta dispersión de la información provoca varios problemas habituales: dificultad para tener una visión global de todo lo que hay que hacer, falta de claridad a la hora de priorizar, sensación de desorden y, en muchos casos, olvido de tareas importantes. Cuando además se combinan estudios, proyectos personales y otras responsabilidades, estos problemas se acentúan y la organización del tiempo se vuelve todavía más compleja.

En el contexto del ciclo de Desarrollo de Aplicaciones Multiplataforma, el autor de este proyecto se ha enfrentado precisamente a esta situación: gestionar simultáneamente asignaturas, prácticas, proyectos propios y tareas personales. A partir de esta necesidad real surge el proyecto **Gestor de Tareas**, cuyo propósito es ofrecer una aplicación que permita centralizar las tareas en un único lugar, agruparlas por categorías, destacar lo más relevante en cada momento y acceder a todo ello desde distintos dispositivos (ordenador y móvil), mejorando así la organización personal y reduciendo la sensación de caos.

1.2. Objetivos del proyecto

El objetivo principal de este proyecto es desarrollar una aplicación de gestión de tareas orientada a cualquier usuario que necesite organizar su día a día, centralizando en un único lugar sus tareas personales, académicas y profesionales. La aplicación nace a partir de una necesidad real del autor, pero se diseña con la intención de que pueda ser utilizada por cualquier persona que busque una herramienta sencilla y accesible para mejorar su organización.

A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Permitir el registro e inicio de sesión de usuarios, de manera que cada persona disponga de su propio espacio de trabajo y sus tareas queden aisladas del resto.
- Ofrecer un sistema de gestión de tareas que incluya creación, edición, completado y eliminación, asociado a categorías que representen los principales ámbitos de la vida diaria (trabajo/estudios, doméstico, salud, ocio/personal, etc.).
- Proporcionar vistas y filtros que ayuden a priorizar, como la posibilidad de centrarse en las tareas del día actual o de filtrar por categoría y estado.
- Diseñar y desarrollar un backend REST con Spring Boot, utilizando una arquitectura por capas, acceso a base de datos mediante JPA/Hibernate, mecanismos de validación y manejo de errores, y un sistema de autenticación basado en tokens JWT.
- Desarrollar interfaces cliente que consuman la API desde distintos dispositivos, comenzando por una aplicación web y sentando las bases para una experiencia específica en dispositivos móviles.
- Utilizar el proyecto como herramienta de aprendizaje práctico, profundizando en el desarrollo de APIs REST con Spring Boot, el diseño de modelos de datos relacionales, el consumo de servicios desde frontends modernos y el despliegue de aplicaciones en entornos reales.
- Dejar preparada la base del sistema para futuras funcionalidades avanzadas, como el envío de notificaciones, la generación de estadísticas de uso (tanto para el usuario como para el desarrollador), la gestión de grupos de usuarios (por ejemplo, grupos de clase con roles diferenciados) y la ordenación inteligente de tareas en función de su prioridad, fecha y tiempo estimado.

1.3. Alcance de la versión actual

La versión actual del proyecto Gestor de Tareas cuenta con un backend funcional desplegado en la nube, una base de datos relacional en producción y una interfaz web completa desarrollada como aplicación PWA. El sistema permite cubrir el flujo principal de uso de la aplicación: registro de usuarios, inicio de sesión, gestión de tareas personales, organización por categorías, trabajo colaborativo mediante grupos, filtrado avanzado, ordenación inteligente, notificaciones y uso desde escritorio o móvil.

El backend está desarrollado con Spring Boot y expone una API REST protegida mediante autenticación JWT. El sistema utiliza access token y refresh token, de forma que el usuario puede mantener la sesión activa sin tener que volver a iniciar sesión constantemente. La mayor parte de los endpoints requieren autenticación y el backend comprueba la propiedad de los recursos antes de permitir operaciones sobre tareas,

categorías, grupos, asignaciones o preferencias. La API está desplegada en Render y se conecta a una base de datos PostgreSQL alojada en Neon.

A nivel funcional, la aplicación permite crear, consultar, modificar, completar y eliminar tareas. Cada tarea puede tener título, descripción, prioridad, tiempo estimado, fecha de entrega y categoría. El sistema calcula automáticamente el estado de cada tarea según su fecha y si ha sido completada o no, distinguiendo casos como tareas en curso, vencidas, completadas, completadas con retraso o sin fecha. Además, se ofrecen vistas y consultas específicas para tareas del día, tareas próximas a vencer, tareas vencidas y tareas asignadas desde grupos.

La gestión de categorías también forma parte del alcance actual. Cada usuario dispone de sus propias categorías, incluidas categorías base protegidas creadas automáticamente al registrar la cuenta. Estas categorías pueden tener nombre, color e icono, y se utilizan tanto para organizar las tareas como para mejorar su representación visual en el frontend. El sistema impide eliminar categorías base protegidas y mantiene la integridad de las tareas cuando se elimina una categoría personalizada.

La versión actual incorpora además un sistema colaborativo basado en grupos. Los usuarios pueden crear grupos, unirse mediante código de invitación, consultar miembros, editar datos del grupo, gestionar roles, transferir la propiedad, salir de un grupo o eliminarlo cuando tienen permisos suficientes. Dentro de los grupos, los usuarios con permisos pueden crear asignaciones de tareas para todo el grupo o para una selección de miembros. Estas asignaciones generan tareas individuales para cada destinatario, lo que permite mantener el control personal de cada tarea sin perder la trazabilidad de su origen grupal. También se incluye el seguimiento de entregas, validación y reapertura de tareas asignadas.

Uno de los elementos principales de la versión actual es el filtro combinado de tareas. Este filtro permite consultar en una misma respuesta tareas personales y tareas con origen en grupos, aplicando criterios como origen, grupo, categoría, prioridad, estado, palabras clave, tiempo máximo, fecha exacta, fecha límite y criterio de ordenación. El estado del filtro puede guardarse en servidor por usuario, de modo que la aplicación puede recuperar posteriormente los últimos criterios utilizados. En el frontend, este sistema se presenta como un filtro avanzado integrado en la pantalla principal.

La aplicación también incluye una ordenación inteligente de tareas. Esta funcionalidad no se plantea como inteligencia artificial, sino como una heurística determinista y explicable que calcula una puntuación interna a partir de factores como fecha de entrega, prioridad, duración estimada, antigüedad, categoría y estado de revisión en tareas de grupo. El resultado permite mostrar al usuario una vista recomendada de

tareas, priorizando las que tienen más presión temporal o mayor importancia, sin devolver al cliente la puntuación interna calculada.

En la versión actual se ha implementado también un sistema de notificaciones PWA. El usuario dispone de una campanita interna donde puede consultar notificaciones activas, cerrarlas individualmente o cerrar todas. También puede configurar preferencias de notificación, activar avisos 24 horas antes del vencimiento, activar recordatorios inteligentes por tarea y recibir notificaciones asociadas a nuevas asignaciones de grupo. Además, el sistema permite registrar una suscripción Web Push desde el navegador para mostrar notificaciones del sistema cuando el navegador o la PWA lo permiten.

La interfaz principal está desarrollada con React y Vite, desplegada en Vercel y configurada como PWA instalable. El frontend consume la API REST mediante JWT, gestiona automáticamente la renovación del access token, muestra tareas, categorías, grupos, asignaciones, filtros, notificaciones y formularios de creación o edición. La aplicación se ha adaptado visualmente para escritorio y móvil, con tarjetas responsive, modales adaptados a pantallas pequeñas, panel de vistas flotante, filtro avanzado compacto, campanita en formato modal móvil y fondos optimizados para mejorar el rendimiento visual en listas de tareas.

El alcance actual incluye también el despliegue funcional del sistema completo. El frontend se publica en Vercel, el backend en Render y la base de datos en Neon. En producción se han configurado variables de entorno para JWT, base de datos, Web Push, zona horaria y self-ping del backend. El sistema dispone de un endpoint público de health check para comprobar la disponibilidad del backend y facilitar la estabilidad durante las pruebas o demostraciones.

Quedan fuera del alcance actual algunas funcionalidades previstas para futuras iteraciones, como la validación de cuentas por correo electrónico, captcha en el registro, integración con Google, sincronización con Google Calendar, estadísticas avanzadas, exportación e importación de tareas, paginación avanzada de listados y un modelo de tareas grupales compartidas reales sin duplicación individual. Estas funcionalidades se consideran ampliaciones futuras sobre una base ya funcional y desplegada.

2. Marco teórico / Estado del arte

2.1. Gestión de tareas y productividad (visión general)

La gestión de tareas se ha convertido en una actividad constante en la vida diaria de muchas personas, tanto en el ámbito personal como en el académico y profesional. A lo largo del día aparecen compromisos de distinta naturaleza: trabajos pendientes, llamadas que hacer, trámites administrativos, recados, citas o plazos de entrega. Cuando esta información se acumula en la cabeza o queda repartida entre diferentes sitios (notas rápidas, mensajes, correos, listas improvisadas, etc.), resulta fácil perder de vista qué hay que hacer y en qué momento.

Para afrontar este problema, cada usuario desarrolla su propio sistema de organización. Algunos utilizan métodos estructurados, como los propuestos en la literatura de productividad personal (por ejemplo, enfoques inspirados en “Getting Things Done” o en el uso sistemático de listas y revisiones periódicas), mientras que otros optan por soluciones más informales. En prácticamente todos los casos aparecen ideas comunes: **anotar las tareas fuera de la memoria**, asignarles alguna forma de prioridad, distinguir lo urgente de lo que puede esperar y agruparlas por áreas de la vida en las que se enmarcan.

Una de las motivaciones principales de estos sistemas es reducir la **carga mental** y el estrés asociados a tener demasiadas cosas pendientes sin un control claro. Contar con un lugar fiable donde registrar lo que hay que hacer permite liberar la mente y tomar decisiones más rápidas sobre en qué centrarse en cada momento. Las herramientas digitales modernas de gestión de tareas buscan precisamente ofrecer ese “punto único de referencia”: un entorno donde las tareas puedan registrarse, clasificarse, priorizarse y consultarse desde distintos dispositivos, evitando así la dispersión de información y facilitando una organización más eficiente del tiempo y de los recursos personales.

2.2. Arquitectura web, API REST y JWT (conceptos básicos)

La aplicación se basa en una arquitectura web de tipo **cliente-servidor**. En este modelo, uno o varios clientes (por ejemplo, la aplicación web ejecutándose en el navegador) envían peticiones a un servidor central, que es el encargado de procesarlas y de acceder a la base de datos cuando es necesario. El servidor expone una serie de funcionalidades a través de una API, mientras que los clientes se encargan de la parte de presentación y de la interacción con el usuario.

En este proyecto, el servidor está implementado como un **backend** desarrollado con Spring Boot que expone una **API REST**. Una API REST (Representational State Transfer) es un conjunto de endpoints accesibles mediante HTTP que permiten trabajar con recursos como usuarios, tareas o categorías utilizando los métodos estándar del protocolo (GET, POST, PUT, DELETE, etc.). El cliente no accede nunca directamente a la base de datos, sino que realiza peticiones a estas rutas (por ejemplo, a un recurso de tareas) y recibe como respuesta datos estructurados en formato **JSON**. Este enfoque facilita la separación entre la lógica de negocio en el servidor y las distintas interfaces cliente que consumen la API.

El acceso a determinadas operaciones de la API está protegido mediante un mecanismo de autenticación basado en **JSON Web Tokens (JWT)**. Cuando un usuario inicia sesión con sus credenciales, el servidor valida la información y, si es correcta, genera un token JWT que contiene los datos necesarios para identificar al usuario. A partir de ese momento, el cliente incluye este token en cada petición a los endpoints protegidos, en la cabecera `Authorization` con el formato `Bearer <token>`. El servidor verifica el token en cada solicitud y decide si permite o no el acceso. Este esquema permite trabajar con una arquitectura **sin estado (stateless)** en el servidor: no es necesario mantener sesiones de usuario en memoria, ya que la información relevante viaja incluida en el propio token. En el entorno de producción desplegado en la nube, toda la comunicación entre cliente y servidor se realiza a través de conexiones seguras **HTTPS**, de modo que las credenciales y los tokens se transmiten cifrados.

2.3. Tecnologías usadas y motivos de elección

El proyecto **Gestor de Tareas** se ha desarrollado utilizando un conjunto de tecnologías modernas orientadas al desarrollo web full stack. La elección de cada herramienta no solo responde a criterios técnicos, sino también a la trayectoria de aprendizaje del autor durante el ciclo de Desarrollo de Aplicaciones Multiplataforma: en muchos casos se han seleccionado tecnologías que ya conocía por asignaturas previas y, en otros, tecnologías que sabía que iba a necesitar en cursos posteriores y que le interesaba aprender de forma práctica.

En la capa de servidor se ha elegido **Java** junto con el framework **Spring Boot** como base del backend. Spring Boot permite crear aplicaciones web en Java de forma estructurada, integrando módulos como Spring Web para la exposición de endpoints HTTP, Spring Data JPA para el acceso a datos y Spring Security para la gestión de la autenticación y autorización. El acceso a la base de datos se realiza mediante **JPA/Hibernate**, utilizando entidades Java anotadas que se mapean a tablas relacionales. Este enfoque facilita trabajar con un modelo de objetos claro en el código y delegar en el framework la generación de las consultas necesarias. La elección de Spring Boot está alineada con los contenidos del ciclo formativo y con el objetivo personal del autor de profundizar en el desarrollo de APIs REST en Java.

Para el almacenamiento de la información se ha optado por un sistema de gestión de bases de datos relacional. El proyecto comenzó utilizando **MariaDB**, aprovechando la experiencia previa adquirida en la asignatura de Bases de Datos. Posteriormente, tanto el entorno de desarrollo como el de producción se migraron a **PostgreSQL**, con el objetivo de trabajar con un SGBD muy extendido en entornos profesionales y compatible con servicios gestionados en la nube. En la versión actual, la base de datos se ejecuta en **PostgreSQL** alojado en el servicio **Neon**, lo que permite disponer de una instancia gestionada y accesible desde el backend sin necesidad de configurar un servidor propio. El uso de una base de datos relacional facilita la definición de relaciones claras entre usuarios, tareas y categorías, y encaja con los contenidos vistos en el ciclo.

La interfaz principal de usuario se ha implementado como una **aplicación web de una sola página (SPA)** utilizando **React**, construida con la herramienta Vite. React se ha elegido por ser una de las bibliotecas más utilizadas actualmente para el desarrollo de interfaces web y por su presencia en numerosos recursos formativos, lo que convierte el proyecto en una oportunidad para aprender a consumir una API REST desde un frontend moderno. El diseño se apoya en **Tailwind CSS** para la gestión de estilos, lo que permite definir una base visual reutilizable y facilita futuras mejoras de la interfaz. La aplicación web está configurada para comportarse como **aplicación web progresiva**

(PWA), de modo que puede instalarse desde el propio navegador tanto en escritorio como en dispositivos móviles y utilizarse de forma similar a una aplicación nativa ligera.

En cuanto al despliegue, se han elegido servicios en la nube que se integran directamente con el repositorio del proyecto y que ofrecen planes gratuitos adecuados para un entorno académico. El backend de Spring Boot se encuentra desplegado en **Render**, utilizando contenedores Docker generados a partir del código fuente. El frontend en React se despliega en **Vercel**, que construye y publica automáticamente la aplicación a partir de los commits realizados en el repositorio. La base de datos PostgreSQL se aloja en **Neon**, proporcionando una instancia gestionada accesible desde el backend. Todo el código del proyecto se versiona mediante **Git** y se aloja en **GitHub**, lo que permite mantener un historial de cambios, organizar el trabajo por versiones y conectar el despliegue con el control de versiones.

La combinación de estas tecnologías permite construir una aplicación funcional y, al mismo tiempo, ofrece un entorno realista para el aprendizaje de conceptos clave: desarrollo de APIs REST con Spring Boot, trabajo con bases de datos relacionales, creación de interfaces web con React y gestión del despliegue en servicios cloud actuales.

3. Especificación de requisitos

3.1. Requisitos funcionales

3.1.1. Requisitos funcionales implementados en la versión actual

A continuación se describen los requisitos funcionales principales de la versión actual del sistema Gestor de Tareas, organizados por áreas de funcionamiento.

RF-01. Registro de usuarios

El sistema debe permitir el registro de nuevos usuarios, solicitando al menos nombre, correo electrónico y contraseña. El correo electrónico debe ser único en la base de datos y cumplir un formato válido.

RF-02. Inicio de sesión de usuarios

El sistema debe permitir que un usuario registrado inicie sesión mediante su correo electrónico y contraseña. Si las credenciales son correctas, el backend debe generar los tokens necesarios para la sesión autenticada.

RF-03. Refresh de sesión

El sistema debe permitir renovar el access token utilizando un refresh token válido, sin obligar al usuario a volver a introducir sus credenciales en cada caducidad del token de acceso.

RF-04. Acceso protegido a funcionalidades

Todas las operaciones protegidas de la API, salvo registro, login, refresh y documentación, deben requerir un token JWT válido en la cabecera Authorization con el formato Bearer <token>.

RF-05. Consulta de la cuenta autenticada

El sistema debe permitir que un usuario autenticado consulte su propia cuenta mediante un endpoint específico centrado en el usuario autenticado.

RF-06. Eliminación de la cuenta autenticada

El sistema debe permitir que un usuario autenticado elimine su propia cuenta, resolviendo previamente las dependencias necesarias para mantener la coherencia de los datos asociados.

RF-07. Aislamiento de datos por usuario

Cada usuario debe poder acceder únicamente a sus propias tareas, categorías y estados guardados de filtros. El sistema debe impedir el acceso, modificación o eliminación de recursos ajenos.

RF-08. Creación de tareas

El sistema debe permitir al usuario autenticado crear nuevas tareas, indicando al menos título, prioridad, tiempo estimado y una categoría válida, además de poder añadir descripción y fecha de entrega.

RF-09. Edición de tareas

El sistema debe permitir modificar los datos de una tarea existente del usuario autenticado, incluyendo título, descripción, categoría, prioridad, tiempo estimado o fecha de entrega.

RF-10. Marcado de tareas como completadas

El sistema debe permitir marcar una tarea como completada, registrando la fecha de completado y el usuario que realiza la acción.

RF-11. Eliminación de tareas

El sistema debe permitir eliminar tareas pertenecientes al usuario autenticado.

RF-12. Consulta individual y colectiva de tareas

El sistema debe permitir listar todas las tareas del usuario autenticado y consultar una tarea concreta por su identificador.

RF-13. Vista de tareas del día

El sistema debe ofrecer una vista específica para consultar las tareas cuya fecha de entrega coincide con la fecha actual.

RF-14. Vista de tareas próximas a vencer

El sistema debe ofrecer una vista específica para consultar tareas no completadas con fecha de entrega futura, ordenadas por proximidad de vencimiento.

RF-15. Vista de tareas vencidas

El sistema debe ofrecer una vista específica para consultar tareas no completadas cuya fecha de entrega ya ha pasado.

RF-16. Ordenación de tareas

El sistema debe permitir obtener las tareas ordenadas por distintos criterios, como título, prioridad, tiempo estimado o fecha de entrega.

RF-17. Filtrado individual de tareas

El sistema debe permitir filtrar tareas por prioridad, estado, categoría, tiempo estimado y palabras clave presentes en el título o la descripción.

RF-18. Cálculo automático del estado de las tareas

El sistema debe calcular automáticamente el estado lógico de cada tarea en función de su fecha de entrega, de si está completada o no, de la fecha de completado y de la fecha actual, distinguiendo entre estados como en curso, vencida, completada, completada con retraso o sin fecha.

RF-19. Gestión de categorías

El sistema debe permitir crear nuevas categorías, listar categorías, buscarlas por nombre parcial, modificarlas y eliminarlas cuando correspondan.

RF-20. Categorías base protegidas

El sistema debe crear automáticamente categorías base protegidas para cada usuario y debe impedir su eliminación, aplicando además restricciones sobre su modificación.

RF-21. Integridad de tareas al eliminar categorías

Cuando se elimina una categoría no protegida, las tareas que estaban asociadas a ella deben permanecer en el sistema y pasar a quedar sin categoría asignada.

RF-22. Personalización visual de categorías

El sistema debe permitir asociar a las categorías un color y un icono para mejorar su representación visual en el cliente.

RF-23. Creación y gestión de grupos

El sistema debe permitir crear grupos, consultarlos, modificarlos, activarlos o inactivarlos y eliminarlos, respetando las reglas de ownership y permisos dentro del grupo.

RF-24. Gestión de miembros y roles dentro de grupos

El sistema debe permitir añadir miembros a un grupo, expulsarlos, cambiar su rol, salir del grupo y transferir el ownership del grupo cuando corresponda.

RF-25. Invitación por código a grupos

El sistema debe permitir consultar el código de invitación de un grupo, regenerarlo y unirse a un grupo mediante un código válido.

RF-26. Asignación de tareas desde grupos

El sistema debe permitir que un creador o admin de grupo asigne tareas a miembros del grupo mediante duplicación individual, ya sea a todo el grupo o a una selección manual de destinatarios.

RF-27. Seguimiento de asignaciones de grupo

El sistema debe permitir listar asignaciones de grupo y consultar el detalle de una asignación concreta, incluyendo el estado individual de cada destinatario.

RF-28. Validación y reapertura de entregas de grupo

El sistema debe permitir a los admins de grupo validar entregas y reabrir las, registrando comentario de revisión y manteniendo la trazabilidad del estado de cada asignación individual.

RF-29. Consulta de tareas con origen en grupos

El sistema debe permitir que el usuario autenticado consulte específicamente las tareas que le han sido generadas desde asignaciones de grupo, tanto de forma general como filtradas por grupo concreto.

RF-30. Filtro combinado de tareas

El sistema debe permitir aplicar un filtro combinado sobre una lista unificada de tareas personales y tareas con origen en grupos, utilizando distintos criterios opcionales en una misma petición.

RF-31. Criterios avanzados del filtro combinado

El filtro combinado debe permitir trabajar, al menos, con los siguientes criterios: origen de la tarea, grupo concreto, categoría, prioridades múltiples, estados múltiples, palabras clave, tiempo máximo, fecha exacta, fecha hasta un día concreto y criterio explícito de ordenación.

RF-32. Exclusión de tareas completadas en el filtro combinado

El sistema debe permitir aplicar una opción de soloPorCompletar para excluir del resultado las tareas completadas y las completadas con retraso.

RF-33. Persistencia del estado del filtro combinado

El sistema debe permitir guardar por usuario el último estado utilizado del filtro combinado y recuperarlo posteriormente, devolviendo un estado por defecto funcional cuando aún no exista uno persistido.

RF-34. Respuesta enriquecida del filtro combinado

El sistema debe devolver, dentro del resultado del filtro combinado, tanto los datos base de la tarea como los datos de categoría, el origen de la tarea y la información de revisión grupal cuando aplique.

RF-35. Validación de datos de entrada

El sistema debe validar los datos recibidos en las operaciones principales de usuario, tareas, categorías, grupos, asignaciones y filtros, comprobando campos obligatorios, tamaños máximos, formatos y coherencia lógica entre valores.

RF-36. Gestión coherente de errores funcionales

El sistema debe responder con códigos HTTP adecuados y mensajes comprensibles cuando se produzcan errores de validación, autenticación, autorización, conflicto, recurso inexistente o solicitud incoherente.

RF-37. Documentación interactiva de la API

El sistema debe exponer documentación interactiva mediante Swagger/OpenAPI, de modo que las operaciones disponibles puedan consultarse y probarse desde una interfaz web.

RF-38. Ordenación inteligente de tareas

El sistema debe permitir obtener una lista recomendada de tareas mediante un criterio de ordenación inteligente. Esta ordenación debe combinar factores como fecha de entrega, prioridad, tiempo estimado, antigüedad, categoría y estado de revisión grupal, sin exponer al cliente la puntuación interna utilizada para calcular el orden.

RF-39. Vista de tareas recomendadas

El sistema debe exponer una vista específica de tareas recomendadas, accesible desde el frontend, que reutilice el filtro combinado y la lógica de ordenación inteligente para mostrar al usuario una lista priorizada de tareas pendientes accionables.

RF-40. Filtro avanzado en la interfaz principal

El frontend debe permitir al usuario aplicar filtros avanzados desde la pantalla principal, combinando criterios como origen, grupo, categoría, prioridades, estados, fechas, palabras clave, tiempo máximo y ordenación. El resultado debe mostrarse como una vista propia dentro del dashboard, sin obligar al usuario a cambiar de pantalla.

RF-41. Guardado automático del estado del filtro avanzado

El frontend debe guardar automáticamente en servidor el estado del filtro avanzado cuando el usuario aplica o limpia filtros, de forma que los controles puedan restaurarse al volver a entrar en la aplicación o desde otro dispositivo.

RF-42. Bandeja interna de notificaciones

El sistema debe permitir al usuario consultar notificaciones activas desde una campanita interna, cerrar notificaciones individualmente y cerrar todas las notificaciones pendientes.

RF-43. Preferencias de notificación

El sistema debe permitir configurar preferencias de notificación por usuario, incluyendo la activación general de notificaciones, avisos 24 horas antes del vencimiento, prioridades incluidas en dichos avisos, grupos incluidos, recordatorio inteligente y avisos por asignaciones de grupo.

RF-44. Recordatorio inteligente por tarea

El sistema debe permitir activar o desactivar un recordatorio inteligente en tareas personales con fecha de entrega. La fecha del recordatorio debe calcularse automáticamente en función de la fecha de entrega y del tiempo estimado de la tarea.

RF-45. Notificaciones Web Push en navegador/PWA

El sistema debe permitir registrar y desactivar una suscripción Web Push del navegador o dispositivo del usuario, de forma que las notificaciones internas puedan mostrarse

también como notificaciones del sistema cuando el navegador, los permisos y la configuración lo permitan.

RF-46. Uso de la aplicación como PWA

El frontend debe poder instalarse como aplicación web progresiva y funcionar como una experiencia web instalable, tanto en escritorio como en dispositivos móviles compatibles.

RF-47. Interfaz responsive para móvil

La interfaz debe adaptarse a pantallas móviles, manteniendo usables las vistas principales de tareas, categorías, grupos, filtros avanzados, notificaciones, autenticación y modales principales.

3.1.2. Requisitos funcionales previstos para versiones futuras

Además de las funcionalidades descritas en el apartado anterior, el sistema Gestor de Tareas cumple una serie de requisitos no funcionales relacionados con seguridad, despliegue, mantenibilidad, usabilidad y experiencia de usuario.

RNF-01. Seguridad de acceso

El sistema garantiza que solo los usuarios autenticados puedan acceder a las operaciones protegidas de la API. Para ello, el backend utiliza un mecanismo de autenticación basado en JWT y comprueba en cada petición que el token incluido en la cabecera Authorization es válido antes de permitir el acceso.

RNF-02. Renovación de sesión mediante refresh token

El sistema permite renovar el access token mediante un refresh token válido. Esto mejora la experiencia de usuario, ya que evita tener que iniciar sesión constantemente, y separa el token usado para acceder a los recursos protegidos del token usado para renovar la sesión.

RNF-03. Aislamiento de datos entre usuarios

El sistema asegura que los datos de un usuario no sean visibles ni modificables por otros usuarios. Las operaciones sobre tareas, categorías, grupos, asignaciones, filtros guardados, preferencias y notificaciones comprueban la propiedad del recurso o los permisos del usuario autenticado antes de aplicar cambios.

RNF-04. Restricción de orígenes mediante CORS

La API está configurada para aceptar peticiones desde los orígenes autorizados, incluyendo el dominio de producción del frontend y el entorno local de desarrollo. Esto reduce el riesgo de consumo de la API desde aplicaciones web no autorizadas.

RNF-05. Comunicación segura en producción

En el entorno desplegado, la comunicación entre cliente y servidor se realiza mediante HTTPS. De esta forma, las credenciales, los tokens JWT y el resto de datos intercambiados entre frontend y backend viajan cifrados.

RNF-06. Disponibilidad en entorno cloud

La aplicación está desplegada en servicios cloud: frontend en Vercel, backend en Render y base de datos PostgreSQL en Neon. Esto permite acceder al sistema desde distintos dispositivos sin necesidad de instalar el backend ni la base de datos en local.

RNF-07. Health check y self-ping del backend

El backend dispone de un endpoint público de comprobación de disponibilidad y de un sistema opcional de self-ping. Esta configuración se utiliza para mejorar la estabilidad del servicio en Render durante pruebas y demostraciones, reduciendo el impacto del reposo automático propio de los planes gratuitos.

RNF-08. Coherencia de zona horaria

El entorno de producción está configurado con zona horaria Europe/Madrid mediante variables de entorno específicas. Esto evita desfases en fechas de creación, vencimientos, avisos 24 horas antes y recordatorios inteligentes.

RNF-09. Usabilidad de la interfaz

La interfaz web permite utilizar las funcionalidades principales de la aplicación de forma visual y comprensible: tareas, categorías, grupos, asignaciones, filtros, notificaciones, autenticación y modales de creación o edición. El diseño se ha organizado mediante componentes reutilizables y estilos centralizados.

RNF-10. Diseño responsive y experiencia móvil

La aplicación web está adaptada a pantallas de escritorio y móvil. Las tarjetas, modales, paneles, filtros avanzados, campanita de notificaciones y pantallas de autenticación se han ajustado para mantener una experiencia usable en dispositivos móviles.

RNF-11. Comportamiento PWA

El frontend está configurado como aplicación web progresiva. Puede instalarse desde navegadores compatibles y cuenta con service worker propio para gestionar el comportamiento PWA y la recepción de notificaciones Web Push.

RNF-12. Rendimiento visual en móvil

Se han optimizado los fondos repetidos de tarjetas en móvil sustituyendo fondos SVG por assets WebP rasterizados en tareas, categorías y grupos. Esta decisión reduce el coste de pintado en listas largas sin modificar el diseño de escritorio.

RNF-13. Mantenibilidad del código

El proyecto está organizado siguiendo una separación clara de responsabilidades. En el backend se utilizan capas de controladores, servicios, repositorios, modelos, DTOs, configuración, seguridad y excepciones. En el frontend se trabaja con componentes reutilizables, módulos por funcionalidad, API clients y estilos centralizados.

RNF-14. Manejo homogéneo de errores

El backend centraliza el tratamiento de excepciones mediante un manejador global, devolviendo respuestas HTTP coherentes y cuerpos de error homogéneos. Esto facilita el consumo de la API desde el frontend y simplifica la depuración.

RNF-15. Documentación de API

La API está documentada mediante Swagger/OpenAPI, lo que permite consultar endpoints, modelos de datos y probar operaciones desde una interfaz interactiva durante el desarrollo y la validación.

RNF-16. Control de versiones y trazabilidad

Todo el código del proyecto se versiona mediante Git y se aloja en GitHub. El historial de commits permite seguir la evolución del sistema, recuperar estados anteriores y conectar el flujo de desarrollo con los despliegues en la nube.

3.2. Requisitos no funcionales

3.2.1. Requisitos no funcionales implementados

Además de las funcionalidades descritas en el apartado anterior, el sistema **Gestor de Tareas** debe cumplir una serie de requisitos no funcionales que afectan a su calidad, seguridad y mantenimiento.

RNF-01. Seguridad de acceso

El sistema debe garantizar que solo los usuarios autenticados puedan acceder a las operaciones protegidas de la API. Para ello, el backend utiliza un mecanismo de autenticación basado en tokens JWT y comprueba en cada petición que el token incluido en la cabecera `Authorization` es válido antes de permitir el acceso.

RNF-02. Aislamiento de datos entre usuarios

El sistema debe asegurar que los datos de un usuario no sean visibles ni modificables por otros usuarios. En las operaciones que actúan sobre tareas y categorías se debe verificar que el usuario autenticado coincide con el propietario de los datos sobre los que se quiere operar.

RNF-03. Restricción de orígenes (CORS)

La API debe estar configurada para aceptar peticiones únicamente desde los orígenes autorizados, en particular desde el dominio donde se despliega el frontend (Vercel). Esto impide que aplicaciones no autorizadas consuman la API directamente desde otros sitios web.

RNF-04. Comunicación segura

En el entorno desplegado en la nube, la comunicación entre el cliente y el servidor debe realizarse a través de conexiones HTTPS, de manera que las credenciales, los tokens JWT y el resto de los datos intercambiados viajen cifrados.

RNF-05. Rendimiento adecuado para uso individual

El sistema debe ofrecer tiempos de respuesta razonables para el volumen de datos esperado en un contexto de uso individual o de un número reducido de usuarios. Las operaciones típicas (listado de tareas, creación y actualización de tareas y categorías, autenticación) deben completarse sin demoras apreciables en condiciones normales de red.

RNF-06. Disponibilidad en entorno cloud con recursos limitados

La aplicación debe estar desplegada en servicios en la nube (Render para el backend, Vercel para el frontend y Neon para la base de datos), de modo que pueda accederse a ella sin necesidad de instalaciones locales. No obstante, dado que se utilizan planes gratuitos, se acepta la posibilidad de que el servicio entre en reposo tras un periodo de inactividad y que se produzcan tiempos de arranque inicial más lentos cuando se vuelve a acceder a la aplicación.

RNF-07. Usabilidad básica y sencillez de la interfaz

La interfaz web debe ser sencilla y centrarse en las operaciones principales de gestión de tareas, evitando complejidad innecesaria. Aunque el diseño visual y la adaptación a dispositivos móviles se consideran aún provisionales, la aplicación debe poder utilizarse de forma comprensible por un usuario sin necesidad de un manual extenso.

RNF-08. Portabilidad de la solución

La aplicación web debe poder ejecutarse en navegadores modernos compatibles con las tecnologías utilizadas (HTML5, JavaScript y React). El backend debe poder

desplegarse tanto en el entorno actual basado en contenedores en Render como en otros entornos compatibles con aplicaciones Spring Boot y bases de datos PostgreSQL, siempre que se configuren adecuadamente las variables de entorno y la conexión a la base de datos.

RNF-09. Mantenibilidad del código

El proyecto debe estar organizado de forma que facilite su mantenimiento y evolución. En el backend, el código se estructura en capas (controladores, servicios, repositorios y modelos) y hace uso de buenas prácticas como la separación de responsabilidades, el uso de DTOs cuando es necesario y un manejo centralizado de errores. Además, el uso de frameworks estándar como Spring Boot y JPA simplifica la incorporación de nuevas funcionalidades.

RNF-10. Control de versiones y trazabilidad

Todo el código del proyecto debe estar versionado mediante Git y alojado en un repositorio remoto (GitHub). Los cambios deben registrarse mediante commits, lo que permite disponer de un historial de versiones, recuperar estados anteriores del proyecto y asociar el despliegue en la nube a ramas o commits concretos.

3.2.2. Requisitos no funcionales previstos para versiones futuras

Además de los requisitos no funcionales que ya cumple la versión actual del sistema, se contemplan una serie de mejoras de calidad orientadas a versiones posteriores del proyecto.

RNF-F1. Accesibilidad web

La interfaz deberá mejorar progresivamente su accesibilidad, teniendo en cuenta aspectos como contraste de colores, uso correcto de etiquetas, estados de foco visibles, navegación mediante teclado y textos alternativos cuando corresponda. El objetivo será facilitar el uso de la aplicación a un mayor número de personas.

RNF-F2. Monitorización y registro avanzado

El sistema deberá incorporar mecanismos más completos de registro y monitorización, como logs estructurados, métricas básicas de uso, seguimiento de errores en producción y herramientas que permitan analizar el comportamiento real de la aplicación desplegada.

RNF-F3. Estrategia de copias de seguridad y recuperación

Se deberá definir una política de copias de seguridad de la base de datos y un procedimiento de recuperación ante fallos. Esto permitiría restaurar la información de los usuarios en caso de incidente, reduciendo la posible pérdida de datos.

RNF-F4. Rendimiento avanzado en listados grandes

Aunque la aplicación actual es suficiente para un uso individual o académico, en futuras versiones se deberá mejorar el rendimiento cuando existan grandes volúmenes de tareas, asignaciones o grupos. Esto puede incluir paginación, carga incremental, optimización de consultas y reducción del volumen de datos transferido entre backend y frontend.

RNF-F5. Mayor cobertura de pruebas automáticas

El proyecto deberá ampliar la cobertura de pruebas automáticas, especialmente en servicios críticos, seguridad, filtros avanzados, ordenación inteligente, notificaciones y flujos colaborativos. También convendría sanear tests antiguos que hayan quedado desactualizados por la evolución del backend.

RNF-F6. Migraciones de base de datos versionadas

En una versión más profesional, la evolución del esquema de base de datos debería gestionarse mediante migraciones controladas con herramientas como Flyway o Liquibase, en lugar de depender de cambios manuales o del uso temporal de `ddl-auto=update`.

RNF-F7. App móvil específica

Aunque la versión actual funciona como PWA responsive, una evolución futura podría incorporar una aplicación móvil específica, por ejemplo, con React Native. Esta aplicación reutilizaría la API REST existente y permitiría una experiencia móvil más cercana a una aplicación nativa.

RNF-F8. Configuración avanzada de preferencias de usuario

El sistema podrá ampliar las preferencias configurables por usuario, incluyendo ajustes de interfaz, comportamiento de notificaciones, personalización visual, opciones de accesibilidad y otros parámetros de experiencia de uso.

3.3. Casos de uso / Historias de usuario

3.3.1. Casos de uso implementados

A continuación se describen algunos de los casos de uso principales del sistema **Gestor de Tareas**, centrados en la interacción del usuario con la aplicación.

CU-01. Registrarse en la aplicación

- **Actor principal:** Usuario no registrado.
- **Objetivo:** Crear una nueva cuenta de usuario para poder utilizar la aplicación.
- **Precondiciones:**
 - El usuario dispone de un correo electrónico válido y no registrado previamente.
- **Flujo principal:**
 - El usuario accede a la pantalla de registro de la aplicación.
 - El usuario introduce su correo electrónico y una contraseña confirmada.
 - El sistema crea la cuenta en la base de datos.
 - El sistema genera automáticamente las categorías base protegidas asociadas a esa cuenta.
 - El sistema informa al usuario de que el registro se ha completado correctamente.
- **Postcondición:**
 - El usuario queda registrado y puede iniciar sesión con sus credenciales.
- **Flujos alternativos (errores):**
 - Si el correo ya existe o los datos no son válidos, el sistema muestra un mensaje de error indicando el motivo y no crea la cuenta.

CU-02. Iniciar sesión y renovar sesión

- **Actor principal:** Usuario registrado.
- **Objetivo:** Acceder al espacio personal de tareas del usuario y mantener una sesión autenticada mediante tokens.
- **Precondiciones:**
 - El usuario está registrado en el sistema.
- **Flujo principal:**
 - El usuario accede a la pantalla de inicio de sesión.
 - El usuario introduce su correo electrónico y contraseña.
 - El sistema valida las credenciales contra la base de datos.

- Si los datos son correctos, el sistema genera tanto un token JWT de acceso como uno de refresco asociado al usuario.
- El sistema devuelve el access token al cliente y permite el acceso a la interfaz autenticada.
- cuando sea necesario, el cliente puede solicitar un nuevo access token usando el refresh token mediante el endpoint correspondiente.
- **Postcondición:**
 - El usuario queda autenticado y puede consumir la API protegida utilizando el access token, con posibilidad de renovar la sesión mediante refresh token.
- **Flujos alternativos (errores):**
 - Si las credenciales son incorrectas, el sistema devuelve un error de autenticación y no genera el token.
 - Si el refresh token es inválido o ha expirado, el sistema rechaza la renovación de sesión.

CU-03. Gestionar tareas personales

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Crear, consultar, modificar, completar y eliminar tareas personales propias.
- **Precondiciones:**
 - El usuario ha iniciado sesión y dispone de un token JWT válido.
- **Flujo principal (creación y gestión):**
 - El usuario accede a la sección de tareas de la aplicación.
 - El usuario puede crear una nueva tarea indicando al menos un título, prioridad, categoría válida y tiempo estimado, y opcionalmente descripción y fecha de entrega.
 - El sistema valida los datos y guarda la tarea asociándola al usuario autenticado.
 - El usuario puede consultar la lista de sus tareas y seleccionar una para ver sus detalles.
 - El usuario puede modificar los datos de una tarea existente (por ejemplo, cambiar la categoría, la prioridad o la fecha de entrega) y guardar los cambios.
 - El usuario puede marcar una tarea como completada o desmarcarla si es necesario.
 - El usuario puede eliminar una tarea que ya no necesite.

- **Postcondición:**
 - Las tareas del usuario quedan almacenadas y actualizadas correctamente en el sistema, asociadas a su cuenta, además, el sistema registra la fecha de completado y el usuario que realiza la acción cuando una tarea se marca como completada.
- **Flujos alternativos (errores):**
 - Si los datos de una tarea no son válidos, el sistema devuelve un mensaje de error y no aplica los cambios.
 - Si el usuario intenta acceder o modificar una tarea que no le pertenece, el sistema rechaza la operación.

CU-04. Consultar vistas especiales de tareas

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Consultar rápidamente vistas específicas como tareas del día, tareas próximas a vencer y tareas vencidas.
- **Precondiciones:**
 - El usuario ha iniciado sesión y dispone de tareas creadas, con o sin fecha de entrega.
- **Flujo principal:**
 - El usuario accede a la sección de tareas.
 - El usuario selecciona una vista especializada.
 - El sistema puede devolver:
 - Las tareas del día,
 - Las tareas próximas a vencer,
 - Las tareas vencidas
 - El sistema calcula el criterio temporal correspondiente y devuelve solo las tareas que lo cumplen.
- **Postcondición:**
 - El usuario dispone de una vista centrada en lo que debe realizarse en la fecha actual.
- **Flujos alternativos (errores):**
 - Si no existen tareas para la vista seleccionada, el sistema devuelve una lista vacía.

CU-05. Gestionar categorías de tareas

- **Actor principal:** Usuario autenticado.

- **Objetivo:** Organizar sus tareas en categorías personalizadas, respetando las categorías base del sistema.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
- **Flujo principal:**
 - El usuario accede a la sección de categorías.
 - El sistema muestra la lista de categorías disponibles, incluyendo las categorías base y las creadas por el usuario.
 - El sistema muestra las categorías disponibles del usuario, incluyendo categorías base protegidas y categorías personalizadas
 - El sistema valida el nombre y guarda la nueva categoría asociada al usuario o al contexto definido.
 - El usuario puede modificar el nombre de una categoría que no sea una categoría base protegida.
 - El usuario puede eliminar una categoría propia. Al hacerlo, las tareas que utilizaban esa categoría permanecen en el sistema y pasan a quedar sin categoría asignada.
- **Postcondición:**
 - Las categorías quedan actualizadas y las tareas mantienen su integridad, incluso si alguna categoría ha sido eliminada.
- **Flujos alternativos (errores):**
 - Si se intenta eliminar o modificar una categoría base protegida, el sistema debe rechazar la operación.
 - Si el nombre de una categoría no es válido, el sistema debe mostrar un mensaje de error.
 - Si se intenta operar sobre una categoría ajena, el sistema rechaza la operación.

CU-06. Buscar, filtrar y ordenar tareas

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Localizar rápidamente tareas concretas y organizar su vista mediante filtros y criterios de ordenación.
- **Precondiciones:**
 - El usuario ha iniciado sesión y dispone de tareas creadas.
- **Flujo principal:**
 - El usuario accede a la lista de tareas.
 - El usuario introduce criterios de filtrado (por ejemplo, prioridad, tiempo estimado o texto contenido en el título o descripción).

- El sistema aplica los filtros seleccionados y devuelve solo las tareas que los cumplen.
- El usuario puede elegir un criterio de ordenación (por título, prioridad, tiempo estimado, etc.).
- El sistema muestra la lista resultante de tareas filtradas y ordenadas según las preferencias del usuario.
- **Postcondición:**
 - El usuario obtiene una vista adaptada a los criterios de filtrado y ordenación aplicados.
- **Flujos alternativos (errores):**
 - Si los criterios de filtrado no coinciden con ninguna tarea, el sistema muestra una lista vacía.

CU-07. Crear y administrar un grupo

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Crear un grupo y gestionarlo dentro del sistema colaborativo.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
- **Flujo principal:**
 - El usuario crea un nuevo grupo.
 - El sistema lo registra como creador del grupo y como miembro con rol ADMIN.
 - El usuario puede consultar el grupo, editar sus datos, activarlo o inactivarlo y eliminarlo si corresponde a su ownership.
 - El usuario puede consultar la lista de miembros del grupo.
 - El usuario puede añadir miembros, expulsarlos, cambiarles el rol, salir del grupo o transferir el ownership cuando el flujo lo permita.
- **Postcondición:**
 - El grupo queda operativo y gestionado según las reglas de rol y propiedad.
- **Flujos alternativos (errores):**
 - Si un usuario sin permisos intenta realizar una acción reservada, el sistema la rechaza.
 - Si un recurso grupal no pertenece al usuario o no le corresponde gestionarlo, la operación no se permite.

CU-08. Unirse a un grupo por código de invitación

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Incorporarse a un grupo existente mediante un código de invitación válido.
- **Precondiciones:**
 - El grupo existe y dispone de un código de invitación válido.
- **Flujo principal:**
 - El usuario introduce el código de invitación.
 - El sistema comprueba que el grupo existe, está activo y que el usuario no pertenece ya a él.
 - El sistema añade al usuario al grupo con rol MIEMBRO.
- **Postcondición:**
 - El usuario pasa a formar parte del grupo.
- **Flujos alternativos (errores):**
 - Si el código no existe, es inválido o el grupo está inactivo, la operación se rechaza.
 - Si el usuario ya pertenece al grupo, no puede volver a unirse.

CU-09. Asignar tareas desde un grupo

- **Actor principal:** Creador o admin de grupo.
- **Actores secundarios:** Miembros destinatarios del grupo.
- **Objetivo:** Asignar tareas a miembros del grupo mediante duplicación individual.
- **Precondiciones:**
 - Existe un grupo y el actor tiene permisos de administración sobre él.
- **Flujo principal:**
 - El admin selecciona el grupo.
 - Crea una asignación indicando los datos de la tarea base.
 - Elige si la asignación se aplica a todo el grupo o a una selección manual de miembros.
 - El sistema valida los destinatarios y crea una tarea individual para cada uno de ellos.
 - Cada destinatario recibe la tarea en su propia lista personal.
- **Postcondición:**
 - La asignación queda registrada y cada miembro recibe su tarea generada individualmente.
- **Flujos alternativos (errores):**
 - Si algún destinatario no pertenece al grupo o la selección es incoherente, la operación se rechaza.

- Si el usuario no tiene permisos de administración en el grupo, no puede asignar tareas.

CU-10. Revisar entregas de tareas de grupo

- **Actor principal:** Admin de grupo.
- **Actores secundarios:** Miembros destinatarios de asignaciones.
- **Objetivo:** Consultar y revisar el estado de las entregas derivadas de asignaciones de grupo.
- **Precondiciones:**
 - Existen asignaciones creadas en un grupo que el actor puede administrar.
- **Flujo principal:**
 - El admin consulta la lista de asignaciones del grupo.
 - Selecciona una asignación concreta y visualiza el estado individual de cada destinatario.
 - Puede validar una entrega cuando está en estado ENTREGADA.
 - Puede reabrirla si procede, dejando comentario de revisión.
- **Postcondición:**
 - El estado de revisión de cada asignación individual queda actualizado y trazado.
- **Flujos alternativos (errores):**
 - Si se intenta validar o reabrir desde un estado no permitido, el sistema rechaza la operación.
 - Si el usuario no tiene permisos de administración, la operación no se permite.
 -

CU-11. Consultar tareas con origen en grupos

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Ver únicamente las tareas propias cuyo origen está en asignaciones grupales.
- **Precondiciones:**
 - El usuario pertenece a uno o varios grupos con asignaciones generadas.
- **Flujo principal:**
 - El usuario consulta la vista general de tareas asignadas desde grupos.
 - Puede restringir la consulta a un grupo concreto.
 - El sistema devuelve las tareas generadas para ese usuario junto con metadatos de origen y revisión.
- **Postcondición:**

- El usuario obtiene una vista clara de sus tareas procedentes de grupos.
- **Flujos alternativos (errores):**
 - Si no existen tareas de ese tipo, el sistema devuelve una lista vacía.

CU-12. Aplicar filtros combinados sobre tareas

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Consultar una lista unificada de tareas personales y tareas con origen en grupos utilizando múltiples criterios a la vez.
- **Precondiciones:**
 - El usuario ha iniciado sesión y dispone de tareas personales, tareas de grupo o ambas.
- **Flujo principal:**
 - El usuario envía un conjunto de criterios opcionales al endpoint de filtro combinado.
 - El sistema compone una lista común de tareas personales y tareas con origen en grupos.
 - Aplica sobre ella los criterios recibidos: origen, grupo, prioridad, estado, palabras clave, tiempo máximo, categoría, fecha exacta, fecha hasta y criterio de ordenación.
 - Si se solicita, también excluye las tareas ya completadas mediante la opción soloPorCompletar.
 - El sistema devuelve la lista resultante con datos de categoría, origen y revisión cuando procedan.
- **Postcondición:**
 - El usuario obtiene una vista filtrada y ordenada de forma flexible, preparada para ser utilizada por el frontend principal.
- **Flujos alternativos (errores):**
 - Si se envían combinaciones incoherentes, como origen PERSONAL con grupo informado o fecha exacta junto con fecha hasta, el sistema rechaza la operación con un error controlado.
 - Si se informa un grupo o una categoría no válidos para ese usuario, la operación también es rechazada.

CU-13. Guardar y recuperar el último estado del filtro combinado

- **Actor principal:** Usuario autenticado.

- **Objetivo:** Mantener persistido en servidor el último estado usado del filtro combinado.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
- **Flujo principal:**
 - El usuario guarda el estado actual del filtro combinado.
 - El sistema crea o actualiza el estado persistido asociado a su cuenta.
 - Más adelante, el usuario solicita recuperar ese estado.
 - Si ya existe uno guardado, el sistema lo devuelve.
 - Si no existe todavía, devuelve un estado por defecto funcional.
- **Postcondición:**
 - El usuario dispone de un estado persistido del filtro combinado que puede reutilizarse en futuras sesiones o desde otros dispositivos.
- **Flujos alternativos (errores):**
 - Si el estado que se intenta guardar contiene combinaciones inválidas, el sistema rechaza la operación.

CU-14. Consultar tareas recomendadas mediante ordenación inteligente

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Obtener una lista priorizada de tareas pendientes según un criterio inteligente de ordenación.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
 - El usuario dispone de tareas personales, tareas con origen en grupos o ambas.
- **Flujo principal:**
 - El usuario accede a la vista Inteligente desde la interfaz principal.
 - El frontend solicita al backend la lista de tareas recomendadas.
 - El backend aplica internamente el algoritmo inteligente, combinando factores como fecha de entrega, prioridad, tiempo estimado, antigüedad, categoría y estado de revisión grupal.
 - El sistema excluye las tareas que no son accionables según las reglas del algoritmo, como tareas completadas o entregas grupales ya validadas.
 - El frontend muestra las tareas en el orden recomendado, sin exponer la puntuación interna calculada por el backend.
- **Postcondición:**
 - El usuario obtiene una vista priorizada de tareas sobre las que puede actuar.
- **Flujos alternativos (errores):**

- Si no hay tareas recomendadas, el sistema muestra un estado vacío.
- Si ocurre un error al cargar las recomendaciones, el frontend muestra un mensaje de error controlado.

CU-15. Aplicar filtros avanzados desde la pantalla principal

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Filtrar tareas personales y tareas con origen en grupos desde una misma zona de trabajo.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
 - Existen tareas personales, tareas grupales, categorías o grupos disponibles según el caso.
- **Flujo principal:**
 - El usuario abre el panel de filtros avanzados en la pantalla principal.
 - Selecciona uno o varios criterios de filtrado, como origen, grupo, categoría, prioridad, estado, fecha, palabras clave, tiempo máximo u ordenación.
 - El sistema construye una petición de filtro combinado con los criterios seleccionados.
 - El backend devuelve una lista unificada de tareas que cumplen los criterios.
 - El frontend muestra los resultados filtrados como una vista propia dentro del dashboard.
 - El sistema guarda automáticamente el estado del filtro aplicado para poder restaurarlo posteriormente.
- **Postcondición:**
 - El usuario ve una lista de tareas adaptada a los criterios seleccionados y el estado del filtro queda persistido.
- **Flujos alternativos (errores):**
 - Si los filtros no devuelven resultados, el frontend muestra un estado vacío.
 - Si el backend rechaza una combinación incoherente, el frontend muestra un mensaje de error.

CU-16. Consultar y cerrar notificaciones internas

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Consultar las notificaciones pendientes generadas por el sistema y cerrarlas cuando ya no sean necesarias.
- **Precondiciones:**
 - El usuario ha iniciado sesión.

- Existen notificaciones activas asociadas a su cuenta, o el sistema puede devolver una lista vacía.
- **Flujo principal:**
 - El usuario pulsa la campanita de notificaciones en la interfaz.
 - El frontend solicita al backend las notificaciones activas del usuario.
 - El sistema muestra la lista de notificaciones pendientes.
 - El usuario puede cerrar una notificación concreta.
 - El usuario puede cerrar todas las notificaciones pendientes de una vez.
 - El contador de la campanita se actualiza según las notificaciones activas restantes.
- **Postcondición:**
 - Las notificaciones cerradas dejan de mostrarse como pendientes para el usuario.
- **Flujos alternativos (errores):**
 - Si no hay notificaciones pendientes, el sistema muestra un estado vacío.
 - Si se intenta cerrar una notificación inexistente o ajena, el backend rechaza la operación.

CU-17. Configurar preferencias de notificación

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Personalizar qué tipos de avisos y recordatorios quiere recibir.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
- **Flujo principal:**
 - El usuario abre el panel de preferencias de notificación desde la campanita.
 - El sistema carga sus preferencias actuales o crea una configuración por defecto si todavía no existe.
 - El usuario activa o desactiva las notificaciones generales.
 - El usuario configura avisos 24 horas antes del vencimiento.
 - El usuario selecciona las prioridades y grupos que deben tenerse en cuenta para esos avisos.
 - El usuario activa o desactiva los recordatorios inteligentes y las notificaciones por asignaciones de grupo.
 - El frontend envía la configuración actualizada al backend.
- **Postcondición:**
 - Las preferencias del usuario quedan guardadas y se aplican a la generación posterior de notificaciones.
- **Flujos alternativos (errores):**

- Si se selecciona un grupo que no pertenece al usuario autenticado, el backend rechaza la operación.
- Si la configuración enviada no es válida, el sistema devuelve un error controlado.

CU-18. Activar un recordatorio inteligente en una tarea

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Activar un recordatorio calculado automáticamente para una tarea personal.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
 - La tarea pertenece al usuario autenticado.
 - La tarea no está completada.
 - La tarea tiene fecha de entrega.
- **Flujo principal:**
 - El usuario pulsa la acción de recordatorio inteligente en una tarjeta de tarea.
 - El frontend envía al backend la petición para activar el recordatorio.
 - El backend calcula la fecha programada restando a la fecha de entrega una hora más el tiempo estimado de la tarea.
 - El sistema crea, reactiva o recalcula el recordatorio asociado a la tarea.
 - Cuando llega la fecha programada, el scheduler genera la notificación correspondiente.
- **Postcondición:**
 - La tarea queda asociada a un recordatorio inteligente activo.
- **Flujos alternativos (errores):**
 - Si la tarea no tiene fecha de entrega, el backend rechaza la activación.
 - Si la fecha calculada del recordatorio ya ha vencido, el backend rechaza la activación.
 - Si la tarea no pertenece al usuario autenticado, la operación no se permite.

CU-19. Activar notificaciones del dispositivo mediante Web Push

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Permitir que las notificaciones internas puedan mostrarse también como notificaciones del sistema del navegador o de la PWA.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
 - El navegador es compatible con Service Worker, Notification API y PushManager.

- La aplicación dispone de la clave pública VAPID configurada.
- **Flujo principal:**
 - El usuario activa las notificaciones del dispositivo desde el panel de preferencias.
 - El navegador solicita permiso para mostrar notificaciones.
 - Si el usuario acepta, el frontend registra o reutiliza el service worker.
 - El frontend crea una suscripción push del navegador.
 - El frontend envía al backend los datos públicos de la suscripción.
 - El backend registra la suscripción asociada al usuario autenticado.
 - Cuando se genera una notificación interna compatible, el backend intenta enviarla también mediante Web Push.
 - El navegador muestra la notificación del sistema y, al pulsarla, abre o enfoca la aplicación.
- **Postcondición:**
 - El dispositivo queda registrado para recibir notificaciones Web Push de la aplicación.
- **Flujos alternativos (errores):**
 - Si el navegador no soporta Web Push, el frontend muestra que la función no está disponible.
 - Si el usuario deniega permisos, no se registra la suscripción.
 - Si falla el envío Web Push, la notificación interna sigue existiendo en la campanita.

CU-20. Usar la aplicación como PWA en móvil o escritorio

- **Actor principal:** Usuario autenticado.
- **Objetivo:** Utilizar la aplicación desde navegador o instalada como PWA, manteniendo una experiencia adaptada a escritorio y móvil.
- **Precondiciones:**
 - El usuario accede desde un navegador compatible.
 - La aplicación frontend está desplegada y disponible.
- **Flujo principal:**
 - El usuario accede a la URL del frontend.
 - El navegador carga la aplicación React/PWA.
 - El usuario puede iniciar sesión o registrarse.
 - Una vez autenticado, accede al dashboard principal.
 - En escritorio, la interfaz muestra el layout completo con paneles y tarjetas adaptadas al ancho disponible.
 - En móvil, la interfaz adapta tarjetas, filtros, panel de vistas, notificaciones, modales y pantallas de autenticación al tamaño de pantalla.

- Si el navegador lo permite, el usuario puede instalar la PWA y abrirla como una aplicación independiente.
- **Postcondición:**
 - El usuario puede utilizar las funcionalidades principales de la aplicación tanto desde escritorio como desde móvil.
- **Flujos alternativos (errores):**
 - Si el backend tarda en responder por arranque en frío o latencia de red, la interfaz muestra estados de carga o errores controlados.
 - Si el navegador no permite instalar la PWA, la aplicación sigue funcionando como web normal.

3.3.2. Casos de uso previstos para versiones futuras

Además de los casos de uso implementados, el proyecto contempla varias ampliaciones futuras centradas en seguridad de cuenta, personalización, explotación de datos e integraciones externas.

CU-F1. Verificar cuenta y proteger el registro

- **Actor principal:** Usuario con rol profesor.
- **Actores secundarios:** Usuarios con rol alumno pertenecientes a un grupo.
- **Objetivo:** Permitir que el usuario verifique su cuenta mediante correo electrónico e incorporar un mecanismo de captcha en el registro para reducir registros automatizados.
- **Precondiciones:**
 - Existe al menos un grupo de usuarios creado.
 - El usuario autenticado tiene asignado el rol profesor en ese grupo.
- **Flujo principal:**
 - El profesor accede a la sección de gestión de grupos en la aplicación.
 - El profesor selecciona un grupo existente (por ejemplo, una clase) o crea uno nuevo.
 - El profesor crea una nueva tarea indicando título, descripción, categoría, prioridad y fecha de entrega, asociándola al grupo.
 - El sistema valida los datos de la tarea.
 - El sistema crea una instancia de la tarea para cada alumno perteneciente al grupo o la asocia lógicamente al grupo de forma que se tenga en cuenta en la lista de cada alumno.
 - La tarea pasa a estar visible en la sección de tareas de cada alumno miembro del grupo.
- **Postcondición:**

- Todos los alumnos del grupo ven la nueva tarea asignada por el profesor en sus listas de tareas.

CU-F2. Gestionar perfil de usuario y ajustes de la aplicación

- **Actor principal:** Usuario con rol profesor.
- **Objetivo:** Permitir que el usuario consulte y modifique información básica de su perfil, así como preferencias generales de la aplicación, configuración visual, comportamiento de notificaciones y otros ajustes personales.
- **Precondiciones:**
 - Existen tareas asignadas a un grupo del que el usuario es profesor.
- **Flujo principal:**
 - El profesor accede a la sección de grupos y selecciona un grupo concreto.
 - El sistema muestra la lista de tareas asignadas a ese grupo.
 - El profesor selecciona una tarea para ver su detalle.
 - El sistema muestra, para esa tarea, la lista de alumnos del grupo junto con el estado de cada uno (por ejemplo: pendiente, completada dentro de plazo, completada con retraso).
- **Postcondición:**
 - El profesor dispone de una visión general del progreso de los alumnos respecto a las tareas asignadas al grupo.

CU-F3. Consultar estadísticas, calendario e integraciones externas

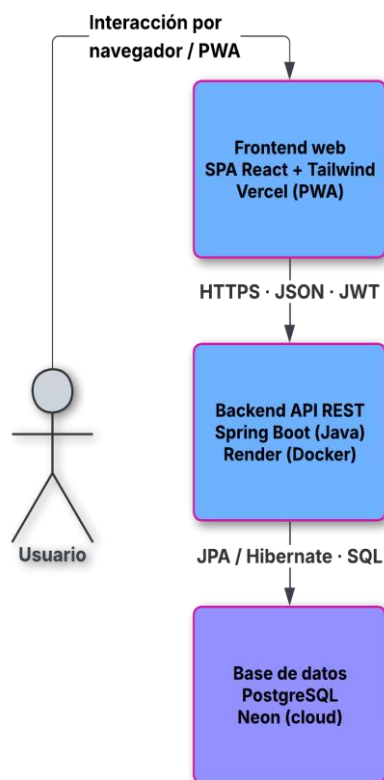
- **Actor principal:** Usuario autenticado.
- **Objetivo:** Ampliar la aplicación con vistas avanzadas de consulta, como estadísticas de tareas, calendario mensual e integración con servicios externos como Google.
- **Precondiciones:**
 - El usuario ha iniciado sesión.
- **Flujo principal:**
 - El usuario accede a una sección avanzada de análisis o integración.
 - El sistema muestra estadísticas de tareas completadas, pendientes o vencidas.
 - El usuario puede consultar tareas distribuidas en una vista de calendario.
 - En futuras versiones, el usuario puede vincular su cuenta con servicios externos para ampliar las posibilidades de la aplicación.
- **Postcondición:**

- El usuario obtiene una visión más completa de su actividad y puede ampliar la utilidad de la aplicación mediante integraciones externas.

4. Diseño

4.1. Arquitectura general del sistema

Arquitectura general del sistema
Gestor de Tareas



La aplicación **Gestor de Tareas** sigue una arquitectura web de tipo **cliente-servidor**, en la que uno o varios clientes consumen los servicios ofrecidos por un backend central, que a su vez se conecta a una base de datos relacional. Sobre esta base se construye una organización interna por capas en el lado del servidor, de manera que la lógica de negocio, el acceso a datos y la exposición de la API REST queden claramente separados.

4.1.1. Componentes principales

Desde un punto de vista físico, el sistema se compone de los siguientes elementos:

- **Cliente web (frontend)**
La interfaz principal de usuario es una aplicación web de una sola página (SPA) desarrollada con React y Vite, desplegada en la plataforma Vercel. Esta aplicación se ejecuta en el navegador del usuario y se comunica con el backend mediante peticiones HTTP en formato JSON. Además, está configurada como

aplicación web progresiva (PWA), por lo que puede instalarse desde navegadores compatibles tanto en escritorio como en dispositivos móviles.

El frontend incorpora un service worker propio para gestionar el comportamiento PWA y la recepción de notificaciones Web Push. También incluye una interfaz responsive adaptada a escritorio y móvil, con vistas para tareas, categorías, grupos, filtros avanzados, notificaciones y autenticación.

- **Servidor de aplicación (backend)**

El backend está desarrollado con Spring Boot y se despliega en la plataforma Render, utilizando contenedores Docker generados a partir del código fuente. Este componente expone una API REST que centraliza la lógica de negocio relacionada con usuarios, tareas, categorías, grupos, asignaciones, filtros avanzados, notificaciones y autenticación.

Además de responder a peticiones directas del frontend, el backend ejecuta procesos programados mediante Spring para generar avisos y recordatorios pendientes. También dispone de un endpoint público de health check y de un sistema opcional de self-ping para mejorar la estabilidad del servicio desplegado durante pruebas y demostraciones.

- **Base de datos**

Los datos de la aplicación se almacenan en una base de datos relacional PostgreSQL alojada en el servicio en la nube Neon. El backend se conecta a esta base de datos mediante JPA/Hibernate, utilizando entidades Java que representan las tablas del modelo.

La base de datos almacena la información principal del sistema: usuarios, tareas, categorías, grupos, miembros, asignaciones, estados guardados del filtro avanzado, notificaciones, recordatorios, preferencias de notificación y suscripciones push. Esta estructura permite mantener tanto el uso individual de la aplicación como las funcionalidades colaborativas y de notificación.

Aunque en la versión actual el cliente principal es la aplicación web, la arquitectura está pensada para que en el futuro, otros clientes, como una aplicación móvil específica, puedan consumir la misma API REST sin necesidad de modificar la lógica del servidor.

4.1.2. Flujo de comunicación

El flujo general de comunicación entre los componentes del sistema parte de la interacción del usuario con la aplicación web, ya sea desde el navegador o desde la PWA instalada. El frontend carga la interfaz React, gestiona el estado visual de la aplicación y envía peticiones HTTP al backend cuando necesita consultar, crear, modificar o eliminar información.

Los datos se intercambian en formato JSON. En las operaciones públicas, como registro, login o refresh de sesión, el cliente puede comunicarse con la API sin access token. En las operaciones protegidas, el frontend incluye en la cabecera Authorization un JWT con el formato Bearer . El backend valida este token antes de ejecutar la lógica de negocio correspondiente.

Cuando el usuario inicia sesión correctamente, el backend devuelve un access token y un refresh token. El access token se utiliza para acceder a los endpoints protegidos. Si el access token caduca, el frontend puede solicitar uno nuevo mediante el endpoint de refresh, utilizando el refresh token correspondiente. De esta forma, la sesión puede mantenerse activa sin obligar al usuario a introducir sus credenciales constantemente.

Una vez validada la petición, el backend delega la operación en la capa de servicios. Si la operación requiere persistencia o consulta de datos, los servicios acceden a la base de datos PostgreSQL mediante repositorios Spring Data JPA y entidades gestionadas por Hibernate. La respuesta se construye de nuevo en formato JSON y se devuelve al frontend, que actualiza la interfaz según el resultado recibido.

En el caso de la PWA, el frontend cuenta además con un service worker propio. Este service worker permite gestionar recursos propios de la aplicación progresiva y recibir notificaciones Web Push cuando el usuario ha dado permiso al navegador y existe una suscripción push registrada. Cuando el backend genera una notificación interna y existe una suscripción válida, intenta enviar también una notificación Web Push al navegador o dispositivo del usuario.

El backend no solo responde a peticiones directas del frontend. También ejecuta procesos programados mediante Spring para revisar recordatorios inteligentes vencidos, avisos 24 horas antes del vencimiento y tareas internas de mantenimiento como el self-ping opcional del servicio desplegado en Render. Estos procesos trabajan sobre la misma base de datos y generan información que posteriormente puede consultarse desde la interfaz.

La API está configurada para aceptar peticiones únicamente desde los orígenes autorizados, como el dominio de producción del frontend y el entorno local de desarrollo. En producción, la comunicación entre cliente y servidor se realiza mediante HTTPS, protegiendo el intercambio de credenciales, tokens y datos de la aplicación.

4.1.3. Arquitectura lógica del backend

Internamente, el backend sigue una organización por capas que separa responsabilidades y facilita el mantenimiento del código. La estructura se inspira en el patrón **Modelo–Vista–Controlador (MVC)** adaptado a una **API REST**: las entidades y reglas de dominio forman el modelo, los controladores REST actúan como capa de entrada, y las respuestas JSON consumidas por el frontend cumplen el papel de representación externa de los datos.

- **Capa de controladores (controller):** recibe las peticiones HTTP, define las rutas de la API y delega la lógica de negocio en los servicios correspondientes. En esta capa se encuentran controladores como `UsuarioController`, `TareaController`, `CategoriaController`, `GrupoController`, `AsignacionGrupoController`, `NotificacionController` y `HealthController`.
- **Capa de servicios (service):** concentra la lógica de negocio principal de la aplicación. Aquí se implementan las reglas de usuarios, tareas, categorías, grupos, asignaciones, filtros avanzados, ordenación inteligente, notificaciones y recordatorios. Esta capa también realiza comprobaciones de ownership y permisos antes de modificar o devolver recursos.
- **Capa de repositorios (repository):** se encarga del acceso a la base de datos mediante **Spring Data JPA**. Los repositorios permiten consultar y persistir entidades como usuarios, tareas, categorías, grupos, asignaciones, estados de filtro, notificaciones, recordatorios, preferencias y suscripciones push.
- **Capa de modelo (model):** define las entidades **JPA** y enumeraciones que representan el dominio de la aplicación. Incluye entidades como `Usuario`, `Tarea`, `Categoria`, `Grupo`, `GrupoMiembro`, `AsignacionGrupo`, `AsignacionGrupoMiembro`, `EstadoFiltroTarea`, `Notificacion`, `RecordatorioTarea`, `PreferenciasNotificacion` y `PushSubscripcion`.
- **Capa de DTOs (dto):** contiene los objetos de transferencia de datos utilizados en peticiones y respuestas. Estos DTOs evitan exponer directamente las entidades internas, permiten adaptar la información devuelta al frontend y centralizan validaciones de entrada mediante anotaciones de Bean Validation.
- **Capa de seguridad (security):** agrupa la configuración y componentes relacionados con la autenticación y autorización. Incluye el filtro JWT, la

configuración de **Spring Security**, el servicio de detalles de usuario, la generación y validación de tokens y las reglas de acceso a endpoints protegidos.

- **Capa de excepciones (exception):** contiene las excepciones personalizadas del proyecto y el manejador global de errores. Esta capa permite transformar errores internos o de negocio en respuestas HTTP homogéneas, con códigos y mensajes adecuados para el cliente.
- **Capa de configuración (config):** reúne configuraciones transversales de la aplicación, como CORS, Swagger/OpenAPI, reloj del sistema, inicialización de datos, comportamiento de la SPA, tareas programadas y propiedades de despliegue.

Además de estas capas generales, el backend incorpora servicios específicos para funcionalidades avanzadas. La **ordenación inteligente** se apoya en componentes de scoring separados, como `TareaScoringContext`, `TareaInteligenteScorer` y `TareaInteligenteRankingService`, lo que permite mantener la fórmula de recomendación aislada del controlador y reutilizable desde diferentes flujos. El sistema de **notificaciones** incluye servicios propios para generar notificaciones internas, procesar recordatorios, gestionar preferencias, manejar suscripciones push y enviar Web Push cuando corresponde.

Esta arquitectura permite que los controladores permanezcan ligeros, que la lógica de negocio esté centralizada en servicios, que el acceso a datos quede aislado en repositorios y que las funcionalidades transversales como seguridad, errores, configuración, notificaciones y scoring puedan evolucionar sin mezclar responsabilidades.

4.1.4. Entornos de ejecución

La aplicación distingue entre dos contextos principales de ejecución: el entorno de desarrollo local y el entorno de producción desplegado en la nube.

- **Entorno de desarrollo local**
En desarrollo, el backend de **Spring Boot** se ejecuta en la máquina del desarrollador, normalmente desde el IDE o mediante Maven. El frontend se ejecuta en modo desarrollo con **Vite**, lo que permite recarga rápida de cambios y validación inmediata de la interfaz. La base de datos puede ejecutarse en local o conectarse a una instancia remota de PostgreSQL, siempre que se configure correctamente la conexión.

En este entorno se utilizan archivos de configuración locales y variables específicas para evitar mezclar datos de desarrollo con los de producción. El frontend puede consumir el backend local desde <http://localhost:8080>, mientras que Vite suele levantar la aplicación en <http://localhost:5173>.

- **Entorno de producción en la nube**
En producción, el sistema se despliega en tres servicios principales. El **frontend** se publica en **Vercel**, donde se construye la aplicación React/Vite y se sirve como PWA. El **backend** se despliega en **Render** como servicio Docker, exponiendo la API REST del proyecto. La **base de datos** se aloja en **Neon**, utilizando PostgreSQL como sistema de gestión relacional.

La comunicación entre frontend y backend se realiza mediante **HTTPS**, intercambiando datos en formato JSON y utilizando tokens JWT para acceder a los endpoints protegidos. La API acepta peticiones desde los orígenes permitidos, incluyendo el dominio de producción del frontend y el entorno local de desarrollo.

En producción se utilizan variables de entorno para configurar aspectos sensibles o dependientes del entorno, como la conexión a base de datos, el secreto JWT, el perfil activo de Spring, la configuración de Web Push, la zona horaria y el self-ping del backend. Las claves privadas, como la clave privada VAPID o secretos de autenticación, no se almacenan en el repositorio.

El backend dispone además de un endpoint público de disponibilidad, GET `/api/health`, utilizado para comprobar que el servicio está levantado. Sobre este endpoint se apoya el sistema opcional de **self-ping**, configurado para reducir el impacto del reposo automático de Render durante pruebas y demostraciones.

Para evitar desfases de fechas entre el entorno cloud y el uso esperado en España, Render se configura explícitamente con la zona horaria **Europe/Madrid**. Esto es importante para el registro de fechas de creación, fechas de vencimiento, avisos 24 horas antes y recordatorios inteligentes.

La configuración de producción mantiene `SPRING_JPA_HIBERNATE_DDL_AUTO=validate`, de modo que el backend valida el esquema de base de datos en lugar de modificarlo automáticamente. Durante fases concretas de creación de nuevas tablas se puede usar temporalmente `update`, pero el estado final recomendado para producción es `validate`.

4.2. Modelo de datos

El modelo de datos del sistema **Gestor de Tareas** ha evolucionado desde un núcleo inicial centrado en usuarios, tareas y categorías hacia una estructura más completa que cubre también colaboración entre usuarios, persistencia de filtros avanzados y sistema de notificaciones. En la versión actual, la base de datos utilizada es **PostgreSQL** y el acceso desde el backend se realiza mediante **JPA/Hibernate**, de modo que las entidades Java se corresponden con tablas del esquema relacional.

En el núcleo individual del sistema se mantienen las entidades **Usuario**, **Tarea** y **Categoria**, que permiten registrar usuarios, gestionar sus tareas personales y organizarlas mediante categorías propias. Sobre esta base se añaden las entidades **Grupo** y **GrupoMiembro**, que permiten modelar la colaboración entre usuarios, los roles dentro de cada grupo y la pertenencia de cada usuario a distintos espacios colaborativos.

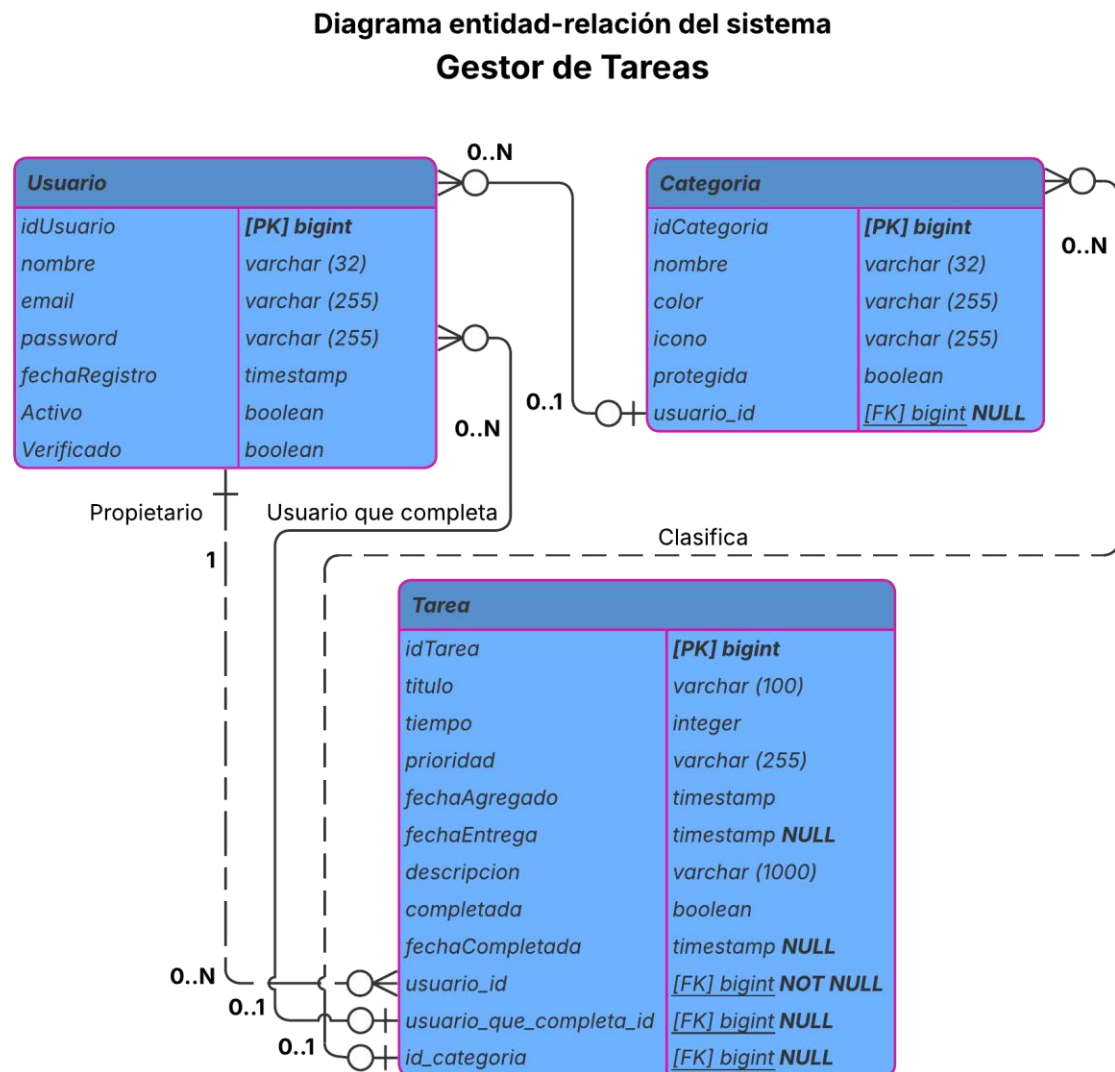
El sistema de asignaciones grupales se representa mediante las entidades **AsignacionGrupo** y **AsignacionGrupoMiembro**. La primera actúa como cabecera de una asignación creada dentro de un grupo, mientras que la segunda representa la parte individual de cada destinatario, enlazando la asignación original con la tarea real generada para cada miembro. Este diseño permite mantener una trazabilidad clara entre una tarea asignada desde un grupo y la tarea individual que recibe cada usuario.

Para la consulta avanzada de tareas, el modelo incorpora la entidad **EstadoFiltroTarea**, que almacena en servidor el último estado del filtro combinado utilizado por cada usuario. Gracias a esta entidad, el frontend puede restaurar los criterios de búsqueda y ordenación usados previamente, incluso si el usuario vuelve a entrar más tarde o utiliza otro dispositivo.

La versión actual también incorpora un bloque específico de notificaciones. Este bloque se apoya en las entidades **Notificacion**, **RecordatorioTarea**, **PreferenciasNotificacion** y **PushSubscripcion**. Con ellas se representan las notificaciones internas visibles en la campanita, los recordatorios programados por tarea, las preferencias de notificación de cada usuario y las suscripciones Web Push registradas desde navegadores o dispositivos compatibles.

Además de estas entidades, el modelo utiliza distintos tipos enumerados para representar aspectos de negocio como la prioridad de una tarea, su estado lógico, los roles dentro de un grupo, el tipo de asignación grupal, el estado de revisión de una entrega, el tipo de notificación, el tipo de recordatorio y el estado del envío push. Estos enums permiten restringir los valores posibles y facilitan que la lógica de negocio sea más clara y mantenible.

4.2.1. Diagrama entidad–relación simplificado



En la figura se muestra un diagrama entidad–relación simplificado del núcleo principal del sistema. Para mantener la legibilidad, el diagrama se centra en las entidades base **Usuario**, **Tarea** y **Categoria**, que forman la parte esencial de la gestión individual de tareas.

Este esquema permite visualizar las relaciones principales del sistema:

- Un **Usuario** puede tener muchas **Tareas** asociadas.
- Un **Usuario** puede tener muchas **Categorías** propias.
- Cada **Tarea** pertenece a un único **Usuario** propietario.
- Una **Tarea** puede estar asociada opcionalmente a una **Categoría**.

- Una **Tarea** puede registrar también el **Usuario** que la completa, lo que permite diferenciar entre el propietario de la tarea y el usuario que realiza la acción en determinados flujos colaborativos.

El modelo completo de la aplicación incluye más entidades que no se representan en este diagrama simplificado para evitar una figura excesivamente grande y difícil de leer. Entre ellas se encuentran las entidades de grupos, miembros, asignaciones, estado guardado del filtro avanzado, notificaciones, recordatorios, preferencias de notificación y suscripciones push. Estas entidades se describen en el apartado siguiente.

4.2.2. Descripción de entidades

Usuario

La entidad Usuario representa a cada persona que utiliza la aplicación. Sus atributos principales son:

- **idUsuario**: identificador único del usuario.
- **nombre**: nombre visible del usuario en la aplicación.
- **email**: dirección de correo electrónico utilizada como identificador de inicio de **sesión**; debe ser única.
- **password**: contraseña cifrada del usuario.
- **fechaRegistro**: fecha y hora en la que se creó la cuenta.
- **activo**: indicador de si la cuenta está activa.
- **verificado**: indicador previsto para marcar si la cuenta ha sido verificada mediante correo electrónico en futuras iteraciones.
- **Relación 1-N con Tarea**: un usuario puede tener muchas tareas asociadas.
- **Relación 1-N con Categoría**: un usuario puede disponer de sus propias categorías, incluidas las categorías base generadas al crear la cuenta y las categorías personalizadas que añada posteriormente.
- **Relación 1-N con Grupo como creador**: un usuario puede ser creador de varios grupos.
- **Relación 1-N con GrupoMiembro**: un usuario puede pertenecer a varios grupos como miembro.
- **Relación 1-1 con EstadoFiltroTarea**: un usuario puede tener asociado un único estado persistido del filtro combinado.

Además, en la versión actual el usuario se relaciona también con el sistema de notificaciones. Puede tener asociadas sus propias **preferencias de notificación**, varias

notificaciones internas, varios **recordatorios programados** y una o varias **suscripciones push** correspondientes a los navegadores o dispositivos desde los que haya activado las notificaciones.

Categoría

La entidad Categoría se utiliza para agrupar y clasificar las tareas. Sus atributos principales son:

- **idCategoría**: identificador único de la categoría.
- **nombre**: nombre de la categoría.
- **color**: código de color en formato hexadecimal (#RRGGBB) utilizado por el **frontend** para representar visualmente la categoría.
- **icono**: nombre o representación del icono asociado a la categoría, pensado para su uso en la interfaz gráfica.
- **protegida**: indicador que permite distinguir las categorías base del sistema de las categorías personalizadas creadas posteriormente por el usuario.
- **usuario**: referencia al Usuario propietario de la categoría. En la versión actual, tanto las categorías base como las categorías personalizadas están asociadas a un usuario concreto, de modo que cada cuenta dispone de sus propias categorías y no comparte este recurso con el resto de usuarios.
- **Relación N-1 con Usuario**: muchas categorías pueden pertenecer a un mismo usuario.
- **Relación 1-N con Tarea**: una categoría puede estar asociada a muchas tareas distintas.

Tarea

La entidad Tarea representa cada unidad de trabajo que el usuario quiere gestionar en la aplicación. Sus atributos principales son:

- **idTarea**: identificador único de la tarea.
- **título**: título descriptivo de la tarea.
- **tiempo**: tiempo estimado para realizar la tarea, expresado en minutos.
- **prioridad**: nivel de prioridad de la tarea, representado por el enum Prioridad (BAJA, MEDIA, ALTA o IMPRESCINDIBLE).
- **fechaAgregado**: fecha y hora en la que la tarea fue creada.
- **fechaEntrega**: fecha límite o fecha objetivo para completar la tarea.
- **descripcion**: texto descriptivo opcional con más detalles sobre la tarea.
- **categoría**: referencia opcional a la entidad Categoría. Si una categoría se elimina, las tareas que la utilizaban permanecen en el sistema y pasan a quedar sin categoría asignada.

- **fechaCompletada:** fecha y hora en la que la tarea se marcó como completada, si procede.
- **completada:** indicador booleano que señala si la tarea está completada o no.
- **usuario:** referencia al Usuario propietario de la tarea.
- **usuarioQueCompleta:** referencia opcional al Usuario que ha marcado la tarea como completada.
- **Relación opcional con AsignacionGrupoMiembro:** una tarea puede haber sido generada como consecuencia de una asignación de grupo.

A partir de estos datos, el sistema calcula el estado de la tarea utilizando el enum Estado, que distingue entre tareas en curso, vencidas, completadas dentro de plazo, completadas con retraso o sin fecha de entrega. Este valor se obtiene dinámicamente a partir de la información almacenada, por lo que no requiere un campo adicional persistido en la tabla.

En la versión actual, una tarea también puede estar vinculada al sistema de recordatorios y notificaciones. Las tareas personales con fecha de entrega pueden tener asociado un **recordatorio inteligente**, calculado a partir de la fecha de vencimiento y del tiempo estimado. Además, determinadas notificaciones internas pueden referenciar una tarea concreta, por ejemplo un aviso 24 horas antes del vencimiento o un recordatorio inteligente ya procesado.

Grupo

La entidad Grupo representa una unidad de colaboración entre usuarios. Permite agrupar miembros, definir un creador, asignar roles de negocio y servir como base para las asignaciones de tareas compartidas. Sus atributos principales son:

- **idGrupo:** identificador único del grupo.
- **nombre:** nombre visible del grupo.
- **descripcion:** texto descriptivo opcional.
- **color:** color asociado al grupo para su representación en interfaz.
- **icono:** icono asociado al grupo.
- **activo:** indicador de si el grupo está activo o inactivo.
- **codigoPublico:** identificador público del grupo.
- **codigoInvitacion:** código utilizado para permitir la incorporación de nuevos miembros.
- **creador:** referencia al Usuario propietario del grupo.
- **Relación N-1 con Usuario:** muchos grupos pueden haber sido creados por un mismo usuario.
- **Relación 1-N con GrupoMiembro:** un grupo puede tener muchos miembros.

- **Relación 1-N con AsignacionGrupo:** un grupo puede tener muchas asignaciones de tareas.

GrupoMiembro

La entidad GrupoMiembro representa la pertenencia de un usuario a un grupo concreto. Es la tabla de unión entre usuarios y grupos y, además, almacena información de negocio propia de esa pertenencia. Sus atributos principales son:

- **idGrupoMiembro:** identificador único de la pertenencia.
- **grupo:** referencia al grupo al que pertenece el usuario.
- **usuario:** referencia al usuario miembro.
- **rol:** rol del usuario dentro del grupo, representado mediante el enum RolGrupo.
- **fechaUnion:** fecha en la que el usuario se incorporó al grupo.
- **anadidoPor:** referencia opcional al usuario que añadió al miembro al grupo, cuando esa acción no se ha producido mediante auto-unión por código.

La combinación grupo + usuario debe ser única para evitar membresías duplicadas.

AsignacionGrupo

La entidad AsignacionGrupo representa la asignación original creada dentro de un grupo. Funciona como “cabecera” de una asignación colectiva y mantiene el contexto general de la tarea distribuida. Sus atributos principales son:

- **idAsignacionGrupo:** identificador único de la asignación.
- **grupo:** referencia al grupo desde el que se realiza la asignación.
- **creador:** referencia al usuario que crea la asignación dentro del grupo.
- **titulo, descripcion, tiempo, prioridad y fechaEntrega:** datos base de la tarea asignada.
- **tipoAsignacion:** modo de asignación utilizado, representado por el enum TipoAsignacionGrupo, distinguiendo entre asignación a todo el grupo o selección manual.
- **fechaAsignacion:** momento en el que se crea la asignación.
- **Relación 1-N con AsignacionGrupoMiembro:** una asignación puede generar varias asignaciones individuales, una por destinatario.

AsignacionGrupoMiembro

La entidad AsignacionGrupoMiembro representa la parte individual de una asignación grupal para un miembro concreto. Es la entidad clave para la trazabilidad del reparto y del seguimiento de entregas. Sus atributos principales son:

- **idAsignacionGrupoMiembro**: identificador único de la asignación individual.
- **asignacionGrupo**: referencia a la asignación grupal original.
- **usuarioMiembro**: referencia al usuario destinatario.
- **tareaGenerada**: referencia a la tarea individual real creada para ese usuario.
- **estadoRevisionAsignacion**: estado de revisión de la asignación individual, representado mediante el enum EstadoRevisionAsignacion.
- **comentarioRevision**: comentario opcional dejado por el admin en procesos de validación o reapertura.
- **fechaRevision**: fecha en la que se revisó la entrega.
- **fechaEntregaInicial**: fecha de entrega inicialmente fijada en la asignación.
- **fechaEntregaActual**: fecha de entrega vigente en ese momento si ha sufrido cambios.

Esta entidad permite saber qué tarea individual procede de qué asignación grupal y en qué estado de entrega o revisión se encuentra cada miembro.

EstadoFiltroTarea

La entidad EstadoFiltroTarea se utiliza para persistir en servidor el último estado del filtro combinado utilizado por un usuario. No representa una tarea ni una categoría, sino una preferencia funcional persistida del sistema. Sus atributos principales son:

- **idEstadoFiltroTarea**: identificador único del estado guardado.
- **usuario**: referencia al usuario propietario del estado guardado.
- **origen**: origen seleccionado en el filtro combinado.
- **idGrupo**: grupo concreto seleccionado, si aplica.
- **prioridad**: prioridad seleccionada en formato legacy, si aplica.
- **prioridades**: conjunto de prioridades seleccionadas en el filtro avanzado.
- **estado**: estado de tarea seleccionado en formato legacy, si aplica.
- **estados**: conjunto de estados seleccionados en el filtro avanzado.
- **palabrasClave**: texto de búsqueda libre, si aplica.
- **tiempoMax**: valor máximo de tiempo estimado utilizado en el filtro.
- **idCategoría**: categoría seleccionada, si aplica.
- **fechaEntregaExacta**: fecha exacta seleccionada, si aplica.
- **fechaEntregaHasta**: fecha límite seleccionada, si aplica.
- **criterioOrdenActivo**: criterio de ordenación actualmente seleccionado.
- **soloPorCompletar**: indicador de si deben excluirse las tareas ya completadas del resultado.
- **updatedAt**: fecha de última actualización del estado persistido.

Esta entidad permite recuperar el último estado del filtro combinado cuando el usuario vuelve a entrar en la aplicación o cambia de dispositivo, dejando preparado el backend para una experiencia de uso más continua en el frontend.

Aunque se mantienen los campos simples prioridad y estado por compatibilidad, la versión actual del filtro avanzado trabaja principalmente con listas de **prioridades** y **estados**, permitiendo seleccionar varios valores a la vez. Esto facilita que el frontend pueda aplicar filtros más flexibles sin tener que simular varias consultas desde el cliente.

RecordatorioTarea

La entidad **RecordatorioTarea** representa una regla interna programada para generar una notificación relacionada con una tarea. No es la notificación visible para el usuario, sino la planificación que indica cuándo debe crearse o procesarse un aviso.

Sus atributos principales son:

- **idRecordatorioTarea**: identificador único del recordatorio.
- **usuario**: referencia al usuario propietario del recordatorio.
- **tarea**: referencia a la tarea asociada al recordatorio.
- **tipo**: tipo de recordatorio, representado por el enum TipoRecordatorioTarea.
- **activo**: indicador de si el recordatorio sigue pendiente de procesarse.
- **fechaProgramada**: fecha y hora en la que debe generarse el aviso.
- **fechaCreacion**: fecha y hora en la que se creó el recordatorio.
- **fechaActualizacion**: fecha de última modificación del recordatorio.
- **fechaProcesado**: fecha en la que el recordatorio fue procesado, si procede.
- **notificacionGenerada**: indicador de si el procesamiento del recordatorio llegó a generar una notificación.

Esta entidad permite separar la programación interna del aviso de la notificación visible que finalmente recibe el usuario.

Notificacion

La entidad **Notificacion** representa el mensaje visible que el usuario puede consultar desde la campanita interna de la aplicación y, si procede, recibir también mediante Web Push.

Sus atributos principales son:

- **idNotificacion**: identificador único de la notificación.
- **usuario**: usuario destinatario de la notificación.
- **tarea**: tarea relacionada con la notificación, si aplica.

- **grupo:** grupo relacionado con la notificación, si aplica.
- **asignacionGrupoMiembro:** asignación individual relacionada con la notificación, si aplica.
- **tipo:** tipo de notificación, representado por el enum `TipoNotificacion`.
- **titulo:** título breve de la notificación.
- **mensaje:** contenido principal mostrado al usuario.
- **fechaProgramada:** fecha prevista para la notificación, cuando existe.
- **fechaCreacion:** fecha en la que se crea la notificación.
- **cerrada:** indicador de si el usuario ha cerrado la notificación.
- **fechaCierre:** fecha en la que el usuario cerró la notificación.
- **pushEstado:** estado del envío Web Push, representado por el enum `EstadoPushNotificacion`.
- **fechaEnvioPush:** fecha en la que se intentó o realizó el envío push.
- **errorPush:** mensaje técnico breve si el envío push falló de forma controlada.

El sistema utiliza el concepto de notificación cerrada, no de notificación leída. La notificación permanece visible hasta que el usuario la cierra desde la interfaz.

PreferenciasNotificacion

La entidad **PreferenciasNotificacion** almacena la configuración de notificaciones de cada usuario.

Sus atributos principales son:

- **idPreferenciasNotificacion:** identificador único de las preferencias.
- **usuario:** usuario al que pertenecen las preferencias.
- **notificacionesActivas:** indica si el usuario quiere recibir notificaciones en general.
- **aviso24hActivo:** indica si están activos los avisos 24 horas antes del vencimiento.
- **prioridadesAviso24h:** conjunto de prioridades que deben tenerse en cuenta para los avisos 24 horas antes.
- **gruposAviso24h:** grupos incluidos en los avisos 24 horas antes para tareas con origen grupal.
- **recordatorioInteligenteActivo:** indica si el usuario permite el uso de recordatorios inteligentes.
- **notificarAsignacionesGrupoActivo:** indica si el usuario quiere recibir notificaciones por nuevas asignaciones de grupo.

Esta entidad permite que cada usuario tenga una configuración independiente sobre qué avisos quiere recibir y en qué condiciones.

PushSubscription

La entidad **PushSubscription** representa una suscripción Web Push registrada desde un navegador o dispositivo concreto del usuario.

Sus atributos principales son:

- **idPushSubscription**: identificador único de la suscripción.
- **usuario**: usuario propietario de la suscripción.
- **endpoint**: endpoint público de la suscripción push generado por el navegador.
- **p256dh**: clave pública de cifrado asociada a la suscripción.
- **auth**: secreto de autenticación de la suscripción.
- **userAgent**: información del navegador o cliente desde el que se creó la suscripción.
- **nombreDispositivo**: nombre descriptivo opcional del dispositivo.
- **activa**: indica si la suscripción sigue activa.
- **fechaAlta**: fecha en la que se registró la suscripción.
- **fechaUltimoUso**: fecha del último uso o actualización de la suscripción.
- **fechaBaja**: fecha en la que se desactivó la suscripción, si procede.

Las suscripciones push no se crean a partir de una lista fija de navegadores, sino cuando el usuario activa las notificaciones del dispositivo desde un navegador compatible. La clave privada VAPID no se almacena en esta entidad ni se expone al frontend.

4.3. Diseño de la API

La aplicación **Gestor de Tareas** expone su funcionalidad principal a través de una **API REST** desarrollada con Spring Boot. Esta API actúa como punto de acceso único para los clientes, actualmente la aplicación web en React/PWA, y se ha diseñado siguiendo convenciones habituales en servicios REST: uso de verbos HTTP estándar, rutas organizadas por recursos y datos en formato JSON.

La API no se limita a operaciones CRUD básicas, sino que agrupa funcionalidades de autenticación, gestión de tareas, categorías, grupos, asignaciones, filtros avanzados, ordenación inteligente, notificaciones, suscripciones Web Push y comprobación de disponibilidad del backend. Esto permite que el frontend pueda consumir todas las funciones de la aplicación desde una interfaz unificada, sin acceder directamente a la base de datos.

La mayoría de endpoints requieren autenticación mediante **JWT** en la cabecera Authorization. Solo permanecen públicos los endpoints necesarios para registro, login, renovación de sesión cuando corresponde, documentación y comprobación básica de

disponibilidad. En las operaciones protegidas, el backend valida tanto la identidad del usuario como la propiedad o permisos sobre los recursos implicados.

Las respuestas de la API se devuelven en formato JSON mediante DTOs específicos, evitando exponer directamente las entidades internas del modelo. Además, el sistema utiliza un formato homogéneo de errores para facilitar el consumo desde el frontend y la depuración durante el desarrollo.

4.3.1. Recursos principales y organización de rutas

La API se organiza en torno a varios recursos principales, cada uno gestionado por uno o varios controladores específicos en el backend. En la versión actual del proyecto, la organización de rutas ya no se limita al bloque clásico de usuarios, tareas y categorías, sino que incorpora también recursos de colaboración mediante grupos, asignaciones de tareas, persistencia del estado del filtro avanzado, ordenación inteligente, notificaciones, suscripciones Web Push y comprobación de disponibilidad del backend.

- **/api/tarea (TareaController)**

Agrupar las operaciones relacionadas con la gestión de tareas del usuario autenticado. Incluye endpoints para crear nuevas tareas, consultarlas, obtener una tarea concreta por su identificador, modificarlas, marcarlas como completadas y eliminarlas. Además, incorpora operaciones de ordenación y filtrado individual por distintos criterios, vistas especializadas como tareas del día, tareas próximas a vencer y tareas vencidas, y endpoints para consultar tareas asignadas desde grupos.

Dentro de este mismo bloque se incluye también el **filtro combinado**, que permite aplicar múltiples criterios sobre una lista unificada de tareas personales y tareas con origen grupal. El recurso de tareas incluye además el guardado y recuperación del último estado persistido del filtro, la vista de **tareas recomendadas** basada en la ordenación inteligente y la activación o desactivación del **recordatorio inteligente** asociado a una tarea.

- **/api/categoria (CategoriaController)**

Recoge las operaciones de gestión de categorías. Permite listar todas las categorías del usuario autenticado, buscarlas por nombre parcial, crear nuevas categorías, modificarlas y eliminarlas, respetando en todo momento tanto la propiedad de los recursos como las restricciones asociadas a las categorías protegidas.

- **/api/usuario (UsuarioController)**

Agrupar las operaciones relacionadas con la cuenta de usuario. Incluye el registro de nuevos usuarios, el inicio de sesión, la renovación del access token mediante refresh token y las operaciones sobre la cuenta autenticada, en particular la consulta de la información del propio usuario y la eliminación de su cuenta. A diferencia de fases anteriores del proyecto, este bloque ya no expone endpoints públicos de listado arbitrario de usuarios.

- **/api/grupo (GrupoController)**

Agrupar las operaciones propias del sistema de grupos. Incluye la creación de grupos, su consulta, modificación, activación o inactivación, eliminación, gestión de miembros, cambio de roles, transferencia de ownership, salida de un grupo e incorporación mediante código de invitación. También incluye los endpoints para consultar y regenerar el código de invitación cuando corresponde según los permisos del usuario autenticado dentro del grupo.

- **/api/grupo (AsignacionGrupoController)**

Sobre el mismo prefijo de recurso de grupo, este controlador reúne las operaciones relacionadas con las asignaciones de tareas dentro de grupos. Incluye la creación de asignaciones por duplicación individual, el listado de asignaciones de un grupo, la consulta del detalle de una asignación concreta y las operaciones de validación y reapertura de entregas individuales.

- **/api/notificaciones (NotificacionController)**

Agrupar las operaciones relacionadas con el sistema de notificaciones del usuario autenticado. Incluye endpoints para listar notificaciones activas, consultar el contador de notificaciones pendientes, cerrar una notificación concreta, cerrar todas las notificaciones, obtener y actualizar preferencias de notificación, registrar una suscripción Web Push y desactivar una suscripción existente.

Estas rutas permiten que el frontend muestre la campanita interna, gestione preferencias de avisos, active notificaciones del dispositivo y sincronice el estado de las notificaciones visibles para el usuario.

- **/api/health (HealthController)**

Exponer un endpoint público de comprobación de disponibilidad del backend. Su finalidad es comprobar que el servicio está levantado sin requerir autenticación, sin consultar la base de datos y sin exponer información sensible. Este endpoint se utiliza también como apoyo para el sistema de self-ping del despliegue en Render.

Las rutas se nombran de forma descriptiva y se apoyan en los verbos HTTP para indicar la operación a realizar. Así, se utilizan **GET** para consultar recursos o vistas específicas,

POST para crear entidades o ejecutar operaciones como login, refresh, creación de asignaciones o filtrado combinado, **PUT** para actualizaciones completas o guardado de estado, **PATCH** para acciones puntuales sobre un recurso, como completar tareas, validar entregas, cerrar notificaciones o cambiar ciertos estados, y **DELETE** para eliminar recursos o desactivar elementos asociados al usuario.

Además de los endpoints funcionales, existe un endpoint de **health check** orientado a comprobar la disponibilidad del backend. Los procesos internos como el scheduler de notificaciones, el aviso 24 horas antes, el envío Web Push o el self-ping no se exponen como endpoints funcionales propios, sino que forman parte de la lógica interna del backend.

4.3.2. Autenticación, autorización y seguridad

La mayoría de los endpoints de la API requieren que el usuario esté autenticado. En la versión actual del proyecto, la autenticación se apoya en un esquema basado en **JWT** con dos tipos de token: un **access token**, utilizado para acceder a los endpoints protegidos, y un **refresh token**, utilizado para renovar la sesión sin obligar al usuario a introducir de nuevo sus credenciales cada vez que caduca el token de acceso.

El proceso de autenticación comienza en el endpoint **POST /api/usuario/login**. El cliente envía el correo electrónico y la contraseña del usuario y, si las credenciales son correctas, el backend genera un access token y un refresh token. A partir de ese momento, el frontend incluye el access token en la cabecera **Authorization** con el formato Bearer <token> para consumir los endpoints protegidos.

Cuando el access token caduca, el frontend puede solicitar uno nuevo mediante el endpoint **POST /api/usuario/refresh**, siempre que disponga de un refresh token válido. Este mecanismo permite mantener la sesión activa durante el uso normal de la aplicación, evitando que el usuario tenga que iniciar sesión repetidamente.

Los endpoints públicos del sistema son los estrictamente necesarios para el funcionamiento básico: registro, login, refresh de sesión, documentación de la API y comprobación de disponibilidad mediante **GET /api/health**. El resto de operaciones relacionadas con tareas, categorías, grupos, asignaciones, filtros guardados, notificaciones, preferencias y suscripciones push requieren autenticación.

Además de la autenticación general, la aplicación aplica reglas de **autorización** a nivel de lógica de negocio. En las operaciones sobre tareas, categorías, estado guardado del filtro, notificaciones, recordatorios y suscripciones push, el backend comprueba que el

recurso pertenezca al usuario autenticado antes de permitir su consulta, modificación o eliminación.

En el sistema de grupos, la autorización se basa en roles de negocio propios del grupo: **creador**, **admin** y **miembro**. Estos roles determinan qué acciones puede realizar cada usuario, como gestionar miembros, cambiar roles, transferir la propiedad, crear asignaciones, validar entregas o reabrir tareas. De esta forma, no existe un rol global de administrador de toda la aplicación, sino permisos contextuales dentro de cada grupo.

Las notificaciones y las suscripciones Web Push también respetan estas reglas de seguridad. Un usuario solo puede consultar y cerrar sus propias notificaciones, modificar sus propias preferencias y registrar o desactivar sus propias suscripciones push. El backend no devuelve información sensible de las suscripciones, como claves completas o datos privados innecesarios, y la clave privada VAPID se mantiene exclusivamente en el backend mediante variables de entorno.

A diferencia de fases anteriores del proyecto, el bloque de usuario ya no expone endpoints públicos de listado o consulta arbitraria de otros usuarios. Las operaciones de usuario se centran en la cuenta autenticada mediante endpoints como `GET /api/usuario/me` y `DELETE /api/usuario/me`, reduciendo así la superficie de exposición innecesaria del sistema.

Para limitar el acceso desde aplicaciones no autorizadas, la configuración de **CORS** restringe los orígenes permitidos. En el estado actual del proyecto se contempla el dominio de producción del frontend desplegado en Vercel y el entorno local de desarrollo. En producción, además, la comunicación entre cliente y servidor se realiza mediante **HTTPS**.

En conjunto, la seguridad del sistema se basa en una combinación de autenticación stateless mediante JWT, renovación controlada de sesión mediante refresh token, autorización contextual sobre recursos propios, roles de grupo, control de orígenes permitidos y comunicación cifrada en el entorno desplegado.

4.3.3. Formato de datos y manejo de errores

La API utiliza **JSON** como formato de intercambio de datos tanto en las peticiones como en las respuestas. En operaciones como creación, actualización, autenticación, filtrado o configuración de preferencias, el cliente envía objetos JSON que se mapean en el backend a clases de petición o **DTOs** específicos. Sobre estos DTOs se aplican validaciones antes de ejecutar la lógica de negocio.

Las respuestas exitosas también se devuelven en formato JSON. Para evitar exponer directamente las entidades internas del modelo, el backend utiliza DTOs de respuesta adaptados a cada caso. Existen DTOs para usuarios, tareas, categorías, grupos, asignaciones, filtro combinado, estado guardado del filtro, notificaciones, preferencias de notificación, recordatorios inteligentes y suscripciones push.

En algunos casos, las respuestas incluyen información enriquecida para facilitar el trabajo del frontend. Por ejemplo, el filtro combinado puede devolver datos de la tarea junto con información de categoría, origen grupal y estado de revisión. Del mismo modo, las respuestas de notificaciones permiten mostrar en la interfaz mensajes activos, contador de pendientes, preferencias del usuario y estado de suscripciones sin exponer datos sensibles innecesarios.

El tratamiento de errores se centraliza mediante un manejador global de excepciones, `GlobalExceptionHandler`, encargado de transformar excepciones de validación, seguridad o negocio en respuestas HTTP homogéneas y comprensibles para el cliente. Esta estrategia permite que el frontend interprete de forma consistente tanto el código HTTP como el cuerpo de la respuesta.

Para los errores simples, la API devuelve un cuerpo con el siguiente formato general:

```
{
  "status": 400,
  "error": "Mensaje breve."
}
```

Cuando el fallo corresponde a validación detallada por campos, la respuesta añade además un bloque `errors` con el detalle de cada campo invalidado:

```
{
  "status": 400,
  "error": "Datos de entrada no válidos.",
  "errors": {
    "campo": "mensaje de validación"
  }
}
```

El sistema distingue entre distintos tipos de error y utiliza códigos HTTP adecuados según el caso. Entre los más habituales se encuentran errores de validación de entrada (400), acceso no autenticado (401), acceso prohibido o falta de permisos (403), recursos no encontrados (404), conflictos de estado o duplicados (409) y errores internos no controlados (500).

Este formato homogéneo se aplica a casos como JSON mal formado, credenciales incorrectas, token inválido o expirado, refresh token inválido, acceso a recursos ajenos,

conflictos por duplicidad, categorías protegidas, combinaciones incoherentes de filtros, estados no permitidos en asignaciones grupales, notificaciones inexistentes, grupos no pertenecientes al usuario, tareas sin fecha para activar recordatorio inteligente o datos inválidos de una suscripción push.

En el sistema de notificaciones y Web Push se diferencia entre errores visibles para el cliente y fallos internos controlados. Por ejemplo, si un usuario intenta cerrar una notificación ajena o activar un recordatorio sobre una tarea que no le pertenece, el backend responde con un error HTTP visible. En cambio, si falla un envío Web Push por una suscripción caducada o por un problema del proveedor push, la notificación interna sigue existiendo y el fallo se trata como un error técnico controlado, sin romper el flujo principal del usuario.

En conjunto, esta estrategia de formato de datos y manejo de errores facilita el consumo de la API desde el frontend, mejora la depuración durante el desarrollo y mantiene una base consistente para seguir ampliando la aplicación.

4.3.4. Documentación y pruebas de la API

La API está documentada mediante **OpenAPI/Swagger**. La configuración de Swagger define la información básica del servicio y permite generar automáticamente una interfaz interactiva desde la que se pueden consultar los endpoints disponibles, los parámetros de cada operación, los modelos de datos utilizados y los posibles códigos de respuesta.

Esta documentación resulta especialmente útil durante el desarrollo, ya que permite revisar de forma rápida la estructura general de la API y comprobar qué rutas están disponibles en cada bloque funcional. En la versión actual, Swagger recoge endpoints de usuarios, tareas, categorías, grupos, asignaciones, filtro combinado, tareas recomendadas, notificaciones, suscripciones push y health check.

Además de Swagger, durante el desarrollo se ha utilizado **Postman** como herramienta principal de validación manual de la API. Con Postman se han probado los flujos principales del backend, incluyendo autenticación, renovación de token, operaciones CRUD, filtros, gestión de grupos, asignaciones, validación y reapertura de entregas, ordenación inteligente, notificaciones, recordatorios, Web Push y comprobación de disponibilidad del backend.

El proyecto también dispone de una base de tests automáticos unitarios y de flujo creada durante fases anteriores del desarrollo. Sin embargo, una parte de esos tests ha quedado desactualizada debido a la evolución del backend, especialmente tras los

cambios en autenticación, endpoints de usuario, grupos, filtros y notificaciones. Por este motivo, en el estado actual del proyecto la validación funcional más fiable se ha realizado mediante pruebas manuales exhaustivas con Postman y pruebas directas desde el frontend.

La situación de los tests automáticos se detalla más adelante en el capítulo de pruebas. Aun así, Swagger y Postman han sido herramientas clave para verificar el comportamiento real de la API, comprobar respuestas correctas, validar errores controlados y asegurar que los endpoints protegidos requieren autenticación mediante JWT.

4.4. Diseño de la interfaz de usuario

La interfaz de usuario de **Gestor de Tareas** se ha diseñado como una aplicación web moderna desarrollada con **React** y **Vite**, configurada además como **PWA** para poder instalarse desde navegadores compatibles. El objetivo principal del diseño es que el usuario pueda gestionar tareas, categorías, grupos, filtros y notificaciones desde una misma experiencia visual, sin depender de pantallas aisladas o flujos excesivamente complejos.

La aplicación se organiza alrededor de un **dashboard principal**, que actúa como centro de trabajo del usuario autenticado. Desde esta pantalla se accede a las vistas principales de tareas, categorías, grupos, vista inteligente y resultados filtrados. El diseño busca mantener una navegación clara y rápida, permitiendo cambiar de contexto sin perder la sensación de estar trabajando dentro de una misma aplicación.

Las tareas se muestran mediante tarjetas visuales reutilizables. Cada tarjeta resume la información más importante de la tarea, como título, prioridad, estado, categoría, fecha de entrega, tiempo estimado y origen personal o grupal. Cuando el usuario necesita más información, puede desplegar la tarjeta para consultar detalles adicionales y acceder a acciones como completar, editar, eliminar, entregar una tarea grupal o activar el recordatorio inteligente cuando corresponde.

La gestión de **categorías** y **grupos** también se representa mediante tarjetas. Las categorías muestran su color, icono, nombre y estado protegido cuando son categorías base. Los grupos muestran información visual propia, miembros, rol del usuario autenticado y acciones disponibles según permisos. Esta aproximación mantiene coherencia entre las distintas secciones de la interfaz y facilita que el usuario reconozca rápidamente cada tipo de recurso.

Uno de los elementos principales de la interfaz es el **filtro avanzado** de tareas. Este filtro permite combinar criterios como origen, grupo, categoría, prioridades, estados, palabras clave, tiempo máximo, fechas y criterio de ordenación. El resultado se muestra como una vista propia dentro del dashboard, llamada resultados filtrados, sin obligar al usuario a abandonar la pantalla principal. Además, el estado del filtro se guarda en servidor para poder restaurarlo en sesiones posteriores.

La aplicación incluye también una vista **Inteligente**, conectada con el backend, que muestra tareas recomendadas mediante el algoritmo de ordenación inteligente. Desde el punto de vista de la interfaz, esta vista se presenta como una forma rápida de consultar qué tareas conviene atender antes, sin mostrar al usuario la puntuación interna calculada por el sistema.

El sistema de **notificaciones** se integra en la interfaz mediante una campanita situada en la zona superior. Desde ella el usuario puede consultar notificaciones activas, cerrar una notificación, cerrar todas, modificar preferencias de notificación, activar o desactivar avisos 24 horas antes del vencimiento, configurar prioridades y grupos incluidos, gestionar recordatorios inteligentes y activar las notificaciones del dispositivo mediante Web Push cuando el navegador lo permite.

Las pantallas de **login y registro** mantienen una identidad visual coherente con el resto de la aplicación. En escritorio se utilizan diseños más amplios y decorativos, mientras que en móvil se prioriza el acceso rápido al formulario sin perder elementos de marca. La autenticación visual se combina con la lógica real de sesión del frontend, incluyendo almacenamiento de tokens, rutas protegidas y renovación automática del access token.

El diseño visual se apoya en estilos centralizados, tokens de color, modo claro/oscuro y componentes reutilizables. Esta decisión permite mantener una estética coherente en botones, tarjetas, inputs, modales, estados, prioridades, categorías y fondos, evitando repartir colores o estilos sueltos por muchos archivos.

La interfaz se ha adaptado también a **dispositivos móviles**. En pantallas pequeñas, las tarjetas reducen su estructura lateral para ganar espacio útil, los modales se adaptan al ancho disponible, el panel de vistas pasa a comportarse como elemento flotante, el panel de notificaciones se muestra como modal y el filtro avanzado se compacta. También se optimizaron los fondos repetidos de tarjetas en móvil utilizando imágenes WebP para reducir el coste visual en listas largas.

En conjunto, la interfaz actual permite utilizar la aplicación tanto desde escritorio como desde móvil, manteniendo las funciones principales accesibles: gestión de tareas, categorías, grupos, asignaciones, filtros avanzados, vista inteligente, notificaciones, autenticación y configuración básica de la experiencia PWA.

5. Implementación

5.1. Backend (Spring Boot)

El backend de **Gestor de Tareas** se ha implementado utilizando **Spring Boot** y una arquitectura por capas que separa claramente la exposición de la API REST, la lógica de negocio, la seguridad, el acceso a datos y la configuración transversal de la aplicación. Esta estructura ha permitido evolucionar el proyecto desde un núcleo inicial de usuarios, tareas y categorías hasta una API más completa con grupos, asignaciones, filtro avanzado, ordenación inteligente, notificaciones, recordatorios y despliegue en la nube.

La implementación del backend se apoya en **Spring Web** para exponer endpoints HTTP, **Spring Data JPA** y **Hibernate** para trabajar con la base de datos PostgreSQL, **Spring Security** para proteger los endpoints mediante JWT, validaciones declarativas sobre DTOs y un manejador global de errores. Además, se han incorporado procesos programados con Spring para la generación de avisos y recordatorios, así como servicios específicos para Web Push y comprobación de disponibilidad del backend.

El objetivo de esta organización es mantener una base estructurada, mantenible y ampliable, donde los controladores actúan como punto de entrada, los servicios concentran la lógica de negocio, los repositorios aíslan el acceso a datos y las capas de seguridad, configuración y excepciones resuelven aspectos comunes de toda la aplicación.

5.1.1. Estructura de paquetes

El código del backend se organiza en varios paquetes principales dentro de `com.tugestor.gestortareas`. Esta estructura permite separar responsabilidades y mantener el proyecto ordenado a medida que se han ido incorporando nuevas funcionalidades.

- **controller**

Contiene los controladores REST que exponen la API bajo las rutas `/api/...`. Estos controladores reciben las peticiones HTTP, validan los datos de entrada cuando procede y delegan la lógica de negocio en los servicios correspondientes.

Ejemplos de clases de este paquete son `UsuarioController`, `TareaController`, `CategoriaController`, `GrupoController`, `AsignacionGrupoController`, `NotificacionController` y `HealthController`.

- **service**

Agrupar las interfaces de servicio y sus implementaciones. En esta capa reside la lógica de negocio principal del sistema: registro de usuarios, gestión de tareas, categorías, grupos, asignaciones, filtro avanzado, ordenación inteligente, notificaciones, recordatorios y Web Push.

Ejemplos de clases de este paquete son `UsuarioService`, `UsuarioServiceImpl`, `TareaService`, `TareaServiceImpl`, `CategoriaService`, `CategoriaServiceImpl`, `GrupoService`, `GrupoServiceImpl`, `AsignacionGrupoService`, `AsignacionGrupoServiceImpl`, `NotificacionService`, `RecordatorioTareaService` y `WebPushService`.

- **service.scoring**

Contiene la lógica específica de cálculo y ranking usada por la ordenación inteligente de tareas. Se separa del servicio principal de tareas para mantener la fórmula aislada, reutilizable y más fácil de ajustar.

Ejemplos de clases de este paquete son `TareaScoringContext`, `TareaInteligenteScorer` y `TareaInteligenteRankingService`.

- **repository**

Contiene los repositorios de Spring Data JPA encargados del acceso a la base de datos. Estos repositorios permiten consultar y persistir entidades sin escribir manualmente todo el SQL necesario para las operaciones habituales.

Ejemplos de clases de este paquete son `UsuarioRepository`, `TareaRepository`, `CategoriaRepository`, `GrupoRepository`, `GrupoMiembroRepository`, `AsignacionGrupoRepository`, `AsignacionGrupoMiembroRepository`, `EstadoFiltroTareaRepository`, `NotificacionRepository`, `RecordatorioTareaRepository`, `PreferenciasNotificacionRepository` y `PushSubscripcionRepository`.

- **model**

Incluye las entidades JPA y los tipos enumerados que representan el modelo de dominio de la aplicación. Estas clases se mapean a tablas de la base de datos y definen relaciones entre usuarios, tareas, categorías, grupos, asignaciones, filtros guardados y notificaciones.

Ejemplos de entidades de este paquete son `Usuario`, `Tarea`, `Categoria`, `Grupo`, `GrupoMiembro`, `AsignacionGrupo`, `AsignacionGrupoMiembro`, `EstadoFiltroTarea`, `Notificacion`, `RecordatorioTarea`, `PreferenciasNotificacion` y `PushSubscripcion`.

También incluye enums como `Prioridad`, `Estado`, `RolGrupo`, `TipoAsignacionGrupo`, `EstadoRevisionAsignacion`, `OrigenTareaFiltro`, `CriterioOrdenTareaCombinado`, `TipoNotificacion`, `TipoRecordatorioTarea` y `EstadoPushNotificacion`.

- **dto**

Contiene los objetos de transferencia de datos utilizados en las peticiones y respuestas de la API. Estos DTOs permiten separar la representación interna de las entidades del formato público que consume el frontend. También incorporan validaciones declarativas para controlar campos obligatorios, formatos, tamaños y coherencia básica de entrada.

Ejemplos de clases de este paquete son `LoginRequest`, `LoginResponse`, `RefreshTokenRequest`, `AccessTokenResponse`, `UsuarioRequest`, `UsuarioResponse`, `TareaRequest`, `TareaResponse`, `TareaFiltroCombinadoResponse`, `FiltroTareaCombinadoRequest`, `EstadoFiltroTareaResponse`, `CategoriaRequest`, `CategoriaResponse`, `GrupoRequest`, `GrupoResponse`, `GrupoMiembroResponse`, `AsignacionGrupoRequest`, `AsignacionGrupoResponse`, `NotificacionResponse`, `PreferenciasNotificacionRequest`, `PreferenciasNotificacionResponse`, `PushSubscripcionRequest` y `RecordatorioInteligenteRequest`.

- **exception**

Agrupar las excepciones personalizadas del proyecto y el manejador global de errores. Esta capa permite convertir errores internos, errores de validación y errores de negocio en respuestas HTTP homogéneas para el cliente.

Ejemplos de clases de este paquete son `GlobalExceptionHandler`, `ErrorResponse`, `EmailDuplicadoException` y `CategoriaProtegidaException`.

- **security**

Contiene los componentes relacionados con la autenticación y autorización. Aquí se gestiona el filtro JWT, la generación y validación de tokens, la carga de usuarios para Spring Security y las reglas generales de acceso a endpoints protegidos.

Ejemplos de clases de este paquete son `JwtAuthenticationFilter`, `JwtService` y `CustomUserDetailsService`.

- **config**

Incluye clases de configuración transversal de la aplicación. En este paquete se centralizan configuraciones de seguridad, CORS, Swagger/OpenAPI, reloj del

sistema, inicialización de datos, soporte de SPA y comportamiento relacionado con el despliegue.

Ejemplos de clases de este paquete son `SeguridadConfig`, `CorsConfig`, `SwaggerConfig`, `ClockConfig`, `DataSeeder` y `SpaRedirectConfig`.

- **resources**

Contiene archivos de configuración y recursos estáticos. Aquí se encuentran ficheros como `application.properties`, `application-prod.properties`, ejemplos de configuración, plantillas de variables y, en versiones anteriores del despliegue, recursos estáticos generados del frontend.

Esta organización por paquetes refleja la separación de responsabilidades del backend y facilita la localización de cada parte del sistema. Además, permite que nuevas funcionalidades, como notificaciones, filtros avanzados u ordenación inteligente, se integren sin romper la estructura general del proyecto.

5.1.2. Servicios principales

La lógica de negocio del backend se concentra en la capa de servicios, que actúa como intermediaria entre los controladores y los repositorios. En esta capa se resuelven tanto las operaciones CRUD básicas como las reglas de propiedad, validación, seguridad contextual y comportamiento específico del dominio.

- **Servicio de usuarios (UsuarioService)**

Se encarga de las operaciones relacionadas con la gestión de usuarios y autenticación:

- Registro de nuevos usuarios a partir de los datos recibidos en los DTOs de petición.
- Validación de unicidad del correo electrónico y gestión del conflicto cuando se intenta registrar una cuenta con un email ya existente.
- Cifrado de contraseñas antes de almacenarlas en la base de datos.
- Apoyo al flujo de autenticación basado en **JWT**, incluyendo el modelo actual de access token y refresh token.
- Creación automática de categorías base protegidas al registrar una nueva cuenta de usuario.
- Obtención de la información de la cuenta del usuario autenticado a partir del contexto de seguridad.
- Eliminación de la propia cuenta del usuario autenticado, resolviendo previamente dependencias relacionadas con tareas, categorías, grupos, filtros guardados y otros datos asociados.

- **Servicio de categorías (CategoriaService)**

Gestiona la creación y mantenimiento de categorías de tareas:

- Creación de nuevas categorías con nombre, color e icono.
- Listado de todas las categorías disponibles para el usuario autenticado.
- Búsqueda de categorías por nombre parcial.
- Actualización de categorías existentes, respetando la propiedad del recurso.
- Eliminación de categorías no protegidas, manteniendo las tareas asociadas y dejándolas sin categoría cuando proceda.
- Aplicación de reglas específicas sobre categorías protegidas y sobre categorías ajenas al usuario autenticado.

- **Servicio de tareas (TareaService)**

Implementa la lógica relacionada con la gestión individual de tareas y con la consulta avanzada de las mismas:

- Creación de nuevas tareas asociadas al usuario autenticado.
- Obtención de la lista de tareas del usuario y consulta de tareas concretas por identificador.
- Actualización y eliminación de tareas, aplicando siempre las reglas de propiedad del recurso.
- Marcado de tareas como completadas y registro de la fecha de completado y del usuario que realiza la acción.
- Aplicación de ordenaciones y filtros individuales sobre las tareas.
- Obtención de vistas especializadas como tareas del día, tareas próximas a vencer y tareas vencidas.
- Cálculo del estado lógico de cada tarea a partir de su fecha de entrega, fecha de completado, indicador de completada y fecha actual.
- Consulta específica de tareas con origen en grupos.
- Construcción y aplicación del **filtro combinado**, que mezcla tareas personales y tareas con origen grupal, soporta múltiples criterios de filtrado y devuelve una respuesta enriquecida con datos de categoría, origen y revisión.
- Guardado y recuperación del último estado persistido del filtro combinado por usuario.
- Integración con la **ordenación inteligente**, tanto desde el filtro combinado como desde el endpoint dedicado de tareas recomendadas.
- Activación y desactivación del **recordatorio inteligente** asociado a tareas personales con fecha de entrega.

- **Servicio de grupos (GrupoService)**

Gestiona la parte principal del sistema colaborativo basado en grupos:

- Creación de grupos y asignación automática del creador como miembro con rol ADMIN.
- Consulta, edición, activación o inactivación y eliminación de grupos.
- Gestión del ownership del grupo y de los permisos asociados.
- Gestión de miembros del grupo: alta, expulsión, cambio de rol, salida del grupo y transferencia de ownership.
- Gestión del código de invitación: consulta, regeneración y unión al grupo mediante código válido.

- **Servicio de asignaciones de grupo (AsignacionGrupoService)**

Implementa la lógica de reparto, seguimiento y revisión de tareas dentro del contexto colaborativo:

- Creación de asignaciones de tareas desde un grupo, tanto para todo el grupo como para una selección manual de destinatarios.
- Generación de tareas individuales reales para cada miembro destinatario mediante duplicación controlada.
- Mantenimiento de la trazabilidad entre la asignación grupal, cada destinatario y la tarea individual generada.
- Listado de asignaciones por grupo y consulta del detalle de una asignación concreta.
- Gestión del ciclo de revisión de entregas individuales, incluyendo estados como pendiente, entregada, validada o reabierta.
- Validación y reapertura de entregas por parte de admins del grupo, con soporte de comentario de revisión y reutilización de la misma tarea individual al reabrir.

- **Servicios de ordenación inteligente**

La ordenación inteligente se separa en componentes propios para evitar mezclar la fórmula de recomendación con el servicio general de tareas.

- `TareaScoringContext` prepara la información necesaria para calcular la puntuación de una tarea personal o de una tarea con origen grupal.
- `TareaInteligenteScorer` calcula la puntuación interna según los factores definidos para fecha, prioridad, tiempo estimado, antigüedad, categoría y estado de revisión.
- `TareaInteligenteRankingService` ordena las tareas según esa puntuación y aplica las reglas duras del algoritmo, dejando fuera tareas que no son accionables en una vista recomendada.

Este diseño permite reutilizar la misma lógica desde el filtro combinado y desde la vista específica de tareas recomendadas, sin duplicar la fórmula ni exponer el score interno al cliente.

- **Servicios de notificaciones y recordatorios**

El sistema de notificaciones se apoya en varios servicios encargados de generar, consultar y procesar avisos para el usuario:

- `NotificacionService` gestiona la creación, consulta y cierre de notificaciones internas.
- `RecordatorioTareaService` gestiona los recordatorios programados asociados a tareas, especialmente el recordatorio inteligente.
- Los procesos programados revisan periódicamente recordatorios vencidos y avisos 24 horas antes del vencimiento.
- Las notificaciones por asignaciones de grupo se generan cuando corresponde según las preferencias del usuario.

La notificación interna funciona como fuente principal del sistema: aunque el envío Web Push no sea posible o falle, la notificación puede seguir estando disponible en la campanita de la aplicación.

- **Servicio de Web Push (`WebPushService`)**

Gestiona el envío de notificaciones del sistema a navegadores o dispositivos que hayan registrado una suscripción push.

- Comprueba si Web Push está habilitado por configuración.
- Localiza las suscripciones activas del usuario destinatario.
- Construye el payload de la notificación.
- Intenta enviar la notificación mediante VAPID.
- Marca el resultado del envío como enviado, fallido o no aplicable.
- Controla fallos de suscripciones caducadas o inválidas sin romper la creación de la notificación interna.

- **Servicios auxiliares de disponibilidad y ejecución programada**

Además de los servicios funcionales principales, el backend incluye lógica auxiliar para mejorar el comportamiento en producción:

- El health check permite comprobar si el backend está levantado mediante `GET /api/health`.
- El self-ping opcional realiza llamadas periódicas al propio endpoint de health check para reducir el impacto del reposo automático de Render durante pruebas y demostraciones.

- La configuración de reloj permite centralizar ciertos cálculos temporales, especialmente en funcionalidades como ordenación inteligente, estados de tareas, avisos y recordatorios.

En todos estos servicios, la lógica que garantiza que un usuario solo puede acceder y modificar los recursos que le corresponden se implementa en la capa de servicio. De este modo, los controladores permanecen relativamente ligeros y las reglas de negocio se mantienen centralizadas, coherentes y más fáciles de evolucionar.

5.1.3. Gestión de errores y validaciones

La validación de datos y el tratamiento de errores se abordan de forma combinada mediante validaciones declarativas sobre DTOs, validaciones de negocio dentro de los servicios y un manejador global de excepciones que centraliza la construcción de respuestas HTTP coherentes para el cliente.

• Validaciones en DTOs

Los DTOs utilizados en las peticiones incorporan anotaciones de **Bean Validation** que definen restricciones sobre los datos de entrada. Estas validaciones permiten comprobar campos obligatorios, formatos, tamaños máximos, valores mínimos y coherencia básica antes de ejecutar la lógica de negocio.

Entre las anotaciones utilizadas se encuentran, por ejemplo, `@NotBlank`, `@NotNull`, `@Email`, `@Size`, `@Min` y otras restricciones aplicadas según el tipo de dato. Los controladores reciben los cuerpos de petición con `@Valid`, de modo que Spring valida automáticamente la entrada y evita que datos claramente incorrectos lleguen a la capa de servicio.

Este tipo de validación se aplica en operaciones como registro de usuarios, login, creación y edición de tareas, creación de categorías, creación de grupos, asignaciones, preferencias de notificación, recordatorios inteligentes, filtros avanzados y suscripciones push.

• Validaciones de negocio en servicios

Además de las validaciones declarativas, la capa de servicios aplica reglas de negocio que no dependen únicamente del formato de los datos. Estas reglas comprueban si una operación tiene sentido dentro del estado actual del sistema.

Algunos ejemplos son:

- Comprobar que un correo electrónico no esté ya registrado.

- Verificar que una tarea, categoría, grupo, notificación o suscripción pertenece al usuario autenticado.
- Impedir la eliminación de categorías base protegidas.
- Validar que un usuario tenga permisos suficientes dentro de un grupo para gestionar miembros, crear asignaciones, validar entregas o reabrir tareas.
- Evitar combinaciones incoherentes de filtros, como seleccionar a la vez una fecha exacta y una fecha límite hasta.
- Comprobar que una tarea tenga fecha de entrega antes de activar un recordatorio inteligente.
- Evitar activar un recordatorio inteligente cuando la fecha calculada ya ha vencido.
- Comprobar que los grupos seleccionados en preferencias de notificación pertenecen al usuario autenticado.

Estas comprobaciones se realizan en la capa de servicio para mantener los controladores ligeros y centralizar las reglas de negocio en un único lugar.

• Manejo global de errores

El backend utiliza un manejador global de excepciones, **GlobalExceptionHandler**, para transformar excepciones internas o de negocio en respuestas HTTP homogéneas. De esta forma, el frontend recibe siempre una estructura comprensible y puede mostrar mensajes adecuados al usuario.

Para errores simples, la respuesta tiene un formato general como:

```
{
  "status": 400,
  "error": "Mensaje breve."
}
```

Cuando el error corresponde a validación por campos, se añade un bloque errors con el detalle de cada campo:

```
{
  "status": 400,
  "error": "Datos de entrada no válidos.",
  "errors": {
    "campo": "mensaje de validación"
  }
}
```

Esta estructura permite diferenciar entre un error general de negocio y errores concretos de entrada introducidos por el usuario.

• Códigos HTTP utilizados

El sistema utiliza códigos HTTP según el tipo de error producido:

- **400 Bad Request** para datos inválidos, JSON mal formado o combinaciones incoherentes.
- **401 Unauthorized** cuando falta autenticación o el token no es válido.
- **403 Forbidden** cuando el usuario está autenticado pero no tiene permisos sobre el recurso.
- **404 Not Found** cuando el recurso solicitado no existe o no puede localizarse para el usuario actual.
- **409 Conflict** cuando existe un conflicto funcional, como duplicados o acciones bloqueadas por reglas de negocio.
- **500 Internal Server Error** para fallos internos no controlados.

La distinción entre estos códigos ayuda a que la API sea más predecible y facilita tanto la depuración como el consumo desde el frontend.

• Errores de seguridad y ownership

Un punto importante de la gestión de errores es la separación entre autenticación y autorización. Si el usuario no está autenticado, la API responde con un error de autenticación. Si el usuario está autenticado pero intenta operar sobre un recurso que no le pertenece o sobre el que no tiene permisos, el backend responde con un error de acceso prohibido o recurso no encontrado, según el caso.

Esta lógica se aplica a tareas, categorías, grupos, asignaciones, estado guardado del filtro, notificaciones, preferencias, recordatorios y suscripciones push. De esta forma se evita que un usuario pueda modificar o consultar datos de otro usuario.

• Errores en notificaciones y Web Push

El sistema diferencia entre errores visibles para el cliente y fallos internos controlados. Si el usuario intenta cerrar una notificación ajena, activar un recordatorio sobre una tarea que no le pertenece o registrar una suscripción push con datos inválidos, el backend devuelve un error HTTP visible.

En cambio, si una notificación interna se crea correctamente pero falla el envío Web Push por una suscripción caducada, un navegador no disponible o un problema del proveedor push, el fallo se trata de forma controlada. La notificación interna sigue existiendo en la campanita y el error técnico no rompe el flujo principal del usuario.

• Inventario de errores visibles

Durante la evolución del backend se mantiene un inventario auxiliar de respuestas de error visibles para el cliente. Cuando un bloque de backend añade, cambia o elimina mensajes de error visibles, también se actualiza este inventario para conservar la trazabilidad entre endpoints, códigos HTTP y mensajes devueltos.

Esta práctica ayuda a mantener coherente la documentación de la API, facilita las pruebas manuales con Postman y permite detectar si un cambio funcional afecta al contrato que consume el frontend.

5.1.4. Manejo de categorías protegidas y otras lógicas especiales

Además de las operaciones CRUD básicas, el backend incorpora varias reglas funcionales específicas que dan coherencia al comportamiento de la aplicación. Estas reglas se implementan principalmente en la capa de servicios, de forma que los controladores se limitan a recibir la petición y delegar la lógica correspondiente.

- **Categorías protegidas**

Cada usuario dispone de un conjunto de categorías base protegidas, creadas automáticamente al registrar la cuenta. Estas categorías sirven como punto de partida para organizar tareas y no pueden eliminarse como una categoría normal.

El backend distingue entre categorías protegidas y categorías personalizadas mediante el campo `protegida`. Cuando una categoría está protegida, el sistema impide su eliminación y aplica restricciones sobre su modificación. Esta lógica evita que el usuario borre accidentalmente categorías base necesarias para mantener una organización inicial coherente.

Cuando se elimina una categoría personalizada, las tareas asociadas no se eliminan. En su lugar, el backend deja esas tareas sin categoría, manteniendo la integridad de la información y evitando pérdidas de datos.

- **Aislamiento de categorías por usuario**

En versiones anteriores del proyecto existía el riesgo de que las categorías base se compartieran entre usuarios. En la versión actual, tanto las categorías base como las personalizadas están asociadas al usuario correspondiente. Esto garantiza que cada cuenta tenga su propio conjunto de categorías y que un usuario no pueda modificar categorías de otro.

- **Asignaciones de grupo mediante duplicación individual**

El sistema colaborativo utiliza un modelo de asignaciones por duplicación individual. Cuando un admin crea una asignación dentro de un grupo, el backend genera una tarea real para cada destinatario. De esta forma, cada miembro trabaja sobre su propia tarea individual, pero el sistema mantiene la trazabilidad con la asignación original.

Este diseño simplifica el seguimiento de entregas, permite validar o reabrir el trabajo de cada miembro por separado y evita que varios usuarios modifiquen exactamente la misma tarea compartida. Las tareas grupales compartidas reales quedan como una posible mejora futura.

• Estados de revisión en asignaciones de grupo

Las asignaciones individuales tienen un estado de revisión propio. Esto permite distinguir si una tarea de grupo está pendiente, entregada, validada o reabierta. El backend controla qué cambios de estado son válidos y qué usuario puede realizarlos.

Por ejemplo, un miembro puede completar o entregar su tarea asignada, mientras que un admin del grupo puede validar la entrega o reabirla con un comentario de revisión. Esta lógica mantiene la trazabilidad del proceso colaborativo sin mezclarla con el estado general de una tarea personal.

• Filtro combinado y filtro avanzado

El backend incluye un filtro combinado capaz de mezclar tareas personales y tareas con origen en grupos en una misma respuesta. Este filtro acepta criterios opcionales como origen, grupo, categoría, prioridades, estados, palabras clave, tiempo máximo, fecha exacta, fecha hasta y criterio de ordenación.

El sistema también persiste por usuario el último estado utilizado del filtro. Esto permite que el frontend restaure los controles del filtro avanzado cuando el usuario vuelve a entrar en la aplicación o utiliza otro dispositivo.

Para mantener compatibilidad, el backend conserva campos simples como prioridad y estado, pero la versión actual del filtro avanzado trabaja principalmente con listas de prioridades y estados.

• Ordenación inteligente de tareas

La ordenación inteligente se implementa como una heurística determinista y explicable. El backend calcula internamente una puntuación a partir de factores como fecha de entrega, prioridad, tiempo estimado, antigüedad, categoría y estado de revisión grupal.

Esta puntuación no se devuelve al frontend. El cliente recibe únicamente la lista ordenada de tareas recomendadas. Además, antes de calcular el ranking, el sistema aplica reglas duras para excluir tareas que no son accionables, como tareas completadas, tareas completadas con retraso o tareas de grupo ya entregadas o validadas.

La lógica de scoring está separada en clases específicas, lo que permite ajustar la fórmula sin modificar los controladores ni duplicar cálculo en distintos endpoints.

• Recordatorio inteligente por tarea

El recordatorio inteligente permite generar un aviso automático para una tarea personal con fecha de entrega. La fecha programada se calcula restando a la fecha de entrega una hora más el tiempo estimado de la tarea.

Por ejemplo, si una tarea vence a las 18:00 y tiene un tiempo estimado de 30 minutos, el recordatorio se programa para las 16:30. De esta manera, el usuario recibe el aviso con margen suficiente para poder realizar la tarea antes del vencimiento.

El backend valida que la tarea pertenezca al usuario autenticado, que tenga fecha de entrega y que la fecha calculada del recordatorio no haya vencido ya.

• Notificaciones internas

La notificación interna es la fuente principal del sistema de avisos. Las notificaciones se almacenan en base de datos y se muestran al usuario desde la campanita de la interfaz.

Las notificaciones pueden generarse por diferentes motivos, como avisos 24 horas antes del vencimiento, recordatorios inteligentes vencidos o nuevas asignaciones de grupo. El usuario puede cerrar una notificación concreta o cerrar todas las notificaciones activas.

El sistema utiliza el concepto de notificación cerrada, no de notificación leída. Esto simplifica el comportamiento: una notificación permanece visible hasta que el usuario decide cerrarla.

• Preferencias de notificación

Cada usuario dispone de preferencias propias de notificación. Estas preferencias permiten activar o desactivar las notificaciones en general, los avisos 24 horas antes del vencimiento, el recordatorio inteligente, las notificaciones de asignaciones de grupo y los filtros de prioridad o grupo aplicables a ciertos avisos.

El backend valida que los grupos seleccionados en las preferencias pertenezcan al usuario autenticado. Así se evita que un usuario configure avisos sobre grupos a los que no pertenece.

- **Web Push como envío complementario**

El sistema puede enviar notificaciones Web Push cuando el usuario ha activado las notificaciones del dispositivo y el navegador ha registrado una suscripción válida. Este envío es complementario a la notificación interna.

Si el envío Web Push falla, la notificación interna no se pierde. El backend registra el fallo de forma controlada y mantiene la notificación disponible en la campanita. Esta decisión evita que un problema técnico del navegador, del proveedor push o de una suscripción caducada rompa el flujo principal del usuario.

- **Procesos programados**

El backend utiliza procesos programados de Spring para revisar periódicamente recordatorios inteligentes vencidos y avisos 24 horas antes del vencimiento. Estos procesos se ejecutan en segundo plano sin que el usuario tenga que abrir una pantalla concreta o lanzar manualmente una acción.

También existe un proceso opcional de self-ping en producción, utilizado para llamar periódicamente al endpoint de health check y reducir el impacto del reposo automático del backend en Render durante pruebas y demostraciones.

Estas lógicas especiales permiten que la aplicación vaya más allá de un CRUD básico, incorporando reglas reales de negocio, automatización, colaboración, persistencia de preferencias y comportamiento adaptado al entorno de despliegue.

5.2. Frontend web (React / PWA)

El frontend actual de **Gestor de Tareas** se ha implementado como una aplicación web de una sola página utilizando **React** y **Vite**. Esta interfaz consume la API REST del backend y permite al usuario interactuar con las funcionalidades principales del sistema desde el navegador. Además, el frontend está configurado como **PWA**, por lo que puede instalarse desde navegadores compatibles y utilizarse de forma similar a una aplicación ligera de escritorio o móvil.

El código del frontend se organiza por módulos y funcionalidades. Existen carpetas para la configuración general de la aplicación, clientes de API, componentes reutilizables,

layout principal, autenticación, dashboard, tareas, categorías, grupos, filtros y notificaciones. Esta estructura facilita localizar cada parte de la interfaz y evita que toda la lógica visual quede concentrada en un único archivo.

La comunicación con el backend se realiza mediante clientes API específicos. Estos clientes centralizan las llamadas HTTP y utilizan el access token almacenado tras el login para enviar la cabecera **Authorization** con el formato Bearer <token>. El frontend también gestiona la renovación automática de sesión mediante el refresh token cuando el access token caduca. Si la renovación no es posible, se limpia la sesión y el usuario debe volver a iniciar sesión.

Las pantallas de **login** y **registro** están conectadas con los endpoints reales de autenticación y alta de usuario. El frontend valida la información introducida, muestra errores cuando corresponde y, tras un inicio de sesión correcto, permite acceder al dashboard protegido. También se incluye una interfaz visual propia para autenticación, con formularios reutilizables, mostrar/ocultar contraseña y diseño adaptado a escritorio y móvil.

El **dashboard principal** actúa como centro de trabajo de la aplicación. Desde él se accede a las vistas de Mis tareas, Inteligente, Mis categorías y Mis grupos, además de a los resultados filtrados cuando se aplican filtros avanzados. El usuario puede cambiar entre vistas sin abandonar la experiencia principal de la aplicación.

La vista **Mis tareas** carga las tareas reales del usuario autenticado desde el backend. Las tareas se muestran mediante tarjetas expandibles que resumen título, prioridad, estado, categoría, fecha de entrega, tiempo estimado y origen. Desde estas tarjetas se pueden realizar acciones como completar, editar, borrar, entregar tareas con origen grupal o activar el recordatorio inteligente cuando se cumplen las condiciones necesarias.

La vista **Inteligente** consume el endpoint de tareas recomendadas del backend. El frontend no calcula la puntuación de ordenación, sino que recibe la lista ya ordenada por el backend y la representa con el mismo sistema visual de tarjetas. De esta forma, la lógica de recomendación se mantiene centralizada en el servidor y el cliente se limita a mostrar el resultado de forma clara.

La vista **Mis categorías** permite consultar, crear, editar y eliminar categorías. Las categorías base protegidas se muestran sin acciones de edición o borrado cuando corresponde, mientras que las categorías personalizadas pueden gestionarse desde la interfaz. Las categorías utilizan color e icono para mejorar su identificación visual dentro de tareas y filtros.

La vista **Mis grupos** permite gestionar los grupos del usuario. Desde esta sección se puede crear un grupo, unirse mediante código de invitación, consultar miembros, ver o

regenerar códigos, editar datos del grupo, salir, eliminar y realizar acciones según el rol del usuario. También se integran los flujos de asignaciones de grupo, permitiendo al admin revisar entregas, validar tareas o reabrir las con comentario.

El frontend incorpora un **filtro avanzado** conectado al backend. Este filtro permite combinar criterios como origen, grupo, categoría, prioridades, estados, palabras clave, tiempo máximo, fecha exacta, fecha hasta y ordenación. Al aplicar filtros, el frontend construye un body limpio, llama al endpoint de filtro combinado, muestra los resultados como una vista propia y guarda automáticamente el estado del filtro en servidor. Al limpiar filtros, vuelve a Mis tareas y guarda también el estado limpio.

El buscador rápido local funciona como una segunda capa de búsqueda sobre la vista activa. En Mis tareas, Inteligente y Resultados filtrados permite reducir visualmente la lista mostrada sin lanzar una nueva consulta al backend. Este buscador normaliza mayúsculas y acentos para facilitar coincidencias sencillas.

El sistema de **notificaciones** se integra mediante una campanita en el dashboard. Desde ella el usuario puede consultar notificaciones activas, ver el contador de pendientes, cerrar una notificación, cerrar todas y configurar preferencias. El panel permite activar o desactivar notificaciones generales, avisos 24 horas antes, recordatorio inteligente, asignaciones de grupo, prioridades incluidas y grupos incluidos.

La parte PWA incluye un **service worker** propio y soporte para **Web Push**. Cuando el navegador lo permite y el usuario concede permisos, el frontend crea una suscripción push, extrae los datos públicos necesarios y los registra en el backend. Si posteriormente se genera una notificación compatible, el navegador puede mostrar una notificación del sistema. Al pulsar sobre ella, la aplicación se abre o se enfoca en /app.

Visualmente, el frontend utiliza estilos centralizados, modo claro/oscuro, tokens de color y componentes reutilizables. Las tarjetas, botones, inputs, chips, modales, prioridades, estados, categorías, grupos y paneles comparten una base visual coherente. Esto facilita mantener una identidad gráfica común y permite modificar la estética general sin tener estilos dispersos por toda la aplicación.

La interfaz se ha adaptado también a dispositivos móviles. Las tarjetas de tareas reducen elementos decorativos para ganar espacio útil, el panel de vistas pasa a comportarse como elemento flotante, la campanita se muestra como modal, los filtros avanzados se compactan, los modales de asignaciones se convierten en flujo lista/detalle y las pantallas de login y registro priorizan el formulario. Además, los fondos repetidos de tarjetas en móvil se han optimizado utilizando imágenes WebP para reducir el coste visual en listas largas.

La validación principal del frontend se ha realizado mediante `npm run build` y pruebas manuales de los flujos principales: autenticación, carga de tareas, gestión de categorías, gestión de grupos, asignaciones, filtros avanzados, vista inteligente, notificaciones internas, Web Push, PWA instalada y uso responsive en móvil.

5.3. Experiencia móvil y PWA responsive

La versión actual del proyecto no incluye una aplicación móvil nativa independiente, sino una **PWA responsive** basada en el frontend web de React. Esta decisión permite reutilizar la misma base de código para escritorio y móvil, manteniendo una única integración con la API REST del backend.

La aplicación puede abrirse desde el navegador móvil y, en navegadores compatibles, instalarse como aplicación web progresiva. De esta forma, el usuario puede acceder a Gestor de Tareas desde un icono propio, con una experiencia más cercana a una app instalada, pero sin necesidad de desarrollar todavía una aplicación móvil específica con tecnologías como React Native.

Durante el desarrollo se ha realizado un bloque específico de adaptación móvil. El objetivo principal fue mejorar la usabilidad en pantallas pequeñas sin romper el diseño de escritorio. Para ello se ajustaron las tarjetas de tareas, categorías y grupos, se adaptaron los modales principales, se compactó el filtro avanzado, se rediseñó la campanita de notificaciones como modal móvil y se ajustaron las pantallas de login y registro para que el formulario quedara accesible sin scroll excesivo.

En móvil, el panel de vistas se transforma en un elemento flotante que puede moverse por la pantalla. Esto evita ocupar espacio fijo en la parte superior y permite acceder a las vistas principales sin saturar el layout. También se añadieron animaciones sutiles en despleables, tarjetas, filtros y paneles para mejorar la sensación de fluidez sin añadir complejidad excesiva.

Otro punto importante fue la optimización visual de las tarjetas. En escritorio se mantienen fondos SVG decorativos, pero en móvil se sustituyeron los fondos repetidos de tarjetas por imágenes **WebP** rasterizadas. Esta decisión reduce el coste de repintado cuando hay muchas tarjetas en pantalla, especialmente al hacer scroll en listas largas.

La experiencia móvil actual permite utilizar las funciones principales de la aplicación: iniciar sesión, registrarse, consultar tareas, crear y editar tareas, completar tareas, gestionar categorías, consultar grupos, revisar asignaciones, aplicar filtros avanzados, utilizar la vista inteligente, consultar notificaciones y activar notificaciones del dispositivo cuando el navegador lo permite.

Aun así, la PWA responsive no sustituye completamente a una app móvil nativa. Una futura versión podría incorporar una aplicación específica con React Native u otra tecnología móvil, reutilizando la API REST ya desarrollada. Esa evolución permitiría aprovechar mejor características propias del dispositivo, pero queda fuera del alcance actual del proyecto.

5.4. Seguridad

La seguridad de la aplicación **Gestor de Tareas** se basa en **Spring Security** y en un mecanismo de autenticación mediante **JWT**. El objetivo es garantizar que solo los usuarios autenticados puedan acceder a las operaciones protegidas de la API y que cada usuario solo pueda trabajar con los recursos que le corresponden, incluyendo tareas, categorías, grupos, asignaciones, filtros guardados, notificaciones, preferencias y suscripciones push.

El sistema utiliza un esquema basado en **access token** y **refresh token**, lo que permite proteger los endpoints mediante un token de acceso y renovar la sesión sin obligar al usuario a iniciar sesión constantemente. Además, la autorización no se limita a comprobar si existe un usuario autenticado, sino que también valida ownership de recursos y permisos contextuales dentro de grupos.

5.4.1. Arquitectura de seguridad

La arquitectura de seguridad del backend se apoya en **Spring Security** y se configura para trabajar con una API REST sin sesiones tradicionales de servidor. En lugar de mantener una sesión HTTP clásica, el cliente debe enviar un token válido en cada petición protegida.

La configuración principal de seguridad se encarga de definir qué rutas son públicas y qué rutas requieren autenticación. Permanecen públicas las operaciones necesarias para registro, login, refresh de sesión, documentación de la API y health check. El resto de endpoints funcionales de tareas, categorías, grupos, asignaciones, filtros, notificaciones, preferencias y suscripciones push requieren autenticación.

El flujo general se basa en varios componentes:

- **Configuración de seguridad:** define las reglas de acceso, la política stateless, la configuración de CORS y la integración del filtro JWT dentro de la cadena de filtros de Spring Security.

- **Filtro JWT:** intercepta las peticiones entrantes, extrae el token de la cabecera Authorization, comprueba su validez y, si es correcto, registra la autenticación en el contexto de seguridad de Spring.
- **Servicio de JWT:** centraliza la generación, lectura y validación de tokens. Se encarga de extraer el usuario asociado, comprobar la expiración y distinguir el uso correcto de los tokens.
- **Servicio de detalles de usuario:** carga la información necesaria del usuario desde base de datos para que Spring Security pueda construir el contexto autenticado.
- **AuthenticationManager:** participa en el proceso de login, validando las credenciales del usuario antes de generar los tokens correspondientes.

A nivel de autorización, el sistema no se limita a comprobar si el usuario está autenticado. La capa de servicios valida también si el usuario tiene permiso real sobre el recurso solicitado. Esto se aplica a tareas, categorías, grupos, asignaciones, filtros guardados, notificaciones, preferencias y suscripciones push.

En el sistema colaborativo, la autorización se basa en roles propios de cada grupo. Un usuario puede ser creador, admin o miembro dentro de un grupo, y esos roles determinan qué acciones puede realizar. Por ejemplo, no todos los miembros pueden gestionar usuarios, validar entregas o transferir la propiedad de un grupo.

La configuración de CORS limita los orígenes desde los que se puede consumir la API desde navegador. En producción se permite el dominio del frontend desplegado en Vercel y en desarrollo se permite el entorno local usado por Vite. Esta configuración complementa la autenticación JWT, aunque no la sustituye.

En conjunto, la arquitectura de seguridad combina autenticación mediante tokens, autorización por ownership, permisos contextuales en grupos, configuración CORS y comunicación HTTPS en producción.

5.4.2. Autenticación con JWT

La autenticación del sistema se basa en JSON Web Tokens (JWT). Este mecanismo permite identificar al usuario en cada petición protegida sin mantener una sesión tradicional en el servidor. Cuando el usuario inicia sesión correctamente, el backend genera los tokens necesarios y el frontend los utiliza para consumir la API.

El flujo comienza en el endpoint `POST /api/usuario/login`. El usuario introduce su correo electrónico y contraseña desde el formulario de login. El backend valida esas

credenciales mediante Spring Security y, si son correctas, devuelve un **access token** y un **refresh token**.

El **access token** se utiliza para acceder a los endpoints protegidos de la API. En cada petición autenticada, el frontend lo envía en la cabecera HTTP Authorization con el siguiente formato:

`Authorization: Bearer <token>`

Cuando una petición llega al backend, el filtro JWT extrae el token de la cabecera, comprueba que sea válido, verifica que no haya expirado y obtiene el usuario asociado. Si el token es correcto, el backend registra la autenticación en el contexto de seguridad de Spring y permite continuar con la petición.

El **refresh token** se utiliza para renovar la sesión cuando el access token caduca. En lugar de obligar al usuario a iniciar sesión de nuevo, el frontend puede llamar al endpoint `POST /api/usuario/refresh` y solicitar un nuevo access token. Este mecanismo mejora la experiencia de uso y permite mantener sesiones más cómodas sin convertir el access token en un token excesivamente largo.

El backend diferencia el uso de ambos tokens. El access token está pensado para acceder a recursos protegidos, mientras que el refresh token se reserva para renovar la sesión. Esta separación evita que el refresh token se utilice como credencial normal para consumir endpoints funcionales de tareas, categorías, grupos, notificaciones o filtros.

En el frontend, el cliente HTTP centraliza el envío del token y la renovación automática de sesión. Cuando una petición protegida recibe una respuesta de expiración o autorización inválida, el frontend intenta renovar el access token mediante el refresh token. Si la renovación falla, se limpia la sesión y el usuario debe iniciar sesión de nuevo.

Este sistema permite mantener una API REST **stateless**, ya que el servidor no necesita guardar una sesión HTTP tradicional para cada usuario conectado. La identidad del usuario viaja en el token y se valida en cada petición, mientras que la autorización concreta sobre los recursos se comprueba posteriormente en la capa de servicios.

5.4.3. Autorización y restricciones de acceso

La configuración de Spring Security define qué rutas de la API son públicas y cuáles requieren autenticación:

- **Endpoints públicos (sin token):**
 - `POST /api/usuario/add` → registro de nuevos usuarios.

- POST `/api/usuario/login` → inicio de sesión y obtención de JWT.
 - POST `/api/usuario/refresh` → renovación del access token mediante refresh token.
 - Endpoints de documentación de la API:
 - `/swagger-ui/**`
 - `/v3/api-docs/**`
 - `/documentacion-api/**`
 - GET `/api/health` → comprobación pública de disponibilidad del backend.
 - Peticiones OPTIONS a rutas `/api/**`, necesarias para las pre-peticiones CORS.
- **Endpoints protegidos (requieren token JWT válido):**
 - Todas las rutas de gestión de tareas, categorías, grupos, asignaciones y cuenta autenticada requieren que la petición incluya un token válido en la cabecera Authorization.
 - Las rutas de filtro combinado, estado guardado del filtro, tareas recomendadas y recordatorio inteligente también requieren autenticación.
 - Las rutas de notificaciones, preferencias de notificación y suscripciones push requieren token JWT válido y solo permiten operar sobre datos del usuario autenticado.
 - Las operaciones sobre la cuenta autenticada, como **GET `/api/usuario/me`** y **DELETE `/api/usuario/me`**, también requieren autenticación previa.
 - El refresh token no se acepta como sustituto del access token en el acceso normal a recursos protegidos.

Además de esta autorización general, la aplicación aplica restricciones adicionales a nivel de lógica de negocio para garantizar que cada usuario solo pueda operar sobre los recursos que le corresponden. En las operaciones sobre tareas, categorías, estado guardado del filtro, notificaciones, preferencias, recordatorios y suscripciones push, el backend comprueba la propiedad real del recurso antes de permitir su consulta, modificación o eliminación.

En la parte colaborativa, la autorización se basa en roles dentro de cada grupo. El sistema distingue entre creador, admin y miembro, y utiliza esos roles para decidir qué acciones puede realizar cada usuario. Por ejemplo, la gestión de miembros, el cambio de roles, la transferencia de ownership, la creación de asignaciones, la validación de entregas y la reapertura de tareas están restringidas a usuarios con permisos suficientes dentro del grupo correspondiente.

Este enfoque evita depender únicamente de la autenticación general. Aunque un usuario tenga un token válido, no puede operar sobre recursos ajenos ni realizar

acciones para las que no tenga permiso. Si un cliente intenta forzar identificadores en la URL o en el cuerpo de la petición, la capa de servicios vuelve a comprobar ownership y permisos antes de ejecutar la operación.

También se aplica una restricción específica al uso de tokens. El access token es el token válido para consumir endpoints protegidos, mientras que el refresh token queda reservado para renovar la sesión mediante el endpoint correspondiente. De esta forma, el refresh token no se acepta como sustituto del access token en el acceso normal a recursos funcionales de la API.

En conjunto, la autorización del sistema combina reglas generales de Spring Security con comprobaciones de negocio en la capa de servicios. Esto permite proteger tanto recursos individuales como tareas o categorías, como recursos colaborativos asociados a grupos, asignaciones y notificaciones.

5.4.4. Filtro JWT y manejo de errores de seguridad

El filtro JWT es el componente encargado de interceptar las peticiones entrantes antes de que lleguen a los controladores protegidos. Su función principal es comprobar si la petición incluye una cabecera **Authorization** válida y, en ese caso, autenticar al usuario dentro del contexto de seguridad de Spring.

El formato esperado de la cabecera es:

Authorization: Bearer <token>

Cuando una petición llega al backend, el filtro comprueba si la cabecera existe y si comienza por Bearer. Si no existe o no tiene el formato correcto, la petición continúa sin usuario autenticado. En caso de que el endpoint solicitado requiera autenticación, Spring Security rechazará posteriormente la operación.

Si la cabecera contiene un token, el filtro extrae el JWT y delega en el servicio correspondiente la validación del token. Este proceso comprueba que el token sea válido, que no haya expirado y que corresponda a un usuario existente. Si la validación es correcta, el backend construye la autenticación del usuario y la registra en el **SecurityContext** de Spring. A partir de ese momento, las capas posteriores pueden conocer qué usuario está realizando la petición.

El sistema distingue entre **access token** y **refresh token**. El access token es el token utilizado para acceder a los endpoints protegidos de la API. El refresh token se reserva para renovar la sesión mediante el endpoint de refresh y no debe utilizarse como

credencial normal para consultar tareas, categorías, grupos, notificaciones o cualquier otro recurso funcional.

Cuando se produce un error de seguridad, el backend devuelve respuestas HTTP coherentes con el tipo de problema. Si falta autenticación, el token no es válido o el usuario intenta acceder sin credenciales correctas, la API responde con **401 Unauthorized**. Si el usuario está autenticado pero intenta realizar una acción sobre un recurso ajeno o sin permisos suficientes, la respuesta corresponde a **403 Forbidden** o a un error equivalente definido por la lógica de negocio.

Esta separación es importante porque permite distinguir dos situaciones distintas: no saber quién realiza la petición, o saber quién es el usuario pero no permitirle una acción concreta. Por ejemplo, acceder sin token a una ruta protegida es un problema de autenticación, mientras que intentar cerrar una notificación de otro usuario o gestionar miembros de un grupo sin rol suficiente es un problema de autorización.

Los errores de seguridad se integran con el formato general de errores de la API. De esta forma, el frontend puede interpretar de manera consistente los fallos de autenticación, tokens caducados, ausencia de permisos o intentos de acceso a recursos no autorizados. En el caso del frontend, cuando una petición falla por expiración del access token, el cliente intenta renovar la sesión mediante el refresh token. Si la renovación falla, se limpia la sesión local y el usuario debe iniciar sesión de nuevo.

Este enfoque permite mantener una API REST stateless, donde cada petición protegida lleva la información necesaria para identificar al usuario, y al mismo tiempo conserva un manejo de errores claro para el cliente.

5.5. Otros aspectos relevantes

Además de la organización por capas y de la seguridad basada en JWT, el backend de **Gestor de Tareas** incorpora varias piezas de lógica de negocio específicas que mejoran la usabilidad de la aplicación: mecanismos de ordenación y filtrado de tareas, vistas especializadas como las “tareas de hoy” y reglas para el cálculo del estado de cada tarea a partir de sus fechas.

5.5.1. Lógica de ordenación de tareas

La aplicación permite ordenar las tareas mediante criterios simples y mediante una lógica avanzada de recomendación. Los criterios simples permiten ordenar por aspectos directos como título, prioridad, tiempo estimado o fecha de entrega. Además

de estas ordenaciones clásicas, la versión actual incorpora una **ordenación inteligente** pensada para ayudar al usuario a decidir qué tarea debería atender primero.

La ordenación inteligente no se plantea como inteligencia artificial ni como machine learning. Se trata de una **heurística ponderada**, determinista y explicable. Esto significa que el sistema calcula una puntuación interna para cada tarea siguiendo una fórmula conocida, ajustable y razonable, y después ordena las tareas según esa puntuación.

El objetivo de esta lógica no es adivinar perfectamente qué quiere hacer el usuario, sino construir una vista recomendada útil que combine varios factores importantes: urgencia temporal, prioridad declarada, duración estimada, antigüedad, categoría y estado de revisión en tareas con origen grupal.

La fórmula se implementa en el backend, dentro de una estructura separada de scoring, para mantenerla aislada de los controladores y poder reutilizarla desde distintos flujos, como el filtro combinado o la vista de tareas recomendadas. Las clases principales asociadas a esta lógica son TareaScoringContext, TareaInteligenteScorer y TareaInteligenteRankingService.

Antes de calcular la puntuación, el algoritmo aplica una serie de **reglas duras**. Estas reglas funcionan como condiciones previas y evitan que entren en la recomendación tareas que no son realmente accionables:

- COMPLETADA queda fuera.
- COMPLETADA_CON_RETRASO queda fuera.
- Las tareas de grupo en estado ENTREGADA quedan fuera.
- Las tareas de grupo en estado VALIDADA quedan fuera.
- Las tareas SIN_FECHA no suman por urgencia temporal, pero pueden subir por prioridad, duración, antigüedad o categoría.
- Una tarea vencida desde hace demasiado tiempo no debe ganar solo por estar vencida.
- Una tarea larga no debe hundirse si es importante.
- La categoría base solo aporta un bonus pequeño.

La fórmula general del score es la siguiente:

```
score = 100 * (  
  F * 0.45  
  P * 0.35  
  T * 0.12  
  A * 0.05
```

$$C * 0.03 \\) * MPF * MPT * MRG + BVC + BSF$$

Donde:

F = factor de fecha o estado temporal.

P = factor de prioridad.

T = factor de tiempo estimado.

A = factor de antigüedad desde la creación.

C = factor de categoría base.

MPF = multiplicador de prioridad por fecha.

MPT = multiplicador de prioridad por tiempo.

MRG = multiplicador de revisión grupal por fecha.

BVC = bonus de ventana crítica.

BSF = bonus para tareas sin fecha.

Los pesos principales de la fórmula indican la importancia relativa de cada factor. La fecha y la prioridad son los factores más importantes, porque el sistema debe priorizar tareas urgentes e importantes. El tiempo estimado, la antigüedad y la categoría también influyen, pero con menor peso para evitar que dominen sobre la urgencia o la prioridad.

Factor F: fecha y presión temporal

El factor F mide la presión temporal de la tarea. Se calcula comparando la fecha de entrega con la fecha y hora actuales. Todos los cálculos temporales se realizan internamente en horas, para evitar saltos bruscos entre días.

En tareas con fecha futura, cuanto menos tiempo queda para la entrega, mayor es el valor de F. La tabla base es:

0h → F = 1.00
4h → F = 0.99
9h → F = 0.98
13h → F = 0.97
18h → F = 0.96
24h → F = 0.95

30h → $F = 0.90$
36h → $F = 0.85$
42h → $F = 0.80$
48h → $F = 0.75$
54h → $F = 0.68$
60h → $F = 0.61$
66h → $F = 0.54$
72h → $F = 0.47$
4d → $F = 0.40$
5d → $F = 0.33$
6d → $F = 0.26$
7d → $F = 0.20$
8d o más → $F = 0.10$

A partir de ocho días, la tarea mantiene un valor mínimo, pero no recibe un empujón fuerte por fecha porque todavía está demasiado lejos.

En tareas vencidas, el sistema no quiere que una tarea vencida desde hace mucho tiempo gane siempre solo por estar vencida. Por eso el valor de F baja progresivamente a medida que pasa el tiempo:

0-1d vencida → $F = 0.90$
2d vencida → $F = 0.78$
3d vencida → $F = 0.68$
4d vencida → $F = 0.58$
5d vencida → $F = 0.50$
6d vencida → $F = 0.43$
7d vencida → $F = 0.36$
8d vencida → $F = 0.30$
9d vencida → $F = 0.25$
10d vencida → $F = 0.21$
11d vencida → $F = 0.18$
12d vencida → $F = 0.15$
13d vencida → $F = 0.12$
14d vencida → $F = 0.10$
15d vencida → $F = 0.08$
16d vencida → $F = 0.06$
17d vencida → $F = 0.04$
18d vencida → $F = 0.03$
19d vencida → $F = 0.01$

20d vencida $\rightarrow F = 0.00$
21d vencida $\rightarrow F = -0.01$
22d vencida $\rightarrow F = -0.01$
23d vencida $\rightarrow F = -0.02$
24d vencida $\rightarrow F = -0.02$
25d vencida $\rightarrow F = -0.03$
26d vencida $\rightarrow F = -0.03$
27d vencida $\rightarrow F = -0.04$
28d vencida $\rightarrow F = -0.04$
29d vencida $\rightarrow F = -0.05$
30d vencida $\rightarrow F = -0.05$
60d o más vencida $\rightarrow F = -0.10$

De esta forma, una tarea recién vencida sigue siendo relevante, pero una tarea abandonada desde hace muchas semanas deja de dominar la recomendación.

Si una tarea no tiene fecha de entrega:

SIN_FECHA $\rightarrow F = 0.00$

Esto significa que no recibe puntos por urgencia temporal, aunque puede seguir subiendo por otros factores.

Factor P: prioridad

El factor P representa la importancia declarada por el usuario. La escala usada es:

IMPRESINDIBLE $\rightarrow P = 1.00$
ALTA $\rightarrow P = 0.75$
MEDIA $\rightarrow P = 0.40$
BAJA $\rightarrow P = 0.20$

La prioridad influye de dos formas. Primero, aparece directamente dentro de la suma ponderada de la fórmula. Segundo, participa en multiplicadores como MPF y MPT, que ajustan el resultado cuando la prioridad se combina con urgencia temporal o duración estimada.

Factor T: tiempo estimado

El factor T mide cuánto ayuda que una tarea sea manejable por duración. Una tarea corta puede ser más fácil de encajar en el día, pero una tarea larga no debe quedar hundida si es importante.

La escala utilizada es:

1 min → T = 1.15
2-5 min → T = 1.05
6-10 min → T = 1.00
11-15 min → T = 0.95
16-20 min → T = 0.88
21-30 min → T = 0.80
31-45 min → T = 0.68
46-60 min → T = 0.60
61-75 min → T = 0.54
76-90 min → T = 0.49
91-105 min → T = 0.46
106-120 min → T = 0.44
121-150 min → T = 0.36
151-180 min → T = 0.29
181-240 min → T = 0.22
Más de 240 min → T = 0.16

El tiempo funciona principalmente como un bonus de facilidad, no como un castigo absoluto. Por eso, aunque las tareas largas bajan algo por duración, no se pretende que una tarea imprescindible de varias horas quede siempre por debajo de tareas poco importantes pero rápidas.

Para el multiplicador MPT se usa una versión limitada del factor T:

$$T_MPT = \min(T, 1.00)$$

$$\text{costeTiempo} = \max(0, 1 - T_MPT)$$

Esto evita que las tareas extremadamente cortas, cuyo T puede superar 1.00, generen efectos excesivos dentro del multiplicador.

Factor A: antigüedad

El factor A mide cuánto tiempo lleva creada la tarea. Se calcula a partir de `fechaAgregado`, no de `fechaEntrega`.

La escala usada es:

0d → A = 0.00
1d → A = 0.01
2d → A = 0.03
3d → A = 0.04
4d → A = 0.06

5d → A = 0.07

6d → A = 0.09

7d → A = 0.10

10d → A = 0.14

14d → A = 0.19

18d → A = 0.24

22d → A = 0.30

26d → A = 0.35

30d → A = 0.40

40d → A = 0.33

50d → A = 0.27

60d → A = 0.20

70d → A = 0.13

80d → A = 0.07

90d o más → A = 0.00

La idea es dar cierto empujón a tareas que llevan tiempo creadas, pero sin permitir que tareas muy antiguas y abandonadas dominen el ranking indefinidamente. Por eso el valor sube hasta aproximadamente los 30 días y después vuelve a bajar.

Factor C: categoría base

El factor C permite que ciertas categorías base tengan un pequeño peso adicional. No debe dominar frente a la fecha o la prioridad, por eso su peso global en la fórmula es bajo.

Los valores son:

Trabajo/Estudios → C = 1.00

Doméstico → C = 0.60

Ocio/Personal → C = 0.30

Categoría personalizada → C = 0.00

Sin categoría → C = 0.00

Este factor ayuda a dar un pequeño empujón a categorías base consideradas más relevantes, pero sin convertir la categoría en el criterio principal.

Multiplicador MPF: prioridad por fecha

MPF aumenta el score cuando una tarea combina prioridad alta y presión temporal. No sustituye al factor P ni al factor F, sino que amplifica la combinación de ambos.

Primero se calcula la urgencia:

$$\text{urgencia} = \max(F, 0)$$

Después se obtiene un bonus según prioridad:

$$\text{BAJA} \rightarrow \text{bonusPrioridad} = 0.00$$

$$\text{MEDIA} \rightarrow \text{bonusPrioridad} = 0.10$$

$$\text{ALTA} \rightarrow \text{bonusPrioridad} = 0.22$$

$$\text{IMPRESCINDIBLE} \rightarrow \text{bonusPrioridad} = 0.35$$

La fórmula es:

$$\text{MPF} = 1 + (\text{urgencia} * \text{bonusPrioridad})$$

Ejemplos:

$$\text{IMPRESCINDIBLE con } F = 1.00$$

$$\text{MPF} = 1 + (1.00 * 0.35) = 1.35$$

$$\text{IMPRESCINDIBLE con } F = 0.80$$

$$\text{MPF} = 1 + (0.80 * 0.35) = 1.28$$

$$\text{ALTA con } F = 1.00$$

$$\text{MPF} = 1 + (1.00 * 0.22) = 1.22$$

$$\text{MEDIA con } F = 1.00$$

$$\text{MPF} = 1 + (1.00 * 0.10) = 1.10$$

$$\text{BAJA con } F = 1.00$$

$$\text{MPF} = 1.00$$

Si F es negativa, por ejemplo en una tarea vencida hace mucho tiempo, no se amplifica:

$$F < 0 \rightarrow \text{urgencia} = 0 \rightarrow \text{MPF} = 1.00$$

Con esto, las tareas importantes y urgentes suben más, pero las tareas vencidas antiguas no se reviven artificialmente.

Multiplicador MPT: prioridad por tiempo

MPT ajusta el score según la relación entre prioridad y duración. Sirve para penalizar más las tareas largas de baja prioridad y evitar castigar demasiado las tareas largas cuando son importantes.

Para tareas futuras se calcula primero el margen real:

$$\text{margenReal} = \text{horasRestantesHastaEntrega} - \text{duracionHoras}$$

Si el margen real es mayor de 48 horas:

$$\text{MPT} = 1 - (\text{costeTiempo} * \text{PL})$$

Si el margen real es menor o igual a 48 horas, o si la tarea no tiene fecha futura:

$$\text{MPT} = 1 + (\text{T_MPT} * \text{BR}) - (\text{costeTiempo} * \text{PL})$$

Donde:

$$\text{T_MPT} = \min(\text{T}, 1.00)$$

$$\text{costeTiempo} = \max(0, 1 - \text{T_MPT})$$

BR = bonus de rapidez según prioridad.

PL = penalización por duración larga según prioridad.

Los valores de BR y PL son:

BAJA → BR = 0.10, PL = 0.20

MEDIA → BR = 0.08, PL = 0.12

ALTA → BR = 0.04, PL = 0.05

IMPRESINDIBLE → BR = 0.05, PL = 0.00

Interpretación:

BR indica cuánto se premia que una tarea sea corta.

PL indica cuánto se penaliza que una tarea sea larga.

Ejemplo para una tarea de prioridad BAJA y duración superior a 180 minutos:

$$\text{MPT} = 1 + (0.16 * 0.10) - ((1 - 0.16) * 0.20)$$

$$\text{MPT} = 1 + 0.016 - (0.84 * 0.20)$$

$$\text{MPT} = 1 + 0.016 - 0.168$$

$$\text{MPT} = 0.848$$

Ejemplo similar con más de 48 horas de margen real:

$$\text{MPT} = 1 - ((1 - 0.16) * 0.20)$$

$$\text{MPT} = 1 - (0.84 * 0.20)$$

MPT = 0.832

Este multiplicador consigue que una tarea baja y larga baje bastante, una tarea media y corta suba algo, una tarea alta y larga baje muy poco y una tarea imprescindible larga no sea castigada por su duración.

Multiplicador MRG: revisión grupal por fecha

MRG se aplica a tareas con origen grupal, especialmente cuando una entrega ha sido reabierta. El objetivo es que una tarea reabierta y próxima pueda subir de forma moderada, pero sin que una tarea reabierta lejana suba solo por estar reabierta.

La fórmula general es:

Si la tarea reabierta está dentro de una ventana próxima:

$$\text{MRG} = 1 + (\text{urgencia} * \text{BRG})$$

Si la tarea reabierta no está dentro de una ventana próxima:

$$\text{MRG} = 1.00$$

Donde:

$$\text{urgencia} = \max(F, 0)$$

BRG = bonus por revisión grupal

Se considera ventana próxima cuando:

Tarea futura reabierta → aplica MRG si fechaEntrega ≤ 48h

Tarea vencida reabierta → aplica MRG si lleva vencida ≤ 24h

Resto de casos → MRG = 1.00

Valores de BRG:

REABIERTA → BRG = 0.10

PENDIENTE → BRG = 0.00

ENTREGADA → fuera de recomendadas

VALIDADA → fuera de recomendadas

Sin revisión → BRG = 0.00

Fórmula desplegada:

$$\text{MRG} = 1 + (\max(F, 0) * \text{bonusRevisionGrupo})$$

Ejemplo para una tarea reabierta con $F = 0.80$ dentro de ventana próxima:

$$\text{MRG} = 1 + (\max(0.80, 0) * 0.10)$$

$$\text{MRG} = 1 + (0.80 * 0.10)$$

$$\text{MRG} = 1.08$$

Si la tarea está reabierta pero fuera de ventana próxima, MRG se queda en 1.00. Si F es negativa, tampoco se aplica amplificación. Con esto, el sistema da importancia a una tarea reabierta que vuelve a ser urgente, pero no convierte cualquier reapertura en prioridad máxima.

Bonus BVC: ventana crítica

BVC significa Bonus de Ventana Crítica. A diferencia de MPF, MPT y MRG, no es un multiplicador. Se suma al final de la fórmula para empujar tareas que están dentro de una ventana de urgencia real.

La fórmula general es:

Si la tarea tiene fecha futura:

$$\text{BVC} = \min(30, \text{BVF} + \text{BMR})$$

Si la tarea está vencida y lleva vencida 24 horas o menos:

$$\text{BVC} = \min(24, \text{BV} + \text{BR})$$

Si la tarea está vencida y lleva vencida más de 24 horas:

$$\text{BVC} = \text{BV}$$

Si la tarea no tiene fecha:

$$\text{BVC} = 0$$

En tareas con fecha futura, BVC se divide en dos partes:

BVF = Bonus de Ventana de Fecha.

BMR = Bonus de Margen Real.

El margen real se calcula así:

Gestor de Tareas - Documentación técnica

H = horas restantes hasta la fecha de entrega.

D = duración estimada en horas.

MR = margen real = H - D.

BVF mide la proximidad de la fecha:

0-1h → BVF = +12

1-3h → BVF = +11

3-6h → BVF = +10

6-12h → BVF = +9

12-18h → BVF = +6

18-24h → BVF = +4

24-30h → BVF = +2

30-36h → BVF = +1

36h o más → BVF = +0

Sin fecha → BVF = +0

BMR mide cuánto margen real queda para completar la tarea:

MR ≤ 0h → BMR = +20

0-1h → BMR = +18

1-3h → BMR = +16

3-6h → BMR = +10

6-12h → BMR = +6

12-24h → BMR = +3

24h o más → BMR = +0

Sin fecha → BMR = +0

En tareas vencidas, BVC se divide en:

BV = Bonus por Vencimiento reciente.

BR = Bonus de Rescate rápido.

BV mide cuánto tiempo lleva vencida la tarea:

0-3h vencida → BV = +14

3-6h vencida → BV = +12

6-12h vencida → BV = +9

12-24h vencida → BV = +6

24-36h vencida → BV = +3

36-48h vencida $\rightarrow BV = +1$

48h o más vencida $\rightarrow BV = +0$

BR solo se aplica si la tarea lleva vencida 24 horas o menos, y depende de la duración:

1-15 min $\rightarrow BR = +8$

16-30 min $\rightarrow BR = +6$

31-60 min $\rightarrow BR = +4$

61-120 min $\rightarrow BR = +2$

Más de 120 min $\rightarrow BR = +0$

Este bonus hace que las tareas que vencen en pocas horas suban claramente, que las tareas con poco margen real suban más y que las tareas recién vencidas puedan rescatarse si todavía tiene sentido. En cambio, las tareas vencidas antiguas no suben solo por ser rápidas.

Bonus BSF: tareas sin fecha

BSF significa Bonus Sin Fecha. Solo se aplica cuando la tarea no tiene fecha de entrega.

Si `fechaEntrega == null`:

$BSF = basePrioridadSinFecha * factorTiempoSinFecha$

Si `fechaEntrega != null`:

$BSF = 0$

La base por prioridad es:

IMPRESINDIBLE $\rightarrow 12$

ALTA $\rightarrow 6$

MEDIA $\rightarrow 2$

BAJA $\rightarrow 0$

El factor de tiempo para tareas sin fecha es:

1-15 min $\rightarrow 1.00$

16-30 min $\rightarrow 0.90$

31-45 min $\rightarrow 0.80$

46-60 min $\rightarrow 0.70$

61-75 min $\rightarrow 0.60$

76-90 min $\rightarrow 0.50$

91-105 min → 0.42

106-120 min → 0.35

121-150 min → 0.28

151-180 min → 0.22

181-210 min → 0.18

211-240 min → 0.15

Más de 240 min → 0.10

Ejemplos:

Sin fecha, imprescindible y 15 minutos:

$$\text{BSF} = 12 * 1.00 = 12$$

Sin fecha, imprescindible y 120 minutos:

$$\text{BSF} = 12 * 0.35 = 4.2$$

Sin fecha, alta y 30 minutos:

$$\text{BSF} = 6 * 0.90 = 5.4$$

Sin fecha, media y 90 minutos:

$$\text{BSF} = 2 * 0.50 = 1.0$$

Este bonus evita que las tareas sin fecha queden enterradas siempre. Aun así, no debe hacer que compitan normalmente contra tareas con urgencia temporal real, salvo que sean importantes y razonablemente manejables.

Integración con el backend

La ordenación inteligente se integra en el backend de dos formas principales.

Por un lado, el filtro combinado acepta el criterio de ordenación INTELIGENTE. Cuando se utiliza este criterio, el backend filtra primero las tareas según los criterios recibidos y después ordena el resultado aplicando la lógica de scoring.

Por otro lado, existe un endpoint dedicado para obtener tareas recomendadas. Este endpoint reutiliza el filtro combinado y fuerza internamente el criterio inteligente, sin duplicar la fórmula y sin crear un DTO completamente distinto. El cliente recibe una lista de tareas ordenadas, pero no recibe el score interno.

Esta decisión mantiene la lógica centralizada en el backend, evita inconsistencias entre vistas y permite que el frontend muestre la vista Inteligente sin conocer los detalles matemáticos del algoritmo.

5.5.2. Lógica de filtrado de tareas

La aplicación incorpora distintos niveles de filtrado para facilitar que el usuario encuentre rápidamente las tareas que necesita consultar. Esta lógica ha evolucionado desde filtros simples sobre tareas personales hasta un **filtro combinado** capaz de mezclar tareas personales y tareas con origen en grupos.

En primer lugar, el backend ofrece filtros individuales sobre tareas del usuario autenticado. Estos filtros permiten consultar tareas por criterios concretos, como categoría, prioridad, estado, tiempo estimado o palabras clave. También existen vistas específicas como tareas del día, tareas próximas a vencer y tareas vencidas. Estas consultas son útiles para casos simples y para mantener endpoints claros durante el desarrollo.

Además de esos filtros individuales, la versión actual incorpora un **filtro combinado de tareas**. Este filtro permite trabajar sobre una lista unificada formada por tareas personales y tareas generadas desde asignaciones de grupo. El usuario puede indicar distintos criterios opcionales en una misma petición y el backend devuelve únicamente las tareas que cumplen esas condiciones.

Los criterios principales del filtro combinado son:

- origen de la tarea, distinguiendo entre tareas personales, tareas de grupo o ambas;
- grupo concreto, cuando se desea limitar la búsqueda a tareas procedentes de un grupo determinado;
- categoría, para filtrar tareas personales asociadas a una categoría concreta;
- prioridades múltiples, permitiendo seleccionar varias prioridades a la vez;
- estados múltiples, permitiendo seleccionar varios estados de tarea;
- palabras clave, aplicadas sobre texto de la tarea;
- tiempo máximo estimado;
- fecha exacta de entrega;
- fecha límite hasta un día determinado;
- criterio de ordenación activo;
- opción para mostrar solo tareas pendientes de completar.

El filtro mantiene compatibilidad con campos simples anteriores, como prioridad y estado, pero la versión actual trabaja principalmente con listas de prioridades y estados. Esto permite al frontend seleccionar varios valores a la vez sin tener que lanzar varias peticiones separadas.

El backend también valida combinaciones incoherentes. Por ejemplo, no tiene sentido indicar un grupo concreto si el origen seleccionado es solo personal, ni aplicar al mismo tiempo una fecha exacta y una fecha límite hasta. Estas reglas se controlan en la capa de servicios para evitar que el cliente tenga que asumir toda la responsabilidad de coherencia.

El resultado del filtro combinado utiliza un DTO enriquecido. Además de los datos básicos de la tarea, puede incluir información de categoría, origen de la tarea, grupo de procedencia y estado de revisión cuando se trata de una tarea generada desde una asignación grupal. Esto permite que el frontend pinte una misma lista de tareas aunque procedan de contextos distintos.

La aplicación también permite persistir en servidor el último estado del filtro combinado mediante endpoints específicos de guardado y recuperación. De esta forma, el usuario puede cerrar la aplicación y volver a encontrar los controles del filtro en el último estado utilizado. Esta persistencia se asocia al usuario autenticado, no al navegador concreto, por lo que puede reutilizarse desde distintos dispositivos.

En el frontend, esta lógica se representa mediante el **filtro avanzado** de la pantalla principal. Cuando el usuario aplica filtros, el cliente construye una petición limpia, llama al endpoint de filtro combinado y muestra los resultados como una vista propia dentro del dashboard. Cuando limpia los filtros, vuelve a la vista de Mis tareas y guarda también el estado limpio en servidor.

Además del filtro avanzado enviado al backend, el frontend mantiene un buscador rápido local. Este buscador no sustituye al filtro combinado, sino que actúa como una segunda capa visual sobre la lista que ya se está mostrando. Es útil para reducir rápidamente tareas en pantalla sin realizar una nueva petición al servidor.

En conjunto, la lógica de filtrado permite trabajar tanto con consultas simples como con una vista avanzada más flexible, preparada para mezclar tareas personales y colaborativas dentro de una experiencia principal de trabajo.

5.5.3. Gestión de las tareas del día

La aplicación incluye varias consultas temporales que permiten al usuario centrarse en las tareas más relevantes según su fecha de entrega. Estas vistas no sustituyen al filtro avanzado, pero ofrecen accesos rápidos a situaciones habituales: tareas del día, tareas próximas a vencer y tareas vencidas.

La vista de **tareas del día** muestra las tareas cuya fecha de entrega coincide con la fecha actual. Para ello, el backend calcula el intervalo correspondiente al día en curso y consulta las tareas del usuario autenticado que tienen fecha de entrega dentro de ese rango. Esta vista resulta útil para que el usuario pueda centrarse rápidamente en lo que debe atender durante la jornada.

La vista de **tareas próximas a vencer** permite consultar tareas no completadas con fecha de entrega futura, ordenadas por proximidad. Su objetivo es mostrar al usuario qué tareas se acercan a su vencimiento antes de que pasen a estar retrasadas.

La vista de **tareas vencidas** muestra tareas no completadas cuya fecha de entrega ya ha pasado. Esta consulta permite localizar tareas pendientes que han superado su fecha prevista y decidir si deben completarse, editarse, eliminarse o reorganizarse.

Estas vistas se calculan en el backend para mantener la lógica temporal centralizada. El frontend consume los endpoints correspondientes y representa los resultados en la interfaz, pero no decide por sí mismo qué tarea pertenece a cada vista. Esto evita duplicar reglas de fecha entre cliente y servidor.

La coherencia temporal es especialmente importante porque la aplicación está desplegada en la nube. Por ese motivo, el entorno de producción se configura con la zona horaria **Europe/Madrid**, evitando desfases entre las fechas calculadas por el backend y las fechas esperadas por el usuario en España. Esta configuración afecta tanto a las vistas temporales como al cálculo de estados, avisos 24 horas antes y recordatorios inteligentes.

5.5.4. Cálculo del estado de una tarea y coherencia de fechas

El estado de una tarea no se almacena como un campo fijo independiente, sino que se calcula a partir de sus datos principales. Esta decisión evita inconsistencias entre el valor guardado y la situación real de la tarea. El backend determina el estado teniendo en cuenta la fecha de entrega, si la tarea está completada, la fecha de completado y la fecha actual.

Los estados principales que maneja el sistema son:

- **EN_CURSO**

La tarea no está completada y tiene una fecha de entrega futura. Representa una tarea pendiente que todavía está dentro de plazo.

- **VENCIDA**

La tarea no está completada y su fecha de entrega ya ha pasado. Este estado permite identificar tareas retrasadas que todavía requieren atención.

- **COMPLETADA**

La tarea está marcada como completada y se completó dentro del plazo establecido, o no tenía una fecha de entrega que permita considerarla retrasada.

- **COMPLETADA_CON_RETRASO**

La tarea está completada, pero la fecha de completado es posterior a la fecha de entrega. Este estado permite distinguir entre tareas terminadas a tiempo y tareas terminadas después del plazo.

- **SIN_FECHA**

La tarea no tiene fecha de entrega. En este caso no puede considerarse vencida ni próxima a vencer, aunque sigue pudiendo ordenarse o filtrarse por otros criterios como prioridad, tiempo estimado, categoría o antigüedad.

El cálculo del estado se realiza en el backend para centralizar la lógica y evitar que cada cliente tenga que repetir sus propias reglas. El frontend recibe el estado ya calculado y lo utiliza para pintar etiquetas visuales, colores, filtros y acciones disponibles.

Esta lógica también se utiliza en otras partes del sistema. Por ejemplo, las vistas de tareas vencidas o próximas a vencer dependen de la comparación entre fecha de entrega y fecha actual. La ordenación inteligente también tiene en cuenta la presión temporal de la tarea, y las notificaciones necesitan saber si una tarea está pendiente, si tiene fecha y si se aproxima a su vencimiento.

La coherencia de fechas es especialmente importante en producción, ya que el backend se ejecuta en un entorno cloud. Para evitar desfases horarios, el despliegue en Render se configura explícitamente con la zona horaria **Europe/Madrid**. Esto permite que fechas de creación, vencimientos, avisos 24 horas antes y recordatorios inteligentes se calculen de acuerdo con el horario esperado por el usuario.

En conjunto, el cálculo centralizado del estado permite mantener una representación coherente de las tareas en toda la aplicación, tanto en listados normales como en filtros, vista inteligente, notificaciones y flujos colaborativos.

6. Pruebas

La validación del proyecto **Gestor de Tareas** se ha realizado combinando distintas estrategias: compilación del backend, builds del frontend, pruebas manuales de API con Postman, pruebas desde Swagger, pruebas funcionales desde la propia interfaz web/PWA y comprobaciones en el entorno desplegado en la nube.

Durante las primeras fases del desarrollo se creó una base de tests automáticos con **JUnit 5, Spring Boot Test, Mockito, MockMvc** y pruebas de repositorio. Estos tests cubrían servicios, controladores, repositorios, manejo de errores e incluso algunos flujos de integración. Sin embargo, a medida que el backend evolucionó con autenticación renovada, refresh token, grupos, asignaciones, filtro avanzado, ordenación inteligente, notificaciones y Web Push, parte de esa batería quedó desactualizada.

Por este motivo, en el estado actual del proyecto no se toma la suite automática completa como única fuente de validación. Se mantiene como base útil para recuperar y actualizar en futuras iteraciones, pero la validación final de la versión actual se ha realizado principalmente mediante pruebas manuales controladas, pruebas con Postman, compilaciones, builds y comprobaciones funcionales sobre la aplicación real.

Esta decisión se considera una limitación conocida del proyecto, pero también refleja la evolución real del desarrollo. La aplicación ha crecido de forma incremental y algunos tests escritos para una versión anterior ya no representan correctamente el contrato actual de la API. Como mejora futura, será necesario sanear esa batería de tests, eliminar pruebas obsoletas y añadir nuevos casos alineados con la versión final del backend.

6.1. Pruebas del backend

Las pruebas del backend se han centrado en comprobar que la API compila correctamente, que los servicios principales responden como se espera, que las reglas de negocio se cumplen y que los endpoints protegidos respetan autenticación, ownership y permisos.

En cada bloque backend importante se ha utilizado como validación mínima la compilación del proyecto mediante Maven. El comando más habitual ha sido:

```
.\mvnw.cmd -DskipTests compile
```


En algunos bloques, especialmente cuando se necesitaba generar el artefacto final o comprobar el empaquetado, también se ha ejecutado:

```
.\mvnw.cmd -DskipTests clean package
```

Estas validaciones permiten confirmar que el código compila, que las dependencias están correctamente resueltas y que no existen errores básicos de integración entre clases, DTOs, servicios, repositorios y configuración.

La batería automática existente incluye tests unitarios y de integración creados en fases anteriores del proyecto. Entre ellos hay pruebas de servicios, pruebas de repositorios con base de datos en memoria, pruebas de controladores con MockMvc, pruebas del manejador global de errores y pruebas de flujo. No obstante, parte de estos tests hacen referencia a métodos, endpoints o contratos que han cambiado durante la evolución del backend, por lo que actualmente no todos representan el estado real de la aplicación.

En los últimos bloques de desarrollo, cuando se detectó que la suite global estaba afectada por tests legacy, se priorizó la validación funcional mediante compilación y pruebas manuales. Esta situación se ha documentado para evitar presentar como completamente vigente una batería automática que necesita revisión.

Las pruebas funcionales del backend se han realizado principalmente con **Postman**. Con esta herramienta se han validado flujos como registro, login, refresh token, operaciones CRUD de tareas y categorías, creación y gestión de grupos, unión por código, asignaciones de grupo, entrega, validación y reapertura de tareas, filtro combinado, persistencia del estado del filtro, ordenación inteligente, notificaciones internas, recordatorio inteligente, preferencias de notificación, suscripciones push y health check.

También se han probado escenarios de error, como credenciales incorrectas, token ausente o inválido, acceso a recursos ajenos, categorías protegidas, combinaciones inválidas de filtros, grupos no pertenecientes al usuario, activación de recordatorio en tareas sin fecha, intentos de modificar notificaciones ajenas y errores de payload en suscripciones push.

En producción se han validado además comportamientos específicos del entorno cloud. Entre ellos destacan la conexión del backend desplegado en Render con PostgreSQL en Neon, la configuración de CORS con el frontend en Vercel, el endpoint público GET /api/health, el self-ping para mantener el backend despierto durante pruebas, la zona horaria **Europe/Madrid** y el funcionamiento real de notificaciones internas y Web Push.

En resumen, el backend se ha validado de forma práctica mediante compilación, pruebas manuales exhaustivas y comprobación directa en el entorno desplegado. La principal mejora pendiente en este apartado es actualizar la batería automática de tests para que vuelva a reflejar fielmente el estado actual de la API.

6.2. Pruebas de la API

Las pruebas de la API se han realizado principalmente mediante **Postman** y, de forma complementaria, mediante **Swagger UI**. Estas herramientas han permitido ejecutar peticiones reales contra el backend, comprobar cuerpos de respuesta, validar códigos HTTP, revisar errores controlados y confirmar que los endpoints protegidos requieren autenticación mediante JWT.

El primer bloque probado ha sido el de **autenticación**. Se han validado el registro de usuarios, el inicio de sesión, la obtención de access token y refresh token, la renovación de sesión mediante el endpoint de refresh y el rechazo de credenciales incorrectas. También se ha comprobado que los endpoints protegidos no permiten acceso sin token o con token inválido.

En el bloque de **tareas**, se han probado las operaciones principales de creación, consulta, edición, completado y eliminación. Además, se han validado las vistas de tareas del día, tareas próximas a vencer, tareas vencidas, filtros individuales y consulta de tareas por identificador. También se han probado errores como datos inválidos, acceso a tareas inexistentes o intentos de operar sobre recursos que no pertenecen al usuario autenticado.

En el bloque de **categorías**, se han comprobado el listado, creación, edición, búsqueda y eliminación de categorías. Se ha prestado especial atención a las categorías base protegidas, verificando que no puedan eliminarse como categorías normales y que las tareas asociadas a una categoría personalizada no se borren cuando dicha categoría se elimina.

El bloque de **grupos** se ha probado mediante flujos completos de creación de grupo, consulta, edición, activación o inactivación, eliminación, unión mediante código de invitación, regeneración del código, listado de miembros, cambio de roles, salida del grupo, expulsión de miembros y transferencia de ownership. En estas pruebas se han utilizado distintos usuarios para comprobar que las acciones dependen del rol del usuario dentro del grupo.

También se han validado las **asignaciones de grupo**. Se han probado asignaciones a todo el grupo y asignaciones a una selección manual de miembros, comprobando que

se generen tareas individuales para los destinatarios. Después se han probado flujos de entrega, validación y reapertura, verificando que el sistema mantenga la trazabilidad entre la asignación original, el miembro destinatario y la tarea generada.

El **filtro combinado** ha sido uno de los bloques más importantes de prueba. Se han validado peticiones con origen personal, origen grupal y origen mixto, filtros por grupo, categoría, prioridades múltiples, estados múltiples, palabras clave, tiempo máximo, fecha exacta, fecha hasta y criterio de ordenación. También se ha probado el guardado y recuperación del último estado del filtro, así como combinaciones inválidas que deben ser rechazadas por el backend.

La **ordenación inteligente** se ha probado tanto a través del criterio INTELIGENTE dentro del filtro combinado como mediante el endpoint específico de tareas recomendadas. Las pruebas han comprobado que el backend devuelve una lista ordenada sin exponer la puntuación interna y que se excluyen tareas que no deben aparecer como recomendadas, como tareas completadas o entregas grupales ya validadas.

En el bloque de **notificaciones**, se han probado las operaciones para listar notificaciones activas, consultar el contador de pendientes, cerrar una notificación, cerrar todas las notificaciones, obtener preferencias y actualizar preferencias. También se ha validado la generación de avisos 24 horas antes del vencimiento, recordatorios inteligentes y notificaciones por asignaciones de grupo.

La parte de **Web Push** se ha probado registrando y desactivando suscripciones push desde el frontend y verificando en backend que las suscripciones quedan asociadas al usuario autenticado. Además, se ha comprobado que un fallo de envío push no elimina la notificación interna ni rompe el flujo principal de la aplicación.

Por último, se ha probado el endpoint público GET `/api/health`, utilizado para comprobar la disponibilidad del backend sin autenticación. Este endpoint ha servido tanto para pruebas manuales como para validar el comportamiento del self-ping en el entorno desplegado.

Las pruebas de API no se han limitado a respuestas correctas. También se han comprobado errores de validación, JSON mal formado, enums inválidos, credenciales incorrectas, token ausente o caducado, acceso a recursos ajenos, falta de permisos dentro de grupos, categorías protegidas, filtros incompatibles y acciones no permitidas según el estado de una asignación o tarea.

En conjunto, Postman ha sido la herramienta principal para verificar el contrato real de la API durante el desarrollo, mientras que Swagger ha servido como apoyo para revisar rutas, modelos y disponibilidad de endpoints.

6.3. Pruebas del frontend (web y móvil)

Las pruebas del frontend se han realizado sobre la aplicación desarrollada con **React**, **Vite** y configuración **PWA**. El objetivo principal ha sido comprobar que la interfaz consume correctamente la API, que los flujos principales funcionan de extremo a extremo y que la aplicación resulta usable tanto en escritorio como en móvil.

Como validación técnica básica, durante los bloques de frontend se ha ejecutado de forma recurrente el comando:

```
npm run build
```

Este comando permite comprobar que el proyecto compila correctamente, que no existen errores de importación, que las rutas de assets son válidas y que la aplicación puede generarse para producción. En los últimos bloques se ha mantenido únicamente un warning no bloqueante relacionado con el tamaño de algunos chunks de JavaScript, sin impedir el despliegue ni el funcionamiento de la aplicación.

Las pruebas funcionales de escritorio se han realizado directamente sobre la interfaz web. Se han validado flujos como registro, login, mantenimiento de sesión mediante refresh token, cierre de sesión, navegación por el dashboard, carga de tareas reales, creación, edición, completado y borrado de tareas, gestión de categorías, gestión de grupos, unión por código, visualización de miembros, creación de asignaciones, validación y reapertura de tareas grupales.

También se ha probado la vista **Inteligente**, comprobando que el frontend consume el endpoint de tareas recomendadas y muestra las tarjetas en el orden devuelto por el backend. En este caso, el frontend no calcula la puntuación interna, sino que representa el resultado recibido desde la API.

El **filtro avanzado** se ha probado aplicando distintos criterios desde la pantalla principal: origen, grupo, categoría, prioridades múltiples, estados múltiples, palabras clave, tiempo máximo, fecha exacta, fecha hasta y ordenación. También se ha comprobado el guardado automático del estado del filtro al aplicar o limpiar filtros, la restauración de controles al recargar la aplicación y la navegación entre resultados filtrados y vistas normales.

En cuanto al sistema de **notificaciones**, se ha probado la campanita interna, el contador de notificaciones pendientes, el cierre individual, el cierre de todas las notificaciones y el panel de preferencias. También se han probado avisos 24 horas antes del vencimiento, recordatorio inteligente por tarea, notificaciones por asignaciones de grupo y actualización visual del contador.

La parte **Web Push** se ha validado desde navegador compatible, activando permisos de notificación, registrando la suscripción push desde el frontend y comprobando que el

backend podía enviar notificaciones del sistema. También se ha verificado que, aunque Web Push falle o no esté disponible, la notificación interna siga mostrándose en la campanita.

La aplicación se ha probado además como **PWA instalada**, especialmente tras configurar el service worker propio y el manifest. Se ha comprobado que la aplicación puede instalarse, abrirse desde el icono correspondiente y mantener el flujo normal de login, dashboard y notificaciones.

En móvil se ha realizado un bloque específico de pruebas responsive. Se han revisado pantallas y resoluciones aproximadas como 360x800, 390x844 y 430x932. Las pruebas se han centrado en comprobar que no aparezca scroll horizontal, que las tarjetas se lean correctamente, que los botones sean cómodos de pulsar, que los modales se adapten al ancho disponible y que los paneles principales no se salgan de la pantalla.

Entre los elementos revisados en móvil se encuentran las tarjetas de tareas, categorías y grupos, el panel flotante de vistas, el filtro avanzado compacto, la campanita de notificaciones en formato modal, los modales de asignaciones, las pantallas de login y registro, la vista Inteligente y las acciones principales sobre tareas.

También se ha prestado atención al rendimiento visual del scroll en móvil. Para reducir el coste de pintado en listas largas, los fondos repetidos de tarjetas se sustituyeron por imágenes **WebP** en móvil, manteniendo los SVG decorativos en escritorio. Tras este cambio se comprobó que las tarjetas siguieran viéndose correctamente en tareas, categorías y grupos.

En conjunto, las pruebas del frontend han combinado build de producción, pruebas manuales de interfaz, validación responsive, uso como PWA instalada y pruebas reales de comunicación con el backend desplegado.

6.4. Resultados de las pruebas

Los resultados de las pruebas permiten considerar que la versión actual de **Gestor de Tareas** es funcional para el alcance definido en este proyecto. Los flujos principales del sistema han sido comprobados en backend, API, frontend, PWA y entorno desplegado, incluyendo autenticación, tareas, categorías, grupos, asignaciones, filtros avanzados, ordenación inteligente, notificaciones y Web Push.

En el backend, las validaciones de compilación mediante Maven han confirmado que el código se integra correctamente y que los últimos bloques funcionales no introducen errores de compilación. En los bloques más relevantes se ha ejecutado `.\mvnw.cmd -`

`DskipTests compile` y, cuando ha sido necesario validar el empaquetado, `.\mvnw.cmd -DskipTests clean package`.

La API ha sido validada de forma exhaustiva mediante **Postman**, probando tanto respuestas correctas como errores controlados. Estas pruebas han permitido verificar que los endpoints protegidos requieren JWT, que el refresh token permite renovar la sesión, que las operaciones respetan ownership y permisos, y que los errores se devuelven con códigos HTTP y cuerpos coherentes.

Los flujos colaborativos también han sido probados con distintos usuarios, comprobando creación de grupos, unión por código, gestión de miembros, roles, asignaciones, entrega de tareas, validación y reapertura. Estas pruebas han permitido comprobar que el sistema distingue correctamente entre creador, admin y miembro dentro de cada grupo.

El filtro avanzado y la ordenación inteligente han mostrado un comportamiento coherente con lo esperado. El filtro combinado permite mezclar tareas personales y grupales, aplicar múltiples criterios y guardar el estado del filtro. La vista Inteligente devuelve tareas recomendadas desde el backend sin exponer la puntuación interna, respetando las reglas definidas por el algoritmo.

El sistema de notificaciones se ha validado tanto desde backend como desde frontend. Se ha comprobado la generación de notificaciones internas, el contador de la campanita, el cierre individual, el cierre global, las preferencias de notificación, los avisos 24 horas antes del vencimiento y el recordatorio inteligente por tarea. También se ha probado el envío Web Push real en navegador compatible, confirmando que la notificación interna sigue siendo la fuente principal aunque el envío push pueda fallar o no estar disponible.

En el frontend, los builds de producción con `npm run build` se han completado correctamente. El warning de Vite relacionado con chunks JavaScript grandes se mantiene como aviso no bloqueante, pero no impide compilar ni desplegar la aplicación. La interfaz se ha probado manualmente en escritorio y móvil, revisando navegación, tarjetas, formularios, modales, filtros, grupos, notificaciones, PWA instalada y comportamiento responsive.

En producción, se ha comprobado el funcionamiento conjunto del sistema desplegado: frontend en Vercel, backend en Render y base de datos PostgreSQL en Neon. También se ha validado el endpoint `GET /api/health`, el self-ping del backend, la configuración CORS, la zona horaria Europe/Madrid y la conexión real entre frontend, backend y base de datos.

Como limitación importante, la batería de tests automáticos creada durante fases anteriores del proyecto necesita actualización. Existen tests unitarios, de controladores, repositorios y flujo que fueron útiles en etapas anteriores, pero parte de ellos ha quedado desfasada por la evolución del backend. Por ello, no se considera correcto presentar la suite automática completa como validación final vigente de la versión actual.

La principal mejora futura en materia de pruebas será sanear esa batería automática: eliminar o actualizar tests legacy, adaptar los casos a los contratos actuales y añadir nuevas pruebas para refresh token, grupos, asignaciones, filtro avanzado, ordenación inteligente, notificaciones y Web Push. Aun así, para la entrega actual, los flujos principales han sido comprobados de forma práctica mediante Postman, frontend, builds y pruebas en producción.

En conjunto, las pruebas realizadas permiten afirmar que la aplicación se encuentra en un estado funcional y demostrable para su entrega, con una base sólida de backend, una interfaz web/PWA operativa y un despliegue completo en la nube.

7. Despliegue

El proyecto **Gestor de Tareas** se ha preparado para funcionar tanto en entorno local de desarrollo como en un entorno desplegado en la nube. La arquitectura final separa frontend, backend y base de datos en servicios distintos, lo que permite trabajar de forma más parecida a un despliegue real de una aplicación full stack.

En local, el backend se ejecuta como una aplicación **Spring Boot**, el frontend se levanta con **Vite** y la base de datos se gestiona mediante **PostgreSQL**. En producción, el frontend se despliega en **Vercel**, el backend en **Render** y la base de datos en **Neon**. Esta separación facilita actualizar cada parte de forma independiente y permite que la API pueda ser consumida por distintos clientes en el futuro.

7.1. Entorno de desarrollo local

En el entorno de desarrollo local, el backend de Spring Boot se ejecuta desde el IDE o mediante Maven. Durante el desarrollo se ha trabajado principalmente con una configuración local que permite arrancar el servidor en <http://localhost:8080> y probar la API directamente mediante Postman o Swagger.

La configuración local del backend se define mediante archivos de propiedades y variables de entorno. En el repositorio se mantiene un archivo de ejemplo para facilitar que otra persona pueda crear su propia configuración sin subir credenciales reales. El archivo real de configuración local no debe versionarse si contiene usuarios, contraseñas, secretos JWT, claves VAPID u otros datos sensibles.

La base de datos utilizada actualmente es **PostgreSQL**. En desarrollo puede ejecutarse una instancia local o conectarse a una base de datos remota, siempre que el backend tenga configurada correctamente la URL, usuario, contraseña y driver correspondiente. Para revisar tablas, datos y relaciones se ha utilizado una herramienta gráfica como **DBeaver**, que facilita inspeccionar el esquema y comprobar los cambios generados durante el desarrollo.

El frontend se ejecuta en local desde la carpeta frontend-v2 mediante Vite. El servidor de desarrollo suele levantarse en <http://localhost:5173>, permitiendo probar la interfaz con recarga rápida mientras se realizan cambios. Para que el frontend pueda comunicarse con el backend local, la configuración CORS del backend permite este origen durante el desarrollo.

Las variables propias del frontend, como la clave pública de Web Push, se gestionan mediante archivos .env locales y archivos .env.example de plantilla. La clave pública VAPID puede estar disponible en el frontend, pero la clave privada debe permanecer exclusivamente en el backend y no debe subirse al repositorio.

Durante el desarrollo se han utilizado también herramientas de apoyo como **Postman** para probar la API, **Swagger UI** para consultar documentación interactiva de endpoints y **DBeaver** para revisar la base de datos. Estas herramientas han sido especialmente importantes para validar autenticación, filtros, grupos, asignaciones, notificaciones y configuración de despliegue antes de publicar cambios en producción.

El flujo habitual de validación local consiste en arrancar el backend, arrancar el frontend, comprobar la conexión con la base de datos, probar endpoints críticos con Postman cuando corresponde y ejecutar los comandos de compilación o build antes de cerrar cada bloque de trabajo.

7.2. Despliegue en la nube

El despliegue en producción se ha organizado separando las tres partes principales del sistema: frontend, backend y base de datos. Esta separación permite que cada componente pueda actualizarse, configurarse y escalarse de forma independiente.

El **frontend** se despliega en **Vercel**. Esta plataforma se encarga de construir la aplicación React/Vite mediante `npm run build` y publicar el contenido generado en la carpeta `dist`. Vercel proporciona HTTPS, distribución mediante CDN y despliegue automático a partir del repositorio. La aplicación queda disponible como web y también como PWA instalable desde navegadores compatibles.

El **backend** se despliega en **Render** como servicio Docker. Para ello, el proyecto incluye la configuración necesaria para construir la aplicación Spring Boot y ejecutarla como servicio web. Render expone públicamente la API REST y permite configurar variables de entorno para separar los datos sensibles del código fuente.

La **base de datos** se aloja en **Neon**, utilizando PostgreSQL como sistema de gestión relacional. El backend se conecta a esta base de datos mediante la URL de conexión y credenciales configuradas en producción. Esta base de datos contiene el modelo completo de la aplicación: usuarios, tareas, categorías, grupos, asignaciones, filtros guardados, notificaciones, recordatorios, preferencias y suscripciones push.

La comunicación entre frontend y backend se realiza mediante **HTTPS**. El frontend envía peticiones HTTP en formato JSON al backend y, en las operaciones protegidas, incluye el access token JWT en la cabecera Authorization con formato Bearer `<token>`. La configuración de CORS del backend permite el origen de producción del frontend desplegado en Vercel y también el entorno local usado durante el desarrollo.

El backend dispone de un endpoint público de disponibilidad:

`GET /api/health`

Este endpoint permite comprobar que el servicio está levantado sin requerir autenticación ni exponer información sensible. En producción se utiliza además como base del sistema opcional de self-ping, que realiza llamadas periódicas al propio backend para reducir el impacto del reposo automático del servicio en Render durante pruebas y demostraciones.

Para las notificaciones del dispositivo, el despliegue incluye configuración de **Web Push** mediante claves VAPID. La clave pública se configura tanto en backend como en frontend, mientras que la clave privada se mantiene exclusivamente en Render como variable de entorno. De esta forma, el frontend puede crear suscripciones push y el backend puede enviar notificaciones Web Push sin exponer secretos en el cliente ni en el repositorio.

También se ha configurado explícitamente la zona horaria del backend en producción como **Europe/Madrid**. Esta configuración evita desfases en fechas de creación,

vencimientos, avisos 24 horas antes y recordatorios inteligentes, ya que Render puede trabajar por defecto con una zona horaria distinta.

En producción, la configuración de Hibernate se mantiene con `SPRING_JPA_HIBERNATE_DDL_AUTO=validate`. Esto significa que el backend valida que el esquema de base de datos sea compatible, pero no modifica automáticamente las tablas. Durante fases concretas de creación de nuevas tablas se puede usar temporalmente `update`, pero el estado final recomendado para producción es mantener `validate` y, en una versión más profesional, gestionar cambios de esquema mediante migraciones controladas.

7.3. Variables de entorno y configuración

La configuración del proyecto se separa entre entorno local y entorno de producción. El objetivo es evitar que datos sensibles, como contraseñas, secretos JWT o claves privadas VAPID, queden escritos directamente en el código fuente o subidos al repositorio.

En el backend, las variables de entorno se utilizan para configurar la conexión a base de datos, la seguridad JWT, el perfil activo de Spring, la estrategia de generación del esquema, Web Push, zona horaria y self-ping. En producción, estas variables se configuran desde el panel de Render.

Las principales variables utilizadas por el backend son:

- **DB_URL:** URL de conexión JDBC a la base de datos PostgreSQL.
- **DB_USER:** usuario de conexión a la base de datos.
- **DB_PASS:** contraseña del usuario de base de datos.
- **JWT_SECRET:** secreto utilizado para firmar y validar los tokens JWT.
- **SPRING_PROFILES_ACTIVE:** perfil activo de Spring. En producción se utiliza el perfil prod.
- **SPRING_JPA_HIBERNATE_DDL_AUTO:** modo de gestión del esquema de Hibernate. En producción se recomienda `validate`.
- **PORT:** puerto usado por Render para exponer el servicio.
- **TZ:** zona horaria del entorno. En producción se utiliza Europe/Madrid.
- **JAVA_TOOL_OPTIONS:** opciones adicionales de la JVM. Se utiliza para fijar la zona horaria de Java con `-Duser.timezone=Europe/Madrid`.

Para el sistema de Web Push se utilizan estas variables:

- **APP_WEBPUSH_ENABLED:** activa o desactiva el envío Web Push desde backend.
- **APP_WEBPUSH_VAPID_PUBLIC_KEY:** clave pública VAPID. Esta clave también debe estar disponible en el frontend.
- **APP_WEBPUSH_VAPID_PRIVATE_KEY:** clave privada VAPID. Solo debe existir en backend y no debe exponerse al cliente.
- **APP_WEBPUSH_VAPID_SUBJECT:** identificador de contacto usado en la configuración VAPID, normalmente con formato `mailto:....`.
- **APP_WEBPUSH_DEFAULT_URL:** ruta que se abre o enfoca cuando el usuario pulsa una notificación Web Push.
- Para el sistema opcional de self-ping se utilizan estas variables:
- **APP_SELF_PING_ENABLED:** activa o desactiva el self-ping del backend.
- **APP_SELF_PING_URL:** URL del endpoint de health check que se llama periódicamente.
- **APP_SELF_PING_INITIAL_DELAY_MS:** tiempo inicial de espera antes de iniciar el self-ping.
- **APP_SELF_PING_FIXED_DELAY_MS:** intervalo entre ejecuciones del self-ping.

En el frontend, las variables de entorno se configuran mediante archivos `.env` locales y variables propias del despliegue en Vercel. La variable más relevante para Web Push es la clave pública VAPID, que permite al navegador crear la suscripción push. Al tratarse de una clave pública, puede estar disponible en el cliente, a diferencia de la clave privada VAPID, que debe permanecer únicamente en el backend.

El repositorio incluye archivos de ejemplo como `application-example.properties` o `.env.example` para documentar qué valores deben configurarse sin publicar credenciales reales. Los archivos locales con secretos reales deben mantenerse fuera de Git mediante `.gitignore`.

La configuración de zona horaria es especialmente importante en este proyecto. El backend trabaja con fechas de creación, fechas de entrega, avisos 24 horas antes y recordatorios inteligentes. Por ello, en producción se configura tanto `TZ=Europe/Madrid` como `JAVA_TOOL_OPTIONS=-Duser.timezone=Europe/Madrid`, evitando desfases entre la hora del servidor cloud y la hora esperada por el usuario.

En conjunto, esta separación de configuración permite desplegar la misma aplicación en distintos entornos sin modificar el código fuente, manteniendo los secretos protegidos y facilitando la reproducción del proyecto en local o en producción.

8. Manual de usuario

Este apartado describe el uso básico de la aplicación **Gestor de Tareas** desde el punto de vista del usuario final. La aplicación puede utilizarse desde un navegador web o instalada como PWA en dispositivos compatibles

8.1. Primeros pasos

Al acceder a la aplicación, el usuario encuentra las pantallas de **inicio de sesión** y **registro**. Si ya dispone de una cuenta, puede iniciar sesión introduciendo su correo electrónico y contraseña. Si todavía no tiene cuenta, puede acceder al formulario de registro y crear una nueva indicando su nombre, email y contraseña.

Tras iniciar sesión correctamente, el usuario accede al dashboard principal de la aplicación. Desde esta pantalla puede consultar sus tareas, categorías, grupos, vista inteligente, filtros avanzados y notificaciones. La sesión se mantiene mediante tokens, por lo que no es necesario iniciar sesión continuamente mientras el refresh token siga siendo válido.

En la zona principal de la aplicación aparece la navegación entre vistas. El usuario puede acceder a **Mis tareas**, **Mis categorías**, **Mis grupos** y **Vista inteligente**. También dispone de una campanita de notificaciones y de un filtro avanzado de tareas.

Para cerrar sesión, el usuario puede utilizar la opción correspondiente del panel de vistas. Al hacerlo, la sesión local se elimina y la aplicación vuelve a la pantalla de login.

8.2. Gestión de tareas

La vista **Mis tareas** muestra las tareas del usuario autenticado. Cada tarea aparece representada mediante una tarjeta visual que resume la información principal: título, prioridad, estado, tiempo estimado, fecha de entrega, categoría y origen de la tarea.

El usuario puede crear una nueva tarea mediante el botón **Nueva tarea**. El formulario permite indicar título, descripción, prioridad, tiempo estimado, categoría y fecha de entrega. Algunos campos son obligatorios para garantizar que la tarea tenga información mínima suficiente.

Una vez creada una tarea, puede editarse desde su propia tarjeta. La edición permite modificar sus datos principales, como título, descripción, prioridad, tiempo, categoría o fecha de entrega. También puede eliminarse si el usuario ya no la necesita.

Las tareas pueden marcarse como completadas. Cuando se completa una tarea, el sistema registra la fecha de finalización y actualiza su estado. Si la tarea se completa después de la fecha de entrega, el sistema puede mostrarla como completada con retraso.

Las tarjetas de tarea pueden desplegarse para mostrar más información. Desde la vista desplegada se accede a acciones adicionales, como editar, borrar, completar o activar un recordatorio inteligente cuando la tarea cumple las condiciones necesarias.

La vista **Inteligente** muestra una lista recomendada de tareas. Esta vista utiliza la ordenación inteligente calculada por el backend para priorizar tareas según fecha, prioridad, duración, antigüedad y otros factores. El usuario no necesita configurar nada para usarla: basta con acceder a la vista y revisar el orden sugerido.

En la sección **Mis grupos**, el usuario puede trabajar con tareas asignadas desde grupos. Si pertenece a un grupo, puede recibir tareas generadas por asignaciones grupales. Estas tareas aparecen en su lista y pueden entregarse o completarse según el flujo definido. Los administradores de grupo pueden consultar asignaciones, revisar entregas, validarlas o reabrir las con comentario.

8.3. Categorías y filtros

La vista **Mis categorías** permite organizar las tareas mediante categorías. Cada usuario dispone de categorías base protegidas creadas automáticamente y también puede crear categorías personalizadas. Las categorías incluyen nombre, color e icono, lo que facilita reconocerlas visualmente en las tarjetas de tareas y en los filtros.

Las categorías base protegidas no pueden eliminarse como una categoría normal. Las categorías personalizadas sí pueden editarse o eliminarse. Cuando se elimina una categoría personalizada, las tareas asociadas no se borran, sino que quedan sin categoría.

El filtro avanzado permite consultar tareas aplicando varios criterios a la vez. Desde el panel de filtros, el usuario puede seleccionar origen, grupo, categoría, prioridades, estados, palabras clave, tiempo máximo, fecha exacta, fecha hasta y criterio de ordenación.

Al aplicar filtros, la aplicación muestra los resultados filtrados como una vista propia dentro del dashboard. Si el usuario limpia los filtros, la aplicación vuelve a la vista de Mis tareas. El estado del filtro se guarda automáticamente en servidor, por lo que puede recuperarse al volver a entrar en la aplicación.

Además del filtro avanzado, la pantalla principal incluye un buscador rápido local. Este buscador permite reducir rápidamente las tareas visibles en la vista actual sin realizar una nueva consulta al backend. Es útil para localizar tareas concretas por texto, categoría, estado, prioridad o grupo.

8.4. Uso en móvil / PWA

La aplicación puede utilizarse desde un navegador móvil y también puede instalarse como **PWA** en dispositivos compatibles. Al instalarla, el usuario puede abrirla desde un icono propio, de forma similar a una aplicación instalada.

En móvil, la interfaz se adapta al tamaño de pantalla. Las tarjetas ocupan menos espacio horizontal, los modales se ajustan al ancho disponible y el panel de vistas se muestra como un elemento flotante para no ocupar espacio fijo en la parte superior. El filtro avanzado también se compacta para que pueda utilizarse de forma cómoda en pantallas pequeñas.

La campanita de notificaciones se muestra en móvil como un panel tipo modal. Desde ella el usuario puede consultar notificaciones activas, cerrarlas y acceder a preferencias. También puede activar o desactivar avisos 24 horas antes, recordatorio inteligente, notificaciones internas, avisos de asignaciones de grupo y notificaciones del dispositivo cuando el navegador lo permite.

Para recibir notificaciones del sistema, el usuario debe activar las notificaciones del dispositivo y aceptar el permiso del navegador. Si el navegador es compatible, la aplicación registra una suscripción Web Push asociada al usuario. A partir de ese momento, determinadas notificaciones internas pueden mostrarse también como notificaciones del sistema operativo o del navegador.

Aunque la aplicación puede instalarse como PWA, también sigue funcionando correctamente como web normal desde el navegador. Si el dispositivo o navegador no permite instalación o notificaciones push, el usuario puede seguir utilizando las funcionalidades principales desde la interfaz web.

9. Conclusiones y trabajo futuro

El desarrollo de **Gestor de Tareas** ha permitido construir una aplicación full stack funcional, desplegada en la nube y organizada alrededor de una arquitectura cliente-servidor. El proyecto ha evolucionado desde una idea inicial centrada en la gestión

individual de tareas hasta una solución más completa que incluye autenticación, categorías, grupos, asignaciones colaborativas, filtros avanzados, ordenación inteligente, notificaciones, PWA y adaptación móvil.

Uno de los principales aprendizajes del proyecto ha sido la importancia de separar responsabilidades. En el backend, la estructura por capas ha permitido mantener diferenciados los controladores, servicios, repositorios, modelos, DTOs, seguridad, configuración y excepciones. Esta separación ha sido clave para poder incorporar nuevas funcionalidades sin convertir el código en una única pieza difícil de mantener.

A nivel de backend, el proyecto ha supuesto un salto importante respecto a una aplicación Java clásica. El uso de **Spring Boot**, **Spring Security**, **JPA/Hibernate**, **PostgreSQL** y una API REST protegida mediante **JWT** ha permitido trabajar con una estructura mucho más cercana a una aplicación real. Además, la incorporación de refresh token, ownership de recursos, roles dentro de grupos, manejo global de errores y despliegue cloud ha añadido una dimensión más profesional al sistema.

La parte colaborativa también ha sido una de las ampliaciones más relevantes. El sistema de grupos, miembros, roles y asignaciones ha permitido convertir la aplicación en algo más que un gestor personal. Aunque el modelo actual genera tareas individuales para cada destinatario, esta decisión ha facilitado mantener la trazabilidad de entregas, validaciones y reaperturas sin complicar excesivamente el diseño de datos.

Otro elemento importante ha sido la implementación del **filtro avanzado** y de la **ordenación inteligente**. El filtro avanzado permite consultar tareas personales y grupales desde una misma pantalla, combinando criterios como origen, grupo, categoría, prioridad, estado, fechas, palabras clave y ordenación. Por su parte, la ordenación inteligente aporta una vista recomendada basada en una heurística explicable, sin recurrir a inteligencia artificial, pero combinando factores útiles como fecha, prioridad, tiempo estimado, antigüedad y contexto grupal.

El sistema de **notificaciones** ha añadido automatización al proyecto. La aplicación no se limita a mostrar datos cuando el usuario los consulta, sino que puede generar avisos internos, recordatorios inteligentes, avisos 24 horas antes del vencimiento y notificaciones por nuevas asignaciones de grupo. Además, la integración con **Web Push** permite que ciertas notificaciones se muestren también como avisos del navegador o del sistema cuando el usuario ha dado permiso y el dispositivo lo soporta.

En el frontend, el proyecto ha permitido construir una interfaz web completa con **React**, **Vite** y configuración **PWA**. La aplicación incluye autenticación real, dashboard, tarjetas de tareas, categorías, grupos, asignaciones, filtro avanzado, vista inteligente, notificaciones y formularios modales. También se ha trabajado la adaptación responsive para móvil, ajustando tarjetas, paneles, filtros, modales, login, registro y fondos visuales para mejorar el uso en pantallas pequeñas.

El despliegue final en **Vercel**, **Render** y **Neon** ha sido otro punto clave. Gracias a esta separación, el frontend, backend y base de datos funcionan como servicios independientes. También se han configurado variables de entorno, CORS, zona horaria, Web Push, health check y self-ping, acercando el proyecto a un escenario real de publicación y mantenimiento de una aplicación web.

Durante el desarrollo también se han encontrado limitaciones. La batería de tests automáticos creada en fases anteriores quedó parcialmente desactualizada debido a la evolución del backend. Aunque la aplicación se ha probado de forma exhaustiva mediante Postman, pruebas manuales, builds y validación en producción, una mejora importante sería actualizar esos tests para que vuelvan a reflejar el contrato actual de la API.

En conjunto, el proyecto cumple el objetivo principal de ofrecer una aplicación funcional para gestionar tareas personales y colaborativas. Además, ha servido como base de aprendizaje en desarrollo backend, frontend, seguridad, acceso a datos, despliegue, documentación técnica y mantenimiento incremental de una aplicación en crecimiento. La versión actual no debe entenderse como un producto cerrado definitivamente, sino como una base sólida sobre la que se pueden construir nuevas funcionalidades en futuras iteraciones.

9.1. Funcionalidades futuras

Aunque la versión actual de Gestor de Tareas ya cubre el flujo principal de uso, el proyecto queda preparado para seguir creciendo en futuras iteraciones. Las siguientes mejoras se plantean como evolución natural de la aplicación, priorizando primero seguridad, gestión de cuenta y consolidación del sistema antes de añadir funcionalidades más complejas.

Gestión de perfil, cuenta y preferencias

Una de las líneas futuras más importantes es ampliar la gestión del usuario más allá del registro, login y eliminación de cuenta. La aplicación podría incorporar una sección completa de perfil y ajustes de cuenta.

Funcionalidades previstas:

- Perfil de usuario.
- Edición del nombre visible.
- Cambio de correo electrónico.
- Cambio de contraseña.
- Consulta del estado de la cuenta: activa, verificada o pendiente de verificación.

- Ajustes generales de cuenta.
- Eliminación de cuenta más controlada, con confirmaciones y avisos previos.
- Preferencias visuales de la aplicación.
- Preferencias de experiencia de usuario.
- Gestión más avanzada de preferencias de notificación.

Esta sección permitiría que el usuario tuviera mayor control sobre su cuenta y sobre cómo quiere utilizar la aplicación, sin depender únicamente de configuraciones internas o valores por defecto.

Seguridad de registro y verificación de cuenta

Otra línea prioritaria es reforzar la seguridad del registro y la activación de cuentas. Actualmente el sistema permite registrar usuarios e iniciar sesión, pero todavía no incorpora verificación real mediante correo electrónico ni captcha.

Funcionalidades previstas:

- Captcha en el registro de usuarios.
- Validación de cuenta mediante correo electrónico.
- Reenvío de correo de verificación.
- Estado visible de cuenta verificada o no verificada.
- Bloqueo de determinadas acciones grupales si el email no está verificado.

La idea no es impedir el uso individual básico de la aplicación, sino endurecer progresivamente las acciones colaborativas que pueden afectar a otros usuarios, como crear grupos, invitar miembros o asignar tareas.

Mejoras del sistema colaborativo

El sistema de grupos y asignaciones ya permite crear grupos, gestionar miembros, asignar tareas, entregar, validar y reabrir entregas. Aun así, existen mejoras posibles para hacerlo más completo.

Funcionalidades previstas:

- Comentarios más completos en tareas de grupo.
- Respuestas del miembro al comentario de revisión.
- Histórico completo de revisiones.
- Mayor trazabilidad de acciones compartidas.
- Registro más detallado de quién realiza cada acción.
- Tareas grupales compartidas reales, sin duplicación individual.

La versión actual utiliza duplicación individual de tareas para simplificar el control de cada miembro. En una evolución futura, podría estudiarse un modelo de tarea

compartida real, aunque implicaría mayor complejidad en permisos, estados y concurrencia.

Consulta avanzada y organización de tareas

El filtro avanzado y la ordenación inteligente ya están implementados, pero pueden seguir evolucionando según el uso real de la aplicación.

Funcionalidades previstas:

- Paginación avanzada de listados.
- Carga incremental cuando haya muchas tareas.
- Refinamiento del filtro combinado si aparecen nuevas necesidades reales.
- Filtro por estado de revisión grupal.
- Búsqueda más avanzada por varias palabras clave.
- Adaptar la ordenación inteligente al tiempo disponible diario del usuario.
- Ajustar pesos del algoritmo según pruebas de uso acumuladas.

Estas mejoras permitirían que la aplicación se mantuviera cómoda aunque aumentara el volumen de tareas, grupos y asignaciones.

Estadísticas y visualización

La aplicación podría incorporar una sección de análisis para que el usuario conozca mejor su actividad y evolución.

Funcionalidades previstas:

- Estadísticas de tareas creadas.
- Estadísticas de tareas completadas.
- Estadísticas de tareas vencidas.
- Distribución por categorías.
- Distribución por prioridades.
- Evolución por semanas o meses.
- Estadísticas por grupo.
- Progreso de miembros dentro de un grupo.
- Vista calendario mensual.
- Días coloreados según prioridad, estado o carga de tareas.

Estas funciones aportarían una capa de análisis útil para detectar patrones, revisar productividad y entender mejor cómo se están gestionando las tareas.

Exportación e importación de datos

Otra ampliación posible es permitir al usuario sacar o cargar información desde formatos externos.

Funcionalidades previstas:

- Exportación de tareas a CSV.
- Exportación de tareas a PDF.
- Importación de tareas desde CSV.
- Exportación de estadísticas o informes.
- Generación de informes por periodo, categoría o grupo.

Esta línea tendría utilidad tanto práctica como documental, especialmente si el usuario quiere conservar un registro externo de su actividad o compartir información fuera de la aplicación.

Integraciones externas

La integración con servicios externos queda como una línea futura interesante, pero no se considera imprescindible para la versión actual.

Funcionalidades previstas:

- Login o registro con Google.
- Vinculación de cuenta con Google.
- Integración con Google Calendar.
- Creación de eventos de calendario a partir de tareas.
- Sincronización básica de fechas de entrega.

La integración con Google Calendar requeriría gestionar permisos adicionales, tokens externos y sincronización con un servicio de terceros, por lo que se plantea como una mejora posterior y no como parte del alcance actual.

Aplicación móvil específica

La versión actual funciona como PWA responsive, pero en el futuro podría desarrollarse una aplicación móvil específica reutilizando la misma API REST.

Funcionalidades previstas:

- App móvil con React Native u otra tecnología móvil.
- Mejor integración con notificaciones del dispositivo.
- Experiencia móvil más cercana a una aplicación nativa.
- Uso más cómodo en Android.
- Posible aprovechamiento de funciones propias del dispositivo.

Esta evolución permitiría separar mejor la experiencia móvil de la web, aunque implicaría mantener un nuevo cliente además del frontend actual.

Mejoras técnicas y de mantenimiento

También quedan mejoras internas importantes para hacer el proyecto más robusto y mantenible a largo plazo.

Funcionalidades previstas:

- Actualizar y sanear la batería de tests automáticos.
- Añadir más tests para seguridad, grupos, filtros, notificaciones y Web Push.
- Mejorar la cobertura de pruebas de frontend.
- Incorporar migraciones versionadas con Flyway o Liquibase.
- Añadir monitorización y logs más avanzados.
- Definir una estrategia de copias de seguridad.
- Optimizar chunks del frontend si el tamaño crece demasiado.
- Añadir índices o restricciones adicionales en base de datos si el volumen aumenta.
- Mejorar documentación README, ejemplos de request/response y colección Postman.

Estas mejoras no cambian directamente lo que ve el usuario, pero ayudarían a mantener la aplicación con mayor calidad técnica si el proyecto siguiera creciendo.

En conjunto, las funcionalidades futuras se orientan a reforzar la seguridad, mejorar la gestión de cuenta, ampliar la experiencia colaborativa, añadir análisis de datos, integrar servicios externos y consolidar la calidad técnica del proyecto. La versión actual deja una base suficientemente sólida para abordar estas ampliaciones de forma incremental.

10. Anexos

10.1. Documentación interactiva de la API

La API REST del proyecto dispone de documentación interactiva generada mediante **Swagger/OpenAPI**. Esta documentación permite consultar los endpoints disponibles, los métodos HTTP utilizados, los parámetros requeridos, los cuerpos de petición, los modelos de respuesta y los posibles códigos de estado.

La documentación online de la API desplegada puede consultarse en la siguiente dirección:

<https://gestor-backend-tnsg.onrender.com/swagger-ui/index.html#/>

Desde esta interfaz es posible revisar los distintos bloques funcionales de la API, como usuarios, tareas, categorías, grupos, asignaciones, filtro combinado, tareas recomendadas, notificaciones, suscripciones push y health check.

Swagger ha sido utilizado durante el desarrollo como apoyo para comprobar la estructura de la API y revisar rápidamente los contratos disponibles. Para las pruebas funcionales más completas se ha utilizado principalmente Postman, ya que permite guardar colecciones, repetir escenarios y validar flujos completos con autenticación JWT.

10.2. Listado completo de endpoints

A continuación se recoge el inventario de endpoints principales expuestos por la API REST del proyecto.

/api/tarea — TareaController

- PUT /api/tarea/update/{id} — Modificar una tarea existente
- POST /api/tarea/add — Añadir una nueva tarea
- PATCH /api/tarea/completar/{id} — Marcar una tarea como completada
- PATCH /api/tarea/{id}/recordatorio-inteligente — Activar o desactivar recordatorio inteligente
- GET /api/tarea — Listar todas las tareas del usuario autenticado
- GET /api/tarea/{id} — Obtener una tarea por ID
- GET /api/tarea/titulo — Listar tareas ordenadas por título
- GET /api/tarea/tiempo — Listar tareas ordenadas por tiempo estimado
- GET /api/tarea/test — Endpoint auxiliar de prueba
- GET /api/tarea/prioridad — Listar tareas ordenadas por prioridad
- GET /api/tarea/fecha — Listar tareas ordenadas por fecha de entrega
- GET /api/tarea/proximas — Listar tareas próximas a vencer
- GET /api/tarea/vencidas — Listar tareas vencidas
- GET /api/tarea/hoy — Listar tareas programadas para hoy
- GET /api/tarea/filtrar/tiempo/{tiempo} — Filtrar tareas por tiempo estimado
- GET /api/tarea/filtrar/prioridad/{prioridad} — Filtrar tareas por prioridad
- GET /api/tarea/filtrar/palabras/{palabrasClave} — Filtrar tareas por palabras clave
- GET /api/tarea/filtrar/estado/{estado} — Filtrar tareas por estado
- GET /api/tarea/filtrar/categoria/{idCategoria} — Filtrar tareas por categoría
- GET /api/tarea/estado/{id} — Obtener estado de una tarea
- GET /api/tarea/asignadas-grupo — Listar tareas asignadas por grupos

- GET /api/tarea/asignadas-grupo/{idGrupo} — Listar tareas asignadas de un grupo
- POST /api/tarea/filtrar-combinado — Aplicar filtros combinados
- POST /api/tarea/recomendadas — Obtener tareas recomendadas
- GET /api/tarea/filtro-combinado/save — Obtener filtro combinado guardado
- PUT /api/tarea/filtro-combinado/save — Guardar filtro combinado
- DELETE /api/tarea/delete/{id} — Eliminar una tarea

/api/grupo — GrupoController

- PUT /api/grupo/update/{id} — Modificar un grupo
- POST /api/grupo/add — Crear un grupo
- POST /api/grupo/join — Unirse a un grupo por código
- POST /api/grupo/{id}/invitation-code/regenerate — Regenerar código de invitación
- POST /api/grupo/{id}/miembros/add — Añadir miembro
- PATCH /api/grupo/active/{id} — Activar o inactivar grupo
- PATCH /api/grupo/{id}/transfer-ownership — Transferir ownership
- PATCH /api/grupo/{id}/miembros/rol/{idUserario} — Cambiar rol de miembro
- GET /api/grupo — Listar grupos del usuario
- GET /api/grupo/{id} — Obtener grupo por ID
- GET /api/grupo/{id}/miembros — Listar miembros
- GET /api/grupo/{id}/invitation-code — Ver código de invitación
- DELETE /api/grupo/{id}/miembros/delete/{idUserario} — Expulsar miembro
- DELETE /api/grupo/{id}/leave — Salir de un grupo
- DELETE /api/grupo/delete/{id} — Eliminar un grupo

/api/categoria — CategoriaController

- PUT /api/categoria/update/{id} — Modificar una categoría
- POST /api/categoria/add — Añadir una categoría
- GET /api/categoria — Listar categorías
- GET /api/categoria/nombre/{nombreParcial} — Buscar categorías por nombre
- DELETE /api/categoria/delete/{id} — Eliminar una categoría

/api/usuario — UsuarioController

- POST /api/usuario/refresh — Renovar access token
- POST /api/usuario/login — Autenticar usuario
- POST /api/usuario/add — Registrar usuario
- GET /api/usuario/me — Obtener cuenta autenticada
- DELETE /api/usuario/me — Eliminar cuenta autenticada

/api/grupo — AsignacionGrupoController

- POST /api/grupo/{id}/asignaciones/add — Crear asignación de grupo
- GET /api/grupo/{id}/asignaciones — Listar asignaciones de grupo
- GET /api/grupo/{id}/asignaciones/{idAsignacion} — Ver detalle de asignación
- PATCH /api/grupo/asignaciones/{idAsignacionGrupoMiembro}/validate — Validar entrega
- PATCH /api/grupo/asignaciones/{idAsignacionGrupoMiembro}/reopen — Reabrir entrega

/api/notificaciones — NotificacionController

- GET /api/notificaciones — Listar notificaciones activas
- GET /api/notificaciones/count — Contar notificaciones activas
- PATCH /api/notificaciones/{id}/cerrar — Cerrar una notificación
- PATCH /api/notificaciones/cerrar-todas — Cerrar todas las notificaciones
- GET /api/notificaciones/preferencias — Obtener preferencias de notificación
- PUT /api/notificaciones/preferencias — Actualizar preferencias de notificación
- POST /api/notificaciones/push-subscripciones — Registrar suscripción push
- DELETE /api/notificaciones/push-subscripciones — Desactivar suscripción push

/api/health — HealthController

- GET /api/health — Comprobar disponibilidad del backend

10.3. Inventario de errores y respuestas DTO

Además de la documentación interactiva de Swagger y del listado de endpoints, el proyecto dispone de un inventario auxiliar de errores visibles y respuestas DTO. Este documento se mantiene como una hoja de cálculo externa para facilitar su consulta, filtrado y actualización.

El inventario público puede consultarse en la siguiente dirección:

https://cesurformacion0-my.sharepoint.com/:x/g/personal/miguel_s440324_cesurformacion_com/IQDtaPoW9SXXQZXfNtU70f_mAX-eAGFzkn6-QrjqNOWV_70?e=g1t1yG&nav=MTVfezAwMDAwMDAwLTAwMDEtMDAwMC0wMTAwLTAwMDAwMDAwMDAwMH0

Este archivo recoge información de apoyo sobre respuestas visibles para el cliente, códigos HTTP, mensajes de error y estructuras DTO utilizadas por la API. Su objetivo es complementar la documentación técnica principal y facilitar la trazabilidad entre endpoints, validaciones y respuestas devueltas por el backend.

Durante la evolución del proyecto, cuando un bloque de backend ha añadido, cambiado o eliminado respuestas de error visibles para el cliente, se ha revisado este inventario para mantenerlo alineado con el contrato real de la API.

Al mantenerse como hoja de cálculo externa, resulta más cómodo consultar, filtrar y actualizar la información que si se incluyera completa dentro de la memoria técnica.

10.4. Esquema físico de la base de datos

Además del diagrama entidad–relación simplificado incluido en el apartado 4.2.1, se conserva como anexo una vista física completa del esquema de base de datos obtenida desde **DBeaver**.

Esta captura representa el estado real del esquema en PostgreSQL, incluyendo las tablas principales del sistema y sus relaciones: usuarios, tareas, categorías, grupos, miembros, asignaciones, estado guardado del filtro, notificaciones, recordatorios, preferencias de notificación, suscripciones push y tablas auxiliares.

A diferencia del diagrama simplificado utilizado en el cuerpo principal de la memoria, esta vista no busca explicar visualmente el modelo de forma didáctica, sino mostrar una representación completa del esquema físico generado en la base de datos. Por ese motivo puede resultar más densa, pero sirve como evidencia técnica del modelo final persistido.

